

The Unstable Book

Welcome to the Unstable Book! This book consists of a number of chapters, each one organized by a "feature flag." That is, when using an unstable feature of Rust, you must use a flag, like this:

```
#![feature(coroutines, coroutine_trait, stmt_expr_attributes)]

use std::ops::{Coroutine, CoroutineState};
use std::pin::Pin;

fn main() {
    let mut coroutine = #[coroutine] || {
        yield 1;
        return "foo"
    };

    match Pin::new(&mut coroutine).resume(()) {
        CoroutineState::Yielded(1) => {}
        _ => panic!("unexpected value from resume"),
    }
    match Pin::new(&mut coroutine).resume(()) {
        CoroutineState::Complete("foo") => {}
        _ => panic!("unexpected value from resume"),
    }
}
```

The `coroutines` feature [has a chapter](#) describing how to use it.

Because this documentation relates to unstable features, we make no guarantees that what is contained here is accurate or up to date. It's developed on a best-effort basis. Each page will have a link to its tracking issue with the latest developments; you might want to check those as well.

Compiler flags

autodiff

The tracking issue for this feature is: [#124509](#).

This feature allows you to differentiate functions using automatic differentiation. Set the `-Zautodiff=<options>` compiler flag to adjust the behaviour of the autodiff feature. Multiple options can be separated with a comma. Valid options are:

`PrintTA` - print Type Analysis Information `PrintAA` - print Activity Analysis Information
`PrintPerf` - print Performance Warnings from Enzyme `Print` - prints all intermediate transformations
`PrintModBefore` - print the whole module, before running opts
`PrintModAfterOpts` - print the whole module just before we pass it to Enzyme
`PrintModAfterEnzyme` - print the module after Enzyme differentiated everything
`LooseTypes` - Enzyme's loose type debug helper (can cause incorrect gradients) `Inline` - runs Enzyme specific Inlining
`NoModOptAfter` - do not optimize the module after Enzyme is done
`EnableFncOpt` - tell Enzyme to run LLVM Opts on each function it generated
`NoVecUnroll` - do not unroll vectorized loops `RuntimeActivity` - allow specifying activity at runtime

branch-protection

The tracking issue for this feature is: [#113369](#).

This option lets you enable branch authentication instructions on AArch64. This option is only accepted when targeting AArch64 architectures. It takes some combination of the following values, separated by a `,`.

- `pac-ret` - Enable pointer authentication for non-leaf functions.
- `pc` - Use PC as a diversifier using PAuthLR instructions
- `leaf` - Enable pointer authentication for all functions, including leaf functions.
- `b-key` - Sign return addresses with key B, instead of the default key A.
- `bti` - Enable branch target identification.

`leaf`, `b-key` and `pc` are only valid if `pac-ret` was previously specified. For example, `-Z branch-protection=bti,pac-ret,leaf` is valid, but `-Z branch-protection=bti,leaf,pac-ret` is not.

Rust's standard library does not ship with BTI or pointer authentication enabled by default. In Cargo projects the standard library can be recompiled with pointer authentication using the nightly `build-std` feature.

cf-protection

The tracking issue for this feature is: [#93754](#).

This option enables control-flow enforcement technology (CET) on x86; a more detailed description of CET is available [here](#). Similar to `clang`, this flag takes one of the following values:

- `none` - Disable CET completely (this is the default).
- `branch` - Enable indirect branch tracking (`IBT`).
- `return` - Enable shadow stack (`SHSTK`).
- `full` - Enable both `branch` and `return` .

This flag only applies to the LLVM backend: it sets the `cf-protection-branch` and `cf-protection-return` flags on LLVM modules. Note, however, that all compiled modules linked together must have the flags set for the compiled output to be CET-enabled. Currently, Rust's standard library does not ship with CET enabled by default, so you may need to rebuild all standard modules with a `cargo` command like:

```
$ RUSTFLAGS="-Z cf-protection=full" cargo +nightly build -Z build-std --target x86_64-unknown-linux-gnu
```

Detection

An ELF binary is CET-enabled if it has the `IBT` and `SHSTK` tags, e.g.:

```
$ readelf -a target/x86_64-unknown-linux-gnu/debug/example | grep feature:
Properties: x86 feature: IBT, SHSTK
```

Troubleshooting

To display modules that are not CET enabled, examine the linker errors available when `cet-report` is enabled:

```
$ RUSTC_LOG=rustc_codegen_ssa::back::link=info rustc-custom -v -Z cf-  
protection=full -C link-arg="-Wl,-z,cet-report=warning" -o example example.rs  
...  
/usr/bin/ld: /.../build/x86_64-unknown-linux-gnu/stage1/lib/rustlib/x86_64-  
unknown-linux-gnu/lib/libstd-d73f7266be14cb8b.rlib(std-  
d73f7266be14cb8b.std.f7443020-cgu.12.rcgu.o): warning: missing IBT and SHSTK  
properties
```

codegen-backend

The tracking issue for this feature is: [#77933](#).

This feature allows you to specify a path to a dynamic library to use as rustc's code generation backend at runtime.

Set the `-Zcodegen-backend=<path>` compiler flag to specify the location of the backend. The library must be of crate type `dylib` and must contain a function named `__rustc_codegen_backend` with a signature of `fn() -> Box<dyn rustc_codegen_ssa::traits::CodegenBackend>`.

Example

See also the [codegen-backend/hotplug](#) test for a working example.

```
use rustc_codegen_ssa::traits::CodegenBackend;

struct MyBackend;

impl CodegenBackend for MyBackend {
    // Implement codegen methods
}

#[no_mangle]
pub fn __rustc_codegen_backend() -> Box<dyn CodegenBackend> {
    Box::new(MyBackend)
}
```

Unstable codegen options

All of these options are passed to `rustc` via the `-C` flag, short for "codegen". The flags are stable but some of their values are individually unstable, and also require using `-Z unstable-options` to be accepted.

linker-flavor

In addition to the stable set of linker flavors, the following unstable values also exist:

- `ptx`: use `rust-ptx-linker` for Nvidia NVPTX GPGPU support.
- `bpf`: use `bpf-linker` for eBPF support.
- `llbc`: for linking in LLVM bitcode. Install the preview rustup components `llvm-bitcode-linker` and `llvm-tools` to use as a self-contained linker by passing `-Zunstable-options -Clink-self-contained=+linker` together with `-Clinker-flavor=llbc`. Can currently only be used for Nvidia NVPTX targets (`nvptx64-nvidia-cuda`).

Additionally, a set of more precise linker flavors also exists, for example allowing targets to declare that they use the LLD linker by default. The following values are currently unstable, and the goal is for them to become stable, and preferred in practice over the existing stable values:

- `gnu`: unix-like linker with GNU extensions
- `gnu-ld`: `gnu` using LLD
- `gnu-cc`: `gnu` using a C/C++ compiler as the linker driver
- `gnu-ld-cc`: `gnu` using LLD and a C/C++ compiler as the linker driver
- `darwin`: unix-like linker for Apple targets
- `darwin-ld`: `darwin` using LLD
- `darwin-cc`: `darwin` using a C/C++ compiler as the linker driver
- `darwin-ld-cc`: `darwin` using LLD and a C/C++ compiler as the linker driver
- `wasm-ld`: unix-like linker for Wasm targets, with LLD
- `wasm-ld-cc`: unix-like linker for Wasm targets, with LLD and a C/C++ compiler as the linker driver
- `unix`: basic unix-like linker for "any other Unix" targets (Solaris/illumos, L4Re, MSP430, etc), not supported with LLD.
- `unix-cc`: `unix` using a C/C++ compiler as the linker driver

- `msvc-lld` : MSVC-style linker for Windows and UEFI, with LLD
- `em-cc` : emscripten compiler frontend, similar to `wasm-ld-cc` with a different interface

link-self-contained

This flag generally controls whether the linker will use libraries and objects shipped with Rust instead of those in the system. The stable boolean values for this flag are coarse-grained (everything or nothing), but there exists a set of unstable values with finer-grained control, `-Clink-self-contained` can accept a comma-separated list of components, individually enabled (`+component`) or disabled (`-component`):

- `crt0` : CRT objects (e.g. on `windows-gnu` , `musl` , `wasi` targets)
- `libc` : libc static library (e.g. on `musl` , `wasi` targets)
- `unwind` : libgcc/libunwind (e.g. on `windows-gnu` , `fuchsia` , `fortanix` , `gnullvm` targets)
- `linker` : linker, dlltool, and their necessary libraries (e.g. on `windows-gnu` and for `rust-ld`)
- `sanitizers` : sanitizer runtime libraries
- `mingw` : other MinGW libs and Windows import libs

Out of the above self-contained linking components, `linker` is the only one currently implemented (beyond parsing the CLI options).

It refers to the LLD linker, built from the same LLVM revision used by rustc (named `rust-ld` to avoid naming conflicts), that is distributed via `rustup` with the compiler (and is used by default for the wasm targets). One can also opt-in to use it by combining this flag with an appropriate linker flavor: for example, `-Clinker-flavor=gnu-ld-cc -Clink-self-contained=+linker` will use the toolchain's `rust-ld` as the linker.

control-flow-guard

The tracking issue for this feature is: [#68793](#).

The rustc flag `-Z control-flow-guard` enables the Windows [Control Flow Guard](#) (CFG) platform security feature.

CFG is an exploit mitigation designed to enforce control-flow integrity for software running on supported [Windows platforms \(Windows 8.1 onwards\)](#). Specifically, CFG uses runtime checks to validate the target address of every indirect call/jump before allowing the call to complete.

During compilation, the compiler identifies all indirect calls/jumps and adds CFG checks. It also emits metadata containing the relative addresses of all address-taken functions. At runtime, if the binary is run on a CFG-aware operating system, the loader uses the CFG metadata to generate a bitmap of the address space and marks those addresses that contain valid targets. On each indirect call, the inserted check determines whether the target address is marked in this bitmap. If the target is not valid, the process is terminated.

In terms of interoperability:

- Code compiled with CFG enabled can be linked with libraries and object files that are not compiled with CFG. In this case, a CFG-aware linker can identify address-taken functions in the non-CFG libraries.
- Libraries compiled with CFG can be linked into non-CFG programs. In this case, the CFG runtime checks in the libraries are not used (i.e. the mitigation is completely disabled).

CFG functionality is completely implemented in the LLVM backend and is supported for X86 (32-bit and 64-bit), ARM, and Aarch64 targets. The rustc flag adds the relevant LLVM module flags to enable the feature. This flag will be ignored for all non-Windows targets.

When to use Control Flow Guard

The primary motivation for enabling CFG in Rust is to enhance security when linking against non-Rust code, especially C/C++ code. To achieve full CFG protection, all indirect calls (including any from Rust code) must have the appropriate CFG checks, as added by

this flag. CFG can also improve security for Rust code that uses the `unsafe` keyword.

Another motivation behind CFG is to harden programs against [return-oriented programming \(ROP\)](#) attacks. CFG disallows an attacker from taking advantage of the program's own instructions while redirecting control flow in unexpected ways.

Overhead of Control Flow Guard

The CFG checks and metadata can potentially increase binary size and runtime overhead. The magnitude of any increase depends on the number and frequency of indirect calls. For example, enabling CFG for the Rust standard library increases binary size by approximately 0.14%. Enabling CFG in the SPEC CPU 2017 Integer Speed benchmark suite (compiled with Clang/LLVM) incurs approximate runtime overheads of between 0% and 8%, with a geometric mean of 2.9%.

Testing Control Flow Guard

The rustc flag `-Z control-flow-guard=nochecks` instructs LLVM to emit the list of valid call targets without inserting runtime checks. This flag should only be used for testing purposes as it does not provide security enforcement.

Control Flow Guard in libraries

It is strongly recommended to also enable CFG checks for all linked libraries, including the standard library.

To enable CFG in the standard library, use the `cargo -Z build-std` functionality to recompile the standard library with the same configuration options as the main program.

For example:

```
rustup toolchain install --force nightly
rustup component add rust-src
SET RUSTFLAGS=-Z control-flow-guard
cargo +nightly build -Z build-std --target x86_64-pc-windows-msvc
```

```
rustup toolchain install --force nightly
rustup component add rust-src
$Env:RUSTFLAGS = "-Z control-flow-guard"
cargo +nightly build -Z build-std --target x86_64-pc-windows-msvc
```

Alternatively, if you are building the standard library from source, you can set `control-flow-guard = true` in the `config.toml` file.

coverage-options

This option controls details of the coverage instrumentation performed by `-C instrument-coverage`.

Multiple options can be passed, separated by commas. Valid options are:

- `block`, `branch`, `condition`, `mcdc`: Sets the level of coverage instrumentation. Setting the level will override any previously-specified level.
 - `block` (default): Blocks in the control-flow graph will be instrumented for coverage.
 - `branch`: In addition to block coverage, also enables branch coverage instrumentation.
 - `condition`: In addition to branch coverage, also instruments some boolean expressions as branches, even if they are not directly used as branch conditions.
 - `mcdc`: In addition to condition coverage, also enables MC/DC instrumentation. (Branch coverage instrumentation may differ in some cases.)

debug-info-for-profiling

Introduction

Automatic Feedback Directed Optimization (AFDO) is a method for using sampling based profiles to guide optimizations. This is contrasted with other methods of FDO or profile-guided optimization (PGO) which use instrumented profiling.

Unlike PGO (controlled by the `rustc` flags `-Cprofile-generate` and `-Cprofile-use`), a binary being profiled does not perform significantly worse, and thus it's possible to profile binaries used in real workflows and not necessary to construct artificial workflows.

Use

In order to use AFDO, the target platform must be Linux running on an `x86_64` architecture with the performance profiler `perf` available. In addition, the external tool `create_llvm_prof` from [this repository](#) must be used.

Given a Rust file `main.rs`, we can produce an optimized binary as follows:

```
rustc -O -Zdebug-info-for-profiling main.rs -o main
perf record -b ./main
create_llvm_prof --binary=main --out=code.prof
rustc -O -Zprofile-sample-use=code.prof main.rs -o main2
```

The `perf` command produces a profile `perf.data`, which is then used by the `create_llvm_prof` command to create `code.prof`. This final profile is then used by `rustc` to guide optimizations in producing the binary `main2`.

debug-info-type-line-numbers

This option causes additional type and line information to be emitted in debug info to provide richer information to debuggers. This is currently off by default as it causes some compilation scenarios to be noticeably slower.

default-visibility

The tracking issue for this feature is: <https://github.com/rust-lang/rust/issues/131090>

This flag can be used to override the target's `default_visibility` setting.

This option only affects building of shared objects and should have no effect on executables.

Visibility can be set to one of three options:

- protected
- hidden
- interposable

Hidden visibility

Using `-Zdefault-visibility=hidden` is roughly equivalent to Clang's `-fvisibility=hidden` cmdline flag. Hidden symbols will not be exported from the created shared object, so cannot be referenced from other shared objects or from executables.

Protected visibility

Using `-Zdefault-visibility=protected` will cause rust-mangled symbols to be emitted with "protected" visibility. This signals the compiler, the linker and the runtime linker that these symbols cannot be overridden by the executable or by other shared objects earlier in the load order.

This will allow the compiler to emit direct references to symbols, which may improve performance. It also removes the need for these symbols to be resolved when a shared object built with this option is loaded.

Using protected visibility when linking with GNU ld prior to 2.40 will result in linker errors when building for Linux. Other linkers such as LLD are not affected.

Interposable

Using `-Zdefault-visibility=interposable` will cause symbols to be emitted with "default" visibility. On platforms that support it, this makes it so that symbols can be interposed, which means that they can be overridden by symbols with the same name from the executable or by other shared objects earlier in the load order.

direct_access_external_data

The tracking issue for this feature is: <https://github.com/rust-lang/compiler-team/issues/707>

Option `-Z direct-access-external-data` controls how to access symbols of external data.

Supported values for this option are:

- `yes` - Don't use GOT indirection to reference external data symbols.
- `no` - Use GOT indirection to reference external data symbols.

If the option is not explicitly specified, different targets have different default values.

dump-mono-stats

The `-Z dump-mono-stats` compiler flag generates a file with a list of the monomorphized items in the current crate. It is useful for investigating compile times.

It accepts an optional directory where the file will be located. If no directory is specified, the file will be placed in the current directory.

See also `-Z dump-mono-stats-format` and `-Z print-mono-items`. Unlike `print-mono-items`, `dump-mono-stats` aggregates monomorphized items by definition and includes a size estimate of how large the item is when codegened.

See <https://rustc-dev-guide.rust-lang.org/backend/monomorph.html> for an overview of monomorphized items.

dump-mono-stats-format

The `-Z dump-mono-stats-format` compiler flag controls what file format to use for `-Z dump-mono-stats`. The default is markdown; currently JSON is also supported. JSON can be useful for programmatically manipulating the results (e.g. to find the item that took the longest to compile).

dwarf-version

The tracking issue for this feature is: <https://github.com/rust-lang/rust/issues/103057>

This option controls the version of DWARF that the compiler emits, on platforms that use DWARF to encode debug information. It takes one of the following values:

- **2** : DWARF version 2 (the default on certain platforms, like macOS).
- **4** : DWARF version 4 (the default on certain platforms, like Linux).
- **5** : DWARF version 5.

dylib-lto

This option enables using LTO for the `dylib` crate type. This is currently only used for compiling `rustc` itself (more specifically, the `librustc_driver` dylib).

embed-source

This flag controls whether the compiler embeds the program source code text into the object debug information section. It takes one of the following values:

- `y`, `yes`, `on` or `true`: put source code in debug info.
- `n`, `no`, `off`, `false` or no value: omit source code from debug info (the default).

This flag is ignored in configurations that don't emit DWARF debug information and is ignored on non-LLVM backends. `-Z embed-source` requires DWARFv5. Use `-Z dwarf-version=5` to control the compiler's DWARF target version and `-g` to enable debug info generation.

emit-stack-sizes

The tracking issue for this feature is: [#54192](#)

The rustc flag `-Z emit-stack-sizes` makes LLVM emit stack size metadata.

NOTE: This LLVM feature only supports the ELF object format as of LLVM 8.0. Using this flag with targets that use other object formats (e.g. macOS and Windows) will result in it being ignored.

Consider this crate:

```
#![crate_type = "lib"]

use std::ptr;

pub fn foo() {
    // this function doesn't use the stack
}

pub fn bar() {
    let xs = [0u32; 2];

    // force LLVM to allocate `xs` on the stack
    unsafe { ptr::read_volatile(&xs.as_ptr()); }
}
```

Using the `-Z emit-stack-sizes` flag produces extra linker sections in the output *object file*.


```
$ rustc -C opt-level=3 --emit=obj foo.rs

$ size -A foo.o
foo.o :
section              size  addr
.text                0      0
.text._ZN3foo3foo17he211d7b4a3a0c16eE 1      0
.text._ZN3foo3bar17h1acb594305f70c2eE 22     0
.note.GNU-stack      0      0
.eh_frame            72     0
Total                95

$ rustc -C opt-level=3 --emit=obj -Z emit-stack-sizes foo.rs

$ size -A foo.o
foo.o :
section              size  addr
.text                0      0
.text._ZN3foo3foo17he211d7b4a3a0c16eE 1      0
.stack_sizes         9      0
.text._ZN3foo3bar17h1acb594305f70c2eE 22     0
.stack_sizes         9      0
.note.GNU-stack      0      0
.eh_frame            72     0
Total               113
```

As of LLVM 7.0 the data will be written into a section named `.stack_sizes` and the format is "an array of pairs of function symbol values (pointer size) and stack sizes (unsigned LEB128)".

```
$ objdump -d foo.o

foo.o:      file format elf64-x86-64

Disassembly of section .text._ZN3foo3foo17he211d7b4a3a0c16eE:

0000000000000000 <_ZN3foo3foo17he211d7b4a3a0c16eE>:
    0:   c3                      retq

Disassembly of section .text._ZN3foo3bar17h1acb594305f70c2eE:

0000000000000000 <_ZN3foo3bar17h1acb594305f70c2eE>:
    0:   48 83 ec 10             sub    $0x10,%rsp
    4:   48 8d 44 24 08          lea    0x8(%rsp),%rax
    9:   48 89 04 24             mov    %rax,(%rsp)
   d:   48 8b 04 24             mov    (%rsp),%rax
  11:   48 83 c4 10             add    $0x10,%rsp
  15:   c3                      retq

$ objdump -s -j .stack_sizes foo.o

foo.o:      file format elf64-x86-64

Contents of section .stack_sizes:
 0000 00000000 00000000 00                      .....
Contents of section .stack_sizes:
 0000 00000000 00000000 10                      .....
```

It's important to note that linkers will discard this linker section by default. To preserve the section you can use a linker script like the one shown below.

```
/* file: keep-stack-sizes.x */
SECTIONS
{
    /* `INFO` makes the section not allocatable so it won't be loaded into
    memory */
    .stack_sizes (INFO) :
    {
        KEEP(*(.stack_sizes));
    }
}
```

The linker script must be passed to the linker using a rustc flag like `-C link-arg.`

```
// file: src/main.rs
use std::ptr;

#[inline(never)]
fn main() {
    let xs = [0u32; 2];

    // force LLVM to allocate `xs` on the stack
    unsafe { ptr::read_volatile(&xs.as_ptr()); }
}
```

```
$ RUSTFLAGS="-Z emit-stack-sizes" cargo build --release

$ size -A target/release/hello | grep stack_sizes || echo section was not
found
section was not found

$ RUSTFLAGS="-Z emit-stack-sizes" cargo rustc --release -- \
  -C link-arg=-Wl,-Tkeep-stack-sizes.x \
  -C link-arg=-N

$ size -A target/release/hello | grep stack_sizes
.stack_sizes          90    176272

$ # non-allocatable section (flags don't contain the "A" (alloc) flag)
$ readelf -S target/release/hello
Section Headers:
   [Nr]  Name                               Type           Address         Offset
       Size                               EntSize        Flags   Link  Info  Align
(..)
 [1031] .stack_sizes                          PROGBITS        0000000000002b090 0002b0f0
       000000000000005a 0000000000000000 L         5      0      1

$ objdump -s -j .stack_sizes target/release/hello

target/release/hello:      file format elf64-x86-64

Contents of section .stack_sizes:
 2b090 c0040000 00000000 08f00400 00000000 .....
 2b0a0 00080005 00000000 00000810 05000000 .....
 2b0b0 00000000 20050000 00000000 10400500 .... @..
 2b0c0 00000000 00087005 00000000 00000080 .....p.....
 2b0d0 05000000 00000000 90050000 00000000 .....
 2b0e0 00a00500 00000000 0000.....
```

Author note: I'm not entirely sure why, in *this* case, `-N` is required in addition to `-Tkeep-stack-sizes.x`. For example, it's not required when producing statically

linked files for the ARM Cortex-M architecture.

emscripten-wasm-eh

Use the WebAssembly exception handling ABI to unwind for the `wasm32-unknown-emscripten`. If compiling with this setting, the `emcc` linker should be invoked with `-fwasm-exceptions`. If linking with C/C++ files, the C/C++ files should also be compiled with `-fwasm-exceptions`.

env-set

The tracking issue for this feature is: [#118372](#).

This option flag allows to specify environment variables value at compile time to be used by `env!` and `option_env!` macros. It also impacts `tracked_env::var` function from the `proc_macro` crate.

This information will be stored in the dep-info files. For more information about dep-info files, take a look [here](#).

When retrieving an environment variable value, the one specified by `--env-set` will take precedence. For example, if you want have `PATH=a` in your environment and pass:

```
rustc --env-set PATH=env
```

Then you will have:

```
assert_eq!(env!("PATH"), "env");
```

It will trigger a new compilation if any of the `--env-set` argument value is different. So if you first passed:

```
--env-set A=B --env X=12
```

and then on next compilation:

```
--env-set A=B
```

X value is different (not set) so the code will be re-compiled.

Please note that on Windows, environment variables are case insensitive but case preserving whereas `rustc`'s environment variables are case sensitive. For example, having `Path` in your environment (case insensitive) is different than using `rustc --env-set Path=...` (case sensitive).

export-executable-symbols

The tracking issue for this feature is: [#84161](#).

The `-Zexport-executable-symbols` compiler flag makes `rustc` export symbols from executables. The resulting binary is runnable, but can also be used as a dynamic library. This is useful for interoperating with programs written in other languages, in particular languages with a runtime like Java or Lua.

For example on windows:

```
#[no_mangle]
fn my_function() -> usize {
    return 42;
}

fn main() {
    println!("Hello, world!");
}
```

A standard `cargo build` will produce a `.exe` without an export directory. When the `export-executable-symbols` flag is added

```
export RUSTFLAGS="-Zexport-executable-symbols"
cargo build
```

the binary has an export directory with the functions:

The Export Tables (interpreted `.edata` section contents)

...

```
[Ordinal/Name Pointer] Table
[ 0] my_function
[ 1] main
```

(the output of `objdump -x` on the binary)

Please note that the `#[no_mangle]` attribute is required. Without it, the symbol is not exported.

The equivalent of this flag in C and C++ compilers is the `__declspec(dllexport)`

annotation or the `-rdynamic` linker flag.

--extern Options

- Tracking issue for `--extern` crate modifiers: [#98405](#)
- Tracking issue for `noprelude`: [#98398](#)
- Tracking issue for `priv`: [#98399](#)
- Tracking issue for `nounused`: [#98400](#)
- Tracking issue for `force`: [#111302](#)

The behavior of the `--extern` flag can be modified with `noprelude`, `priv` or `nounused` options.

This is unstable feature, so you have to provide `-Zunstable-options` to enable it.

Examples

Use your own build of the `core` crate.

```
rustc main.rs -Z unstable-options --extern noprelude:core=libcore.rlib
```

To use multiple options, separate them with a comma:

```
rustc main.rs -Z unstable-options --extern  
noprelude,priv,nounused:mydep=mydep.rlib
```

Options

- `noprelude`: Do not add the crate to the external prelude. If used, it will need to be imported using `extern crate`. This is used by the [build-std project](#) to simulate compatibility with sysroot-only crates.
- `priv`: Mark the crate as a private dependency for the `exported_private_dependencies` lint.
- `nounused`: Suppress `unused-crate-dependencies` warnings for the crate.
- `force`: Resolve the crate as if it is used, even if it is not used. This can be used to satisfy compilation session requirements like the presence of an allocator or panic handler.

external-clangrt

This option controls whether the compiler links in its own runtime library for [sanitizers](#). Passing this flag makes the compiler *not* link its own library. For more information, see the section in the sanitizers doc on [working with other languages](#).

fixed-x18

This option prevents the compiler from using the x18 register. It is only supported on `aarch64`.

From the [ABI spec](#):

X18 is the platform register and is reserved for the use of platform ABIs. This is an additional temporary register on platforms that don't assign a special meaning to it.

This flag only has an effect when the x18 register would otherwise be considered a temporary register. When the flag is applied, x18 is always a reserved register.

This flag is intended for use with the shadow call stack sanitizer. Generally, when that sanitizer is enabled, the x18 register is used to store a pointer to the shadow stack. Enabling this flag prevents the compiler from overwriting the shadow stack pointer with temporary data, which is necessary for the sanitizer to work correctly.

Currently, the `-Zsanitizer=shadow-call-stack` flag is only supported on platforms that always treat x18 as a reserved register, and the `-Zfixed-x18` flag is not required to use the sanitizer on such platforms. However, the sanitizer may be supported on targets where this is not the case in the future. One way to do so now on Nightly compilers is to explicitly supply this `-Zfixed-x18` flag with `aarch64` targets, so that the sanitizer is available for instrumentation on targets like `aarch64-unknown-none`, for instance. However, discretion is still required to make sure that the runtime support is in place for this sanitizer to be effective.

It is undefined behavior for `-Zsanitizer=shadow-call-stack` code to call into code where x18 is a temporary register. On the other hand, when you are *not* using the shadow call stack sanitizer, compilation units compiled with and without the `-Zfixed-x18` flag are compatible with each other.

fmt-debug

The tracking issue for this feature is: [#129709](#).

Option `-Z fmt-debug=val` controls verbosity of derived `Debug` implementations and debug formatting in format strings (`{:?}`).

- `full` — `#[derive(Debug)]` prints types recursively. This is the default behavior.
- `shallow` — `#[derive(Debug)]` prints only the type name, or name of a variant of a fieldless enums. Details of the `Debug` implementation are not stable and may change in the future. Behavior of custom `fmt::Debug` implementations is not affected.
- `none` — `#[derive(Debug)]` does not print anything at all. `{:?}` in formatting strings has no effect. This option may reduce size of binaries, and remove occurrences of type names in the binary that are not removed by stripping symbols. However, it may also cause `panic!` and `assert!` messages to be incomplete.

function-return

The tracking issue for this feature is: <https://github.com/rust-lang/rust/issues/116853>.

Option `-Zfunction-return` controls how function returns are converted.

It is equivalent to Clang's and GCC's `-mfunction-return`. The Linux kernel uses it for RETHUNK builds. For details, see [LLVM commit 2240d72f15f3](#) ("[X86] initial -mfunction-return=thunk-extern support") which introduces the feature.

Supported values for this option are:

- `keep`: do not convert function returns.
- `thunk-extern`: convert function returns (`ret`) to jumps (`jmp`) to an external symbol called `__x86_return_thunk`.

Like in Clang, GCC's values `thunk` and `thunk-inline` are not supported.

Only x86 and non-large code models are supported.

instrument-xray

The tracking issue for this feature is: [#102921](#).

Enable generation of NOP sleds for XRay function tracing instrumentation. For more information on XRay, read [LLVM documentation](#), and/or the [XRay whitepaper](#).

Set the `-Z instrument-xray` compiler flag in order to enable XRay instrumentation.

- `-Z instrument-xray` – use the default settings
- `-Z instrument-xray=skip-exit` – configure a custom setting
- `-Z instrument-xray=ignore-loops,instruction-threshold=300` – multiple settings separated by commas

Supported options:

- `always` – force instrumentation of all functions
- `never` – do no instrument any functions
- `ignore-loops` – ignore presence of loops, instrument functions based only on instruction count
- `instruction-threshold=10` – set a different instruction threshold for instrumentation
- `skip-entry` – do no instrument function entry
- `skip-exit` – do no instrument function exit

The default settings are:

- instrument both entry & exit from functions
- instrument functions with at least 200 instructions, or containing a non-trivial loop

Note that `-Z instrument-xray` only enables generation of NOP sleds which on their own don't do anything useful. In order to actually trace the functions, you will need to link a separate runtime library of your choice, such as Clang's [XRay Runtime Library](#).

link-native-libraries

This option allows ignoring libraries specified in `#[link]` attributes instead of passing them to the linker. This can be useful in build systems that manage native libraries themselves and pass them manually, e.g. with `-Clink-arg`.

- `yes` - Pass native libraries to the linker. Default.
- `no` - Don't pass native libraries to the linker.

linker-features

The `-Zlinker-features` compiler flag allows enabling or disabling specific features used during linking, and is intended to be stabilized under the codegen options as `-Clinker-features`.

These feature flags are a flexible extension mechanism that is complementary to linker flavors, designed to avoid the combinatorial explosion of having to create a new set of flavors for each linker feature we'd want to use.

For example, this design allows:

- default feature sets for principal flavors, or for specific targets.
- flavor-specific features: for example, clang offers automatic cross-linking with `--target`, which gcc-style compilers don't support. The *flavor* is still a C/C++ compiler, and we don't want to multiply the number of flavors for this use-case. Instead, we can have a single `+target` feature.
- umbrella features: for example, if clang accumulates more features in the future than just the `+target` above. That could be modeled as `+clang`.
- niche features for resolving specific issues: for example, on Apple targets the linker flag implementing the `as-needed` native link modifier (#99424) is only possible on sufficiently recent linker versions.
- still allows for discovery and automation, for example via feature detection. This can be useful in exotic environments/build systems.

The flag accepts a comma-separated list of features, individually enabled (`+features`) or disabled (`-features`), though currently only one is exposed on the CLI:

- `lld`: to toggle using the lld linker, either the system-installed binary, or the self-contained `rust-lld` linker.

As described above, this list is intended to grow in the future.

One of the most common uses of this flag will be to toggle self-contained linking with `rust-lld` on and off: `-Clinker-features=+lld -Clink-self-contained=+linker` will use the toolchain's `rust-lld` as the linker. Inversely, `-Clinker-features=-lld` would opt out of that, if the current target had self-contained linking enabled by default.

lint-llvm-ir

This flag will add `LintPass` to the start of the pipeline. You can use it to check for common errors in the LLVM IR generated by `rustc`. You can add `-Cllvm-args=-lint-abort-on-error` to abort the process if errors were found.

llvm-module-flag

This flag allows adding a key/value to the `!llvm.module.flags` metadata in the LLVM-IR for a compiled Rust module. The syntax is

```
-Z llvm_module_flag=<name>:<type>:<value>:<behavior>
```

Currently only u32 values are supported but the type is required to be specified for forward compatibility. The `behavior` element must match one of the named LLVM metadata behaviors

location-detail

The tracking issue for this feature is: [#70580](#).

Option `-Z location-detail=val` controls what location details are tracked when using `caller_location`. This allows users to control what location details are printed as part of panic messages, by allowing them to exclude any combination of filenames, line numbers, and column numbers. This option is intended to provide users with a way to mitigate the size impact of `#[track_caller]`.

This option supports a comma separated list of location details to be included. Valid options within this list are:

- `file` - the filename of the panic will be included in the panic output
- `line` - the source line of the panic will be included in the panic output
- `column` - the source column of the panic will be included in the panic output

Any combination of these three options are supported. Alternatively, you can pass `none` to this option, which results in no location details being tracked. If this option is not specified, all three are included by default.

An example of a panic output when using `-Z location-detail=line`:

```
panicked at 'Process blink had a fault', <redacted>:323:0
```

The code size savings from this option are two-fold. First, the `&'static str` values for each path to a file containing a panic are removed from the binary. For projects with deep directory structures and many files with panics, this can add up. This category of savings can only be realized by excluding filenames from the panic output. Second, savings can be realized by allowing multiple panics to be fused into a single panicking branch. It is often the case that within a single file, multiple panics with the same panic message exist -- e.g. two calls to `Option::unwrap()` in a single line, or two calls to `Result::expect()` on adjacent lines. If column and line information are included in the `Location` struct passed to the panic handler, these branches cannot be fused, as the output is different depending on which panic occurs. However if line and column information is identical for all panics, these branches can be fused, which can lead to substantial code size savings, especially for small embedded binaries with many panics.

The savings from this option are amplified when combined with the use of `-Zbuild-std`,

as otherwise paths for panics within the standard library are still included in your binary.

min-function-alignment

The tracking issue for this feature is: <https://github.com/rust-lang/rust/issues/82232>.

The `-Zmin-function-alignment=<align>` flag specifies the minimum alignment of functions for which code is generated. The `align` value must be a power of 2, other values are rejected.

Note that `-Zbuild-std` (or similar) is required to apply this minimum alignment to standard library functions. By default, these functions come precompiled and their alignments won't respect the `min-function-alignment` flag.

This flag is equivalent to:

- `-fmin-function-alignment` for `GCC`
- `-falign-functions` for `Clang`

The specified alignment is a minimum. A higher alignment can be specified for specific functions by using the `repr(align(...))` feature and annotating the function with a `#[repr(align(<align>))]` attribute. The attribute's value is ignored when it is lower than the value passed to `min-function-alignment`.

There are two additional edge cases for this flag:

- targets have a minimum alignment for functions (e.g. on x86_64 the lowest that LLVM generates is 16 bytes). A `min-function-alignment` value lower than the target's minimum has no effect.
- the maximum alignment supported by rust (and LLVM) is `2^29`. Trying to set a higher value results in an error.

move_size_limit

The `-Zmove-size-limit=N` compiler flag enables `large_assignments` lints which will warn when moving objects whose size exceeds `N` bytes.

Lint warns only about moves in functions that participate in code generation. Consequently it will be ineffective for compiler invocation that emit metadata only, i.e., `cargo check` like workflows.

no-jump-tables

The tracking issue for this feature is [#116592](#)

This option enables the `-fno-jump-tables` flag for LLVM, which makes the codegen backend avoid generating jump tables when lowering switches.

This option adds the LLVM `no-jump-tables=true` attribute to every function.

The option can be used to help provide protection against jump-oriented-programming (JOP) attacks, such as with the linux kernel's [IBT](#).

```
RUSTFLAGS="-Zno-jump-tables" cargo +nightly build -Z build-std
```

no-parallel-llvm

This flag disables parallelization of codegen and linking, while otherwise preserving behavior with regard to codegen units and LTO.

This flag is not useful for regular users, but it can be useful for debugging the backend. Codegen issues commonly only manifest under specific circumstances, e.g. if multiple codegen units are used and ThinLTO is enabled. Serialization of these threaded configurations makes the use of LLVM debugging facilities easier, by avoiding the interleaving of output.

no-unique-section-names

This flag currently applies only to ELF-based targets using the LLVM codegen backend. It prevents the generation of unique ELF section names for each separate code and data item when `-Z function-sections` is also in use, which is the default for most targets. This option can reduce the size of object files, and depending on the linker, the final ELF binary as well.

For example, a function `func` will by default generate a code section called `.text.func`. Normally this is fine because the linker will merge all those `.text.*` sections into a single one in the binary. However, starting with [LLVM 12](#), the backend will also generate unique section names for exception handling, so you would see a section name of `.gcc_except_table.func` in the object file and potentially in the final ELF binary, which could add significant bloat to programs that contain many functions.

This flag instructs LLVM to use the same `.text` and `.gcc_except_table` section name for each function, and it is analogous to Clang's `-fno-unique-section-names` option.

on-broken-pipe

The tracking issue for this feature is: [#97889](#)

Note: The ui for this feature was previously an attribute named `#[unix_sigpipe = "..."]`.

Overview

The `-Zon-broken-pipe=...` compiler flag can be used to specify how libstd shall setup `SIGPIPE` on Unix platforms before invoking `fn main()`. This flag is ignored on non-Unix targets. The flag can be used with three different values or be omitted entirely. It affects `SIGPIPE` before `fn main()` and before children get `exec()`'ed:

Compiler flag	<code>SIGPIPE</code> before <code>fn main()</code>	<code>SIGPIPE</code> before child <code>exec()</code>
not used	<code>SIG_IGN</code>	<code>SIG_DFL</code>
<code>-Zon-broken-pipe=kill</code>	<code>SIG_DFL</code>	not touched
<code>-Zon-broken-pipe=error</code>	<code>SIG_IGN</code>	not touched
<code>-Zon-broken-pipe=inherit</code>	not touched	not touched

`-Zon-broken-pipe` not used

If `-Zon-broken-pipe` is not used, libstd will behave in the manner it has since 2014, before Rust 1.0. `SIGPIPE` will be set to `SIG_IGN` before `fn main()` and result in `EPIPE` errors which are converted to `std::io::ErrorKind::BrokenPipe`.

When spawning child processes, `SIGPIPE` will be set to `SIG_DFL` before doing the underlying `exec()` syscall.

–Zon-broken-pipe=kill

Set the `SIGPIPE` handler to `SIG_DFL` before invoking `fn main()`. This will result in your program getting killed if it tries to write to a closed pipe. This is normally what you want if your program produces textual output.

When spawning child processes, `SIGPIPE` will not be touched. This normally means child processes inherit `SIG_DFL` for `SIGPIPE`.

Example

```
fn main() {  
    loop {  
        println!("hello world");  
    }  
}
```

```
$ rustc -Zon-broken-pipe=kill main.rs  
$ ./main | head -n1  
hello world
```

–Zon-broken-pipe=error

Set the `SIGPIPE` handler to `SIG_IGN` before invoking `fn main()`. This will result in `ErrorKind::BrokenPipe` errors if your program tries to write to a closed pipe. This is normally what you want if you for example write socket servers, socket clients, or pipe peers.

When spawning child processes, `SIGPIPE` will not be touched. This normally means child processes inherit `SIG_IGN` for `SIGPIPE`.

Example

```
fn main() {  
    loop {  
        println!("hello world");  
    }  
}
```

```
$ rustc -Zon-broken-pipe=error main.rs  
$ ./main | head -n1  
hello world  
thread 'main' panicked at library/std/src/io/stdio.rs:1118:9:  
failed printing to stdout: Broken pipe (os error 32)  
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

-Zon-broken-pipe=inherit

Leave `SIGPIPE` untouched before entering `fn main()`. Unless the parent process has changed the default `SIGPIPE` handler from `SIG_DFL` to something else, this will behave the same as `-Zon-broken-pipe=kill`.

When spawning child processes, `SIGPIPE` will not be touched. This normally means child processes inherit `SIG_DFL` for `SIGPIPE`.

patchable-function-entry

The `-Z patchable-function-entry=total_nops,prefix_nops` or `-Z patchable-function-entry=total_nops` compiler flag enables nop padding of function entries with 'total_nops' nops, with an offset for the entry of the function at 'prefix_nops' nops. In the second form, 'prefix_nops' defaults to 0.

As an illustrative example, `-Z patchable-function-entry=3,2` would produce:

```
nop
nop
function_label:
nop
//Actual function code begins here
```

This flag is used for hotpatching, especially in the Linux kernel. The flag arguments are modeled after the `-fpatchable-function-entry` flag as defined for both [Clang](#) and [gcc](#) and is intended to provide the same effect.

print=check-cfg

The tracking issue for this feature is: [#125704](#).

This option of the `--print` flag print the list of all the expected cfgs.

This is related to the `--check-cfg` flag which allows specifying arbitrary expected names and values.

This print option works similarly to `--print=cfg` (modulo check-cfg specifics).

<code>--check-cfg</code>	<code>--print=check-cfg</code>
<code>cfg(foo)</code>	<code>foo</code>
<code>cfg(foo, values("bar"))</code>	<code>foo="bar"</code>
<code>cfg(foo, values(none(), "bar"))</code>	<code>foo & foo="bar"</code>
	<i>check-cfg specific syntax</i>
<code>cfg(foo, values(any()))</code>	<code>foo=any()</code>
<code>cfg(foo, values())</code>	<code>foo=</code>
<code>cfg(any())</code>	<code>any()</code>
<code>none</code>	<code>any()=any()</code>

To be used like this:

```
rustc --print=check-cfg -Zunstable-options lib.rs
```

profile-sample-use

`-Zprofile-sample-use=code.prof` directs `rustc` to use the profile `code.prof` as a source for Automatic Feedback Directed Optimization (AFDO). See the documentation of `-Zdebug-info-for-profiling` for more information on using AFDO.

randomize-layout

The tracking issue for this feature is: [#106764](#).

The `-Zrandomize-layout` flag changes the layout algorithm for `repr(Rust)` types defined in the current crate from its normal optimization goals to pseudorandomly rearranging fields within the degrees of freedom provided by the largely unspecified default representation. This also affects type sizes and padding. Downstream instantiations of generic types defined in a crate with randomization enabled will also be randomized.

It can be used to find unsafe code that accidentally relies on unspecified behavior.

Randomization is not guaranteed to use a different permutation for each compilation session. `-Zlayout-seed=<u64>` can be used to supply additional entropy.

Randomization only approximates the intended freedom of `repr(Rust)`. Sometimes two distinct types may still consistently result in the same layout due to limitations of the current implementation. Randomization may become more aggressive over time as our coverage of the available degrees of freedoms improves. Corollary: Randomization is not a safety oracle. Two struct layouts being observably the same under different layout seeds on the current compiler version does not guarantee that future compiler versions won't give them distinct layouts.

Randomization may also become less aggressive in the future if additional guarantees get added to the default layout.

reg-struct-return

The tracking issue for this feature is: <https://github.com/rust-lang/rust/issues/116973>.

Option `-Zreg-struct-return` causes the compiler to return small structs in registers instead of on the stack for extern "C"-like functions. It is UNSOUND to link together crates that use different values for this flag. It is only supported on `x86`.

It is equivalent to Clang's and GCC's `-freg-struct-return`.

regparm

The tracking issue for this feature is: <https://github.com/rust-lang/rust/issues/131749>.

Option `-Zregparm=N` causes the compiler to pass `N` arguments in registers `EAX`, `EDX`, and `ECX` instead of on the stack for `"C"`, `"cdecl"`, and `"stdcall"` fn. It is UNSOUND to link together crates that use different values for this flag. It is only supported on `x86`.

It is equivalent to `Clang`'s and `GCC`'s `-mregparm`.

Supported values for this option are 0-3.

Implementation details: For eligible arguments, llvm `inreg` attribute is set.

remap-cwd-prefix

The tracking issue for this feature is: [#87325](#).

This flag will rewrite absolute paths under the current working directory, replacing the current working directory prefix with a specified value.

The given value may be absolute or relative, or empty. This switch takes precedence over `--remap-path-prefix` in case they would both match a given path.

This flag helps to produce deterministic output, by removing the current working directory from build output, while allowing the command line to be universally reproducible, such that the same execution will work on all machines, regardless of build environment.

Example

```
# This would produce an absolute path to main.rs in build outputs of  
# "./main.rs".  
rustc -Z remap-cwd-prefix=. main.rs
```

remap-path-scope

The tracking issue for this feature is: [#111540](#).

When the `--remap-path-prefix` option is passed to `rustc`, source path prefixes in all output will be affected by default. The `--remap-path-scope` argument can be used in conjunction with `--remap-path-prefix` to determine paths in which output context should be affected. This flag accepts a comma-separated list of values and may be specified multiple times, in which case the scopes are aggregated together. The valid scopes are:

- `macro` - apply remappings to the expansion of `std::file!()` macro. This is where paths in embedded panic messages come from
- `diagnostics` - apply remappings to printed compiler diagnostics
- `debuginfo` - apply remappings to debug informations
- `object` - apply remappings to all paths in compiled executables or libraries, but not elsewhere. Currently an alias for `macro,debuginfo`.
- `all` - an alias for all of the above, also equivalent to supplying only `--remap-path-prefix` without `--remap-path-scope`.

Example

```
# This would produce an absolute path to main.rs in build outputs of
# "./main.rs".
rustc --remap-path-prefix=$(PWD)=/remapped -Zremap-path-scope=object main.rs
```

report-time

The tracking issue for this feature is: [#64888](#)

The `report-time` feature adds a possibility to report execution time of the tests generated via `libtest`.

This is unstable feature, so you have to provide `-Zunstable-options` to get this feature working.

Sample usage command:

```
./test_executable -Zunstable-options --report-time
```

Available options:

`--report-time`

Show execution time of each `test`.
Threshold values `for` colored output can be configured via
``RUST_TEST_TIME_UNIT``, ``RUST_TEST_TIME_INTEGRATION``
and
``RUST_TEST_TIME_DOCTEST`` environment variables.
Expected format of environment variable is
``VARIABLE=WARN_TIME,CRITICAL_TIME``.
Not available `for` `--format=terse`

`--ensure-time`

Treat excess of the `test` execution time `limit` as error.
Threshold values `for` this option can be configured via
``RUST_TEST_TIME_UNIT``, ``RUST_TEST_TIME_INTEGRATION``
and
``RUST_TEST_TIME_DOCTEST`` environment variables.
Expected format of environment variable is
``VARIABLE=WARN_TIME,CRITICAL_TIME``.
``CRITICAL_TIME`` here means the `limit` that should not be exceeded by `test`.

Example of the environment variable format:

```
RUST_TEST_TIME_UNIT=100,200
```

where 100 stands for warn time, and 200 stands for critical time.

Examples

```
cargo test --tests -- -Zunstable-options --report-time
  Finished dev [unoptimized + debuginfo] target(s) in 0.02s
  Running target/debug/deps/example-27fb188025bec02c

running 3 tests
test tests::unit_test_quick ... ok <0.000s>
test tests::unit_test_warn ... ok <0.055s>
test tests::unit_test_critical ... ok <0.110s>

test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out

  Running target/debug/deps/tests-cedb06f6526d15d9

running 3 tests
test unit_test_quick ... ok <0.000s>
test unit_test_warn ... ok <0.550s>
test unit_test_critical ... ok <1.100s>

test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

sanitizer

Sanitizers are tools that help detect and prevent various types of bugs and vulnerabilities in software. They are available in compilers and work by instrumenting the code to add additional runtime checks. While they provide powerful tools for identifying bugs or security issues, it's important to note that using sanitizers can introduce runtime overhead and might not catch all possible issues. Therefore, they are typically used alongside other best practices in software development, such as testing and fuzzing, to ensure the highest level of software quality and security.

The tracking issues for this feature are:

- [#39699](#).
 - [#89653](#).
-

This feature allows for use of one of following sanitizers:

- Those intended for testing or fuzzing (but not production use):
 - [AddressSanitizer](#) a fast memory error detector.
 - [HWAddressSanitizer](#) a memory error detector similar to AddressSanitizer, but based on partial hardware assistance.
 - [LeakSanitizer](#) a run-time memory leak detector.
 - [MemorySanitizer](#) a detector of uninitialized reads.
 - [ThreadSanitizer](#) a fast data race detector.
- Those that apart from testing, may be used in production:
 - [ControlFlowIntegrity](#) LLVM Control Flow Integrity (CFI) provides forward-edge control flow protection.
 - [DataFlowSanitizer](#) a generic dynamic data flow analysis framework.
 - [KernelControlFlowIntegrity](#) LLVM Kernel Control Flow Integrity (KCFI) provides forward-edge control flow protection for operating systems kernels.
 - [MemTagSanitizer](#) fast memory error detector based on Armv8.5-A Memory Tagging Extension.
 - [SafeStack](#) provides backward-edge control flow protection by separating the stack into safe and unsafe regions.
 - [ShadowCallStack](#) provides backward-edge control flow protection (aarch64 only).

To enable a sanitizer compile with `-Zsanitizer=address`, `-Zsanitizer=cfi`, `-Zsanitizer=dataflow`, `-Zsanitizer=hwaddress`, `-Zsanitizer=leak`, `-Zsanitizer=memory`, `-Zsanitizer=memtag`, `-Zsanitizer=shadow-call-stack`, or `-Zsanitizer=thread`. You might also need the `--target` and `build-std` flags. If you're working with other languages that are also instrumented with sanitizers, you might need the `external-clangrt` flag. See the section on [working with other languages](#).

Example:

```
$ RUSTFLAGS=-Zsanitizer=address cargo build -Zbuild-std --target x86_64-unknown-linux-gnu
```

Additional options for sanitizers can be passed to LLVM command line argument processor via LLVM arguments using `llvm-args` codegen option (e.g., `-Cllvm-args=-dfsan-combine-pointer-labels-on-load=false`). See the sanitizer documentation for more information about additional options.

AddressSanitizer

AddressSanitizer is a memory error detector. It can detect the following types of bugs:

- Out of bound accesses to heap, stack and globals
- Use after free
- Use after return (runtime flag `ASAN_OPTIONS=detect_stack_use_after_return=1`)
- Use after scope
- Double-free, invalid free
- Memory leaks

The memory leak detection is enabled by default on Linux, and can be enabled with runtime flag `ASAN_OPTIONS=detect_leaks=1` on macOS.

AddressSanitizer is supported on the following targets:

- `aarch64-apple-darwin`
- `aarch64-unknown-fuchsia`
- `aarch64-unknown-linux-gnu`
- `x86_64-apple-darwin`
- `x86_64-unknown-fuchsia`
- `x86_64-unknown-freebsd`
- `x86_64-unknown-linux-gnu`

AddressSanitizer works with non-instrumented code although it will impede its ability to detect some bugs. It is not expected to produce false positive reports.

See the [Clang AddressSanitizer documentation](#) for more details.

Examples

Stack buffer overflow:

```
fn main() {
    let xs = [0, 1, 2, 3];
    let _y = unsafe { *xs.as_ptr().offset(4) };
}
```

```
$ export RUSTFLAGS=-Zsanitizer=address RUSTDOCFLAGS=-Zsanitizer=address
$ cargo run -Zbuild-std --target x86_64-unknown-linux-gnu
==37882==ERROR: AddressSanitizer: stack-buffer-overflow on address
0x7ffe400e6250 at pc 0x5609a841fb20 bp 0x7ffe400e6210 sp 0x7ffe400e6208
READ of size 4 at 0x7ffe400e6250 thread T0
    #0 0x5609a841fb1f in example::main::h628ffc6626ed85b2
/.../src/main.rs:3:23
    ...
```

```
Address 0x7ffe400e6250 is located in stack of thread T0 at offset 48 in frame
    #0 0x5609a841f8af in example::main::h628ffc6626ed85b2 /.../src/main.rs:1
```

```
This frame has 1 object(s):
    [32, 48) 'xs' (line 2) <== Memory access at offset 48 overflows this
variable
HINT: this may be a false positive if your program uses some custom stack
unwind mechanism, swapcontext or vfork
    (longjmp and C++ exceptions *are* supported)
SUMMARY: AddressSanitizer: stack-buffer-overflow /.../src/main.rs:3:23 in
example::main::h628ffc6626ed85b2
Shadow bytes around the buggy address:
```

```
0x100048014bf0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x100048014c00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x100048014c10: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x100048014c20: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x100048014c30: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0x100048014c40: 00 00 00 00 f1 f1 f1 f1 00 00[f3]f3 00 00 00 00
0x100048014c50: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x100048014c60: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x100048014c70: f1 f1 f1 f1 00 00 f3 f3 00 00 00 00 00 00 00 00
0x100048014c80: 00 00 00 00 00 00 00 00 00 00 00 00 00 f1 f1 f1
```

```

0x100048014c90: 00 00 f3 f3 00 00 00 00 00 00 00 00 00 00 00 00
Shadow byte legend (one shadow byte represents 8 application bytes):
Addressable:          00
Partially addressable: 01 02 03 04 05 06 07
Heap left redzone:    fa
Freed heap region:    fd
Stack left redzone:    f1
Stack mid redzone:    f2
Stack right redzone:   f3
Stack after return:    f5
Stack use after scope: f8
Global redzone:       f9
Global init order:    f6
Poisoned by user:     f7
Container overflow:    fc
Array cookie:         ac
Intra object redzone: bb
ASan internal:        fe
Left alloca redzone:  ca
Right alloca redzone: cb
Shadow gap:          cc
==37882==ABORTING

```

Use of a stack object after its scope has already ended:

```

static mut P: *mut usize = std::ptr::null_mut();

fn main() {
    unsafe {
        {
            let mut x = 0;
            P = &mut x;
        }
        std::ptr::write_volatile(P, 123);
    }
}

```

```

$ export RUSTFLAGS=-Zsanitizer=address RUSTDOCFLAGS=-Zsanitizer=address
$ cargo run -Zbuild-std --target x86_64-unknown-linux-gnu
=====
==39249==ERROR: AddressSanitizer: stack-use-after-scope on address
0x7ffc7ed3e1a0 at pc 0x55c98b262a8e bp 0x7ffc7ed3e050 sp 0x7ffc7ed3e048
WRITE of size 8 at 0x7ffc7ed3e1a0 thread T0
    #0 0x55c98b262a8d in core::ptr::write_volatile::he21f1df5a82f329a
/.../src/rust/src/libcore/ptr/mod.rs:1048:5
    #1 0x55c98b262cd2 in example::main::h628ffc6626ed85b2
/.../src/main.rs:9:9
    ...

```

Address 0x7ffc7ed3e1a0 is located in stack of thread T0 at offset 32 in frame
#0 0x55c98b262bdf in example::main::h628ffc6626ed85b2 /.../src/main.rs:3

This frame has 1 object(s):

[32, 40) 'x' (line 6) <== Memory access at offset 32 is inside this variable

HINT: this may be a false positive if your program uses some custom stack unwind mechanism, swapcontext or vfork

(longjmp and C++ exceptions *are* supported)

SUMMARY: AddressSanitizer: stack-use-after-scope

/.../src/rust/src/libcore/ptr/mod.rs:1048:5 in

core::ptr::write_volatile::he21f1df5a82f329a

Shadow bytes around the buggy address:

```
0x10000fd9fbe0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x10000fd9fbf0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x10000fd9fc00: 00 00 00 00 00 00 00 00 00 00 00 00 00 f1 f1 f1 f1
0x10000fd9fc10: f8 f8 f3 f3 00 00 00 00 00 00 00 00 00 00 00 00
0x10000fd9fc20: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0x10000fd9fc30: f1 f1 f1 f1[f8]f3 f3 f3 00 00 00 00 00 00 00 00
0x10000fd9fc40: 00 00 00 00 00 00 00 00 00 00 00 00 00 f1 f1 f1 f1
0x10000fd9fc50: 00 00 f3 f3 00 00 00 00 00 00 00 00 00 00 00 00
0x10000fd9fc60: 00 00 00 00 00 00 00 00 f1 f1 f1 f1 00 00 f3 f3
0x10000fd9fc70: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x10000fd9fc80: 00 00 00 00 f1 f1 f1 f1 00 00 f3 f3 00 00 00 00
```

Shadow byte legend (one shadow byte represents 8 application bytes):

```
Addressable:          00
Partially addressable: 01 02 03 04 05 06 07
Heap left redzone:    fa
Freed heap region:    fd
Stack left redzone:    f1
Stack mid redzone:     f2
Stack right redzone:   f3
Stack after return:    f5
Stack use after scope: f8
Global redzone:        f9
Global init order:     f6
Poisoned by user:      f7
Container overflow:    fc
Array cookie:          ac
Intra object redzone:  bb
ASan internal:         fe
Left alloca redzone:   ca
Right alloca redzone:  cb
Shadow gap:           cc
```

==39249==ABORTING

ControlFlowIntegrity

See the [Clang ControlFlowIntegrity documentation](#) for more details.

```
#![feature(naked_functions)]  
  
use std::arch::asm;  
use std::mem;  
  
fn add_one(x: i32) -> i32 {  
    x + 1  
}  
  
#[naked]  
pub extern "C" fn add_two(x: i32) {  
    // x + 2 preceded by a landing pad/nop block  
    unsafe {  
        asm!(  
            "  
            nop  
            nop  
            nop  
            nop  
            nop  
            nop  
            nop
```

```

        nop
        nop
        lea eax, [rdi+2]
        ret
    },
    options(noreturn)
);
}
}

fn do_twice(f: fn(i32) -> i32, arg: i32) -> i32 {
    f(arg) + f(arg)
}

fn main() {
    let answer = do_twice(add_one, 5);

    println!("The answer is: {}", answer);

    println!("With CFI enabled, you should not see the next answer");
    let f: fn(i32) -> i32 = unsafe {
        // Offset 0 is a valid branch/call destination (i.e., the function
entry
        // point), but offsets 1-8 within the landing pad/nop block are
invalid
        // branch/call destinations (i.e., within the body of the function).
        mem::transmute::<*const u8, fn(i32) -> i32>((add_two as *const
u8).offset(5))
    };
    let next_answer = do_twice(f, 5);

    println!("The next answer is: {}", next_answer);
}

```

Fig. 1. Redirecting control flow using an indirect branch/call to an invalid destination (i.e., within the body of the function).

```

$ cargo run --release
  Compiling rust-cfi-1 v0.1.0 (/home/rcvalle/rust-cfi-1)
  Finished release [optimized] target(s) in 0.42s
  Running `target/release/rust-cfi-1`
The answer is: 12
With CFI enabled, you should not see the next answer
The next answer is: 14
$

```

Fig. 2. Build and execution of Fig. 1 with LLVM CFI disabled.

```
$ RUSTFLAGS="-Clinker-plugin-lto -Clinker=clang -Clink-arg=-fuse-ld=lld -  
Zsanitizer=cfi" cargo run -Zbuild-std -Zbuild-std-features --release --target  
x86_64-unknown-linux-gnu  
...  
Compiling rust-cfi-1 v0.1.0 (/home/rcvalle/rust-cfi-1)  
Finished release [optimized] target(s) in 1m 08s  
Running `target/x86_64-unknown-linux-gnu/release/rust-cfi-1`  
The answer is: 12  
With CFI enabled, you should not see the next answer  
Illegal instruction  
$
```

Fig. 3. Build and execution of Fig. 1 with LLVM CFI enabled.

When LLVM CFI is enabled, if there are any attempts to change/hijack control flow using an indirect branch/call to an invalid destination, the execution is terminated (see Fig. 3).

Example 2: Redirecting control flow using an indirect branch/call to a function with a different number of parameters

```

use std::mem;

fn add_one(x: i32) -> i32 {
    x + 1
}

fn add_two(x: i32, _y: i32) -> i32 {
    x + 2
}

fn do_twice(f: fn(i32) -> i32, arg: i32) -> i32 {
    f(arg) + f(arg)
}

fn main() {
    let answer = do_twice(add_one, 5);

    println!("The answer is: {}", answer);

    println!("With CFI enabled, you should not see the next answer");
    let f: fn(i32) -> i32 =
        unsafe { mem::transmute::<*const u8, fn(i32) -> i32>(add_two as
*const u8) };
    let next_answer = do_twice(f, 5);

    println!("The next answer is: {}", next_answer);
}

```

Fig. 4. Redirecting control flow using an indirect branch/call to a function with a different number of parameters than arguments intended/passed in the call/branch site.

```

$ cargo run --release
  Compiling rust-cfi-2 v0.1.0 (/home/rcvalle/rust-cfi-2)
  Finished release [optimized] target(s) in 0.43s
  Running `target/release/rust-cfi-2`
The answer is: 12
With CFI enabled, you should not see the next answer
The next answer is: 14
$

```

Fig. 5. Build and execution of Fig. 4 with LLVM CFI disabled.

```
$ RUSTFLAGS="-Clinker-plugin-lto -Clinker=clang -Clink-arg=-fuse-ld=lld -  
Zsanitizer=cfi" cargo run -Zbuild-std -Zbuild-std-features --release --target  
x86_64-unknown-linux-gnu  
...  
Compiling rust-cfi-2 v0.1.0 (/home/rcvalle/rust-cfi-2)  
Finished release [optimized] target(s) in 1m 08s  
Running `target/x86_64-unknown-linux-gnu/release/rust-cfi-2`  
The answer is: 12  
With CFI enabled, you should not see the next answer  
Illegal instruction  
$
```

Fig. 6. Build and execution of Fig. 4 with LLVM CFI enabled.

When LLVM CFI is enabled, if there are any attempts to change/hijack control flow using an indirect branch/call to a function with different number of parameters than arguments intended/passed in the call/branch site, the execution is also terminated (see Fig. 6).

Example 3: Redirecting control flow using an indirect branch/call to a function with different return and parameter types


```

use std::mem;

fn add_one(x: i32) -> i32 {
    x + 1
}

fn add_two(x: i64) -> i64 {
    x + 2
}

fn do_twice(f: fn(i32) -> i32, arg: i32) -> i32 {
    f(arg) + f(arg)
}

fn main() {
    let answer = do_twice(add_one, 5);

    println!("The answer is: {}", answer);

    println!("With CFI enabled, you should not see the next answer");
    let f: fn(i32) -> i32 =
        unsafe { mem::transmute::(<*const u8, fn(i32) -> i32>(add_two as
*const u8) ) };
    let next_answer = do_twice(f, 5);

    println!("The next answer is: {}", next_answer);
}

```

Fig. 7. Redirecting control flow using an indirect branch/call to a function with different return and parameter types than the return type expected and arguments intended/passed at the call/branch site.

```

$ cargo run --release
Compiling rust-cfi-3 v0.1.0 (/home/rcvalle/rust-cfi-3)
Finished release [optimized] target(s) in 0.44s
Running `target/release/rust-cfi-3`
The answer is: 12
With CFI enabled, you should not see the next answer
The next answer is: 14
$

```

Fig. 8. Build and execution of Fig. 7 with LLVM CFI disabled.

```
$ RUSTFLAGS="-Clinker-plugin-lto -Clinker=clang -Clink-arg=-fuse-ld=lld -
Zsanitizer=cfi" cargo run -Zbuild-std -Zbuild-std-features --release --target
x86_64-unknown-linux-gnu
...
Compiling rust-cfi-3 v0.1.0 (/home/rcvalle/rust-cfi-3)
Finished release [optimized] target(s) in 1m 07s
Running `target/x86_64-unknown-linux-gnu/release/rust-cfi-3`
The answer is: 12
With CFI enabled, you should not see the next answer
Illegal instruction
$
```

Fig. 9. Build and execution of Fig. 7 with LLVM CFI enabled.

When LLVM CFI is enabled, if there are any attempts to change/hijack control flow using an indirect branch/call to a function with different return and parameter types than the return type expected and arguments intended/passed in the call/branch site, the execution is also terminated (see Fig. 9).

Example 4: Redirecting control flow using an indirect branch/call to a function with different return and parameter types across the FFI boundary

```
int
do_twice(int (*fn)(int), int arg)
{
    return fn(arg) + fn(arg);
}
```

Fig. 10. Example C library.

```

use std::mem;

#[link(name = "foo")]
extern "C" {
    fn do_twice(f: unsafe extern "C" fn(i32) -> i32, arg: i32) -> i32;
}

unsafe extern "C" fn add_one(x: i32) -> i32 {
    x + 1
}

unsafe extern "C" fn add_two(x: i64) -> i64 {
    x + 2
}

fn main() {
    let answer = unsafe { do_twice(add_one, 5) };

    println!("The answer is: {}", answer);

    println!("With CFI enabled, you should not see the next answer");
    let f: unsafe extern "C" fn(i32) -> i32 = unsafe {
        mem::transmute::<*&const u8, unsafe extern "C" fn(i32) -> i32>(add_two
as *&const u8)
    };
    let next_answer = unsafe { do_twice(f, 5) };

    println!("The next answer is: {}", next_answer);
}

```

Fig. 11. Redirecting control flow using an indirect branch/call to a function with different return and parameter types than the return type expected and arguments intended/passed in the call/branch site, across the FFI boundary.

```

$ make
mkdir -p target/release
clang -I. -Isrc -Wall -c src/foo.c -o target/release/libfoo.o
llvm-ar rcs target/release/libfoo.a target/release/libfoo.o
RUSTFLAGS="-L./target/release -Clinker=clang -Clink-arg=-fuse-ld=lld" cargo
build --release
    Compiling rust-cfi-4 v0.1.0 (/home/rcvalle/rust-cfi-4)
    Finished release [optimized] target(s) in 0.49s
$ ./target/release/rust-cfi-4
The answer is: 12
With CFI enabled, you should not see the next answer
The next answer is: 14
$

```

Fig. 12. Build and execution of Figs. 10–11 with LLVM CFI disabled.

```
$ make
mkdir -p target/release
clang -I. -Isrc -Wall -flto -fsanitize=cfi -fsanitize-cfi-icall-experimental-
normalize-integers -fvisibility=hidden -c -emit-llvm src/foo.c -o
target/release/libfoo.bc
llvm-ar rcs target/release/libfoo.a target/release/libfoo.bc
RUSTFLAGS="-L./target/release -Clinker-plugin-lto -Clinker=clang -Clink-arg=-
fuse-ld=lld -Zsanitizer=cfi -Zsanitizer-cfi-normalize-integers" cargo build -
Zbuild-std -Zbuild-std-features --release --target x86_64-unknown-linux-gnu
...
Compiling rust-cfi-4 v0.1.0 (/home/rcvalle/rust-cfi-4)
Finished release [optimized] target(s) in 1m 06s
$ ./target/x86_64-unknown-linux-gnu/release/rust-cfi-4
The answer is: 12
With CFI enabled, you should not see the next answer
Illegal instruction
$
```

Fig. 13. Build and execution of Figs. 10–11 with LLVM CFI enabled.

When LLVM CFI is enabled, if there are any attempts to redirect control flow using an indirect branch/call to a function with different return and parameter types than the return type expected and arguments intended/passed in the call/branch site, even across the FFI boundary and for extern "C" function types indirectly called (i.e., callbacks/function pointers) across the FFI boundary, the execution is also terminated (see Fig. 13).

HWAddressSanitizer

HWAddressSanitizer is a newer variant of AddressSanitizer that consumes much less memory.

HWAddressSanitizer is supported on the following targets:

- `aarch64-linux-android`
- `aarch64-unknown-linux-gnu`

HWAddressSanitizer requires `tagged-globals` target feature to instrument globals. To enable this target feature compile with `-C target-feature=+tagged-globals`

See the [Clang HWAddressSanitizer documentation](#) for more details.

Example

Heap buffer overflow:

```
fn main() {
    let xs = vec![0, 1, 2, 3];
    let _y = unsafe { *xs.as_ptr().offset(4) };
}
```

```
$ rustc main.rs -Zsanitizer=hwaddress -C target-feature=+tagged-globals -C
linker=aarch64-linux-gnu-gcc -C link-arg=-fuse-ld=lld --target
aarch64-unknown-linux-gnu
```

```
$ ./main
==241==ERROR: HWAddressSanitizer: tag-mismatch on address 0xefdeffff0050 at
pc 0xaaae0ae4a98
READ of size 4 at 0xefdeffff0050 tags: 2c/00 (ptr/mem) in thread T0
#0 0xaaae0ae4a94 (/.../main+0x54a94)
...
```

[0xefdeffff0040,0xefdeffff0060) is a small allocated heap chunk; size: 32
offset: 16

0xefdeffff0050 is located 0 bytes to the right of 16-byte region

[0xefdeffff0040,0xefdeffff0050)

allocated here:

```
#0 0xaaae0acb80c (/.../main+0x3b80c)
...
```

Thread: T0 0xeffe00002000 stack: [0xfffffc28ad000,0xfffffc30ad000) sz: 8388608
tls: [0xfffffaa10a020,0xfffffaa10a7d0)

Memory tags around the buggy address (one tag corresponds to 16 bytes):

```
0xfefceffffef80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00
0xfefceffffef90: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00
0xfefceffffefa0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00
0xfefceffffefb0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00
0xfefceffffefc0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00
0xfefceffffefd0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00
0xfefceffffefe0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00
0xfefceffffeff0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00
```

```
=>0xfefceffff000: d7 d7 05 00 2c [00] 00 00 00 00 00 00 00 00 00 00
00
0xfefceffff010: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00
0xfefceffff020: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00
0xfefceffff030: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00
0xfefceffff040: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00
0xfefceffff050: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00
0xfefceffff060: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00
0xfefceffff070: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00
0xfefceffff080: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00
Tags for short granules around the buggy address (one tag corresponds to 16
bytes):
0xfefceffff0: .. .. .. .. .. .. .. .. .. .. .. .. .. ..
..
=>0xfefceffff000: .. .. 8c .. .. [..] .. .. .. .. .. .. .. ..
..
0xfefceffff010: .. .. .. .. .. .. .. .. .. .. .. .. .. ..
..
See
https://clang.llvm.org/docs/HardwareAssistedAddressSanitizerDesign.html#short-granules
for a description of short granule tags
Registers where the failure occurred (pc 0xaaaae0ae4a98):
x0 2c00efdeffff0050 x1 0000000000000004 x2 0000000000000004 x3
0000000000000000
x4 0000fffffc30ac37 x5 000000000000005d x6 0000fffffc30ac37 x7
0000ffff00000000
x8 2c00efdeffff0050 x9 0200efff00000000 x10 0000000000000000 x11
0200efff00000000
x12 0200effe000000310 x13 0200effe000000310 x14 0000000000000008 x15
5d00fffffc30ac360
x16 0000aaaae0ad062c x17 0000000000000003 x18 0000000000000001 x19
0000fffffc30ac658
x20 4e00fffffc30ac6e0 x21 0000aaaae0ac5e10 x22 0000000000000000 x23
0000000000000000
x24 0000000000000000 x25 0000000000000000 x26 0000000000000000 x27
0000000000000000
x28 0000000000000000 x29 0000fffffc30ac5a0 x30 0000aaaae0ae4a98
SUMMARY: HWAddressSanitizer: tag-mismatch (/.../main+0x54a94)
```

KernelControlFlowIntegrity

The LLVM Kernel Control Flow Integrity (CFI) support to the Rust compiler initially provides forward-edge control flow protection for operating systems kernels for Rust-compiled code only by aggregating function pointers in groups identified by their return and parameter types. (See [LLVM commit cff5bef "KCFI sanitizer"](#).)

Forward-edge control flow protection for C or C++ and Rust -compiled code "mixed binaries" (i.e., for when C or C++ and Rust -compiled code share the same virtual address space) will be provided in later work by defining and using compatible type identifiers (see Type metadata in the design document in the tracking issue [#89653](#)).

LLVM KCFI can be enabled with `-Zsanitizer=kcfi`.

LLVM KCFI is supported on the following targets:

- `aarch64-linux-android`
- `aarch64-unknown-linux-gnu`
- `x86_64-linux-android`
- `x86_64-unknown-linux-gnu`

See the [Clang KernelControlFlowIntegrity documentation](#) for more details.

DataFlowSanitizer

DataFlowSanitizer is a generalised dynamic data flow analysis.

Unlike other Sanitizer tools, this tool is not designed to detect a specific class of bugs on its own. Instead, it provides a generic dynamic data flow analysis framework to be used by clients to help detect application-specific issues within their own code.

DataFlowSanitizer is supported on the following targets:

- `x86_64-unknown-linux-gnu`

See the [Clang DataFlowSanitizer documentation](#) for more details.

KernelAddressSanitizer

KernelAddressSanitizer (KASAN) is a freestanding version of AddressSanitizer which is suitable for detecting memory errors in programs which do not have a runtime environment, such as operating system kernels. KernelAddressSanitizer requires manual

implementation of the underlying functions used for tracking KernelAddressSanitizer state.

KernelAddressSanitizer is supported on the following targets:

- `aarch64-unknown-none`
- `riscv64gc-unknown-none-elf`
- `riscv64imac-unknown-none-elf`
- `x86_64-unknown-none`

See the [Linux Kernel's KernelAddressSanitizer documentation](#) for more details.

LeakSanitizer

LeakSanitizer is run-time memory leak detector.

LeakSanitizer is supported on the following targets:

- `aarch64-unknown-linux-gnu`
- `x86_64-apple-darwin`
- `x86_64-unknown-linux-gnu`

See the [Clang LeakSanitizer documentation](#) for more details.

MemorySanitizer

MemorySanitizer is detector of uninitialized reads.

MemorySanitizer is supported on the following targets:

- `aarch64-unknown-linux-gnu`
- `x86_64-unknown-freebsd`
- `x86_64-unknown-linux-gnu`

MemorySanitizer requires all program code to be instrumented. C/C++ dependencies need to be recompiled using Clang with `-fsanitize=memory` option. Failing to achieve that will result in false positive reports.

See the [Clang MemorySanitizer documentation](#) for more details.

Example

Detecting the use of uninitialized memory. The `-Zbuild-std` flag rebuilds and instruments the standard library, and is strictly necessary for the correct operation of the tool. The `-Zsanitizer-memory-track-origins` enables tracking of the origins of uninitialized memory:

```
use std::mem::MaybeUninit;

fn main() {
    unsafe {
        let a = MaybeUninit::<[usize; 4]>::uninit();
        let a = a.assume_init();
        println!("{}", a[2]);
    }
}
```

```
$ export \
  RUSTFLAGS='-Zsanitizer=memory -Zsanitizer-memory-track-origins' \
  RUSTDOCFLAGS='-Zsanitizer=memory -Zsanitizer-memory-track-origins'
$ cargo clean
$ cargo run -Zbuild-std --target x86_64-unknown-linux-gnu
==9416==WARNING: MemorySanitizer: use-of-uninitialized-value
    #0 0x560c04f7488a in core::fmt::num::imp::fmt_u64::haa293b0b098501ca
$RUST/build/x86_64-unknown-linux-
gnu/stage1/lib/rustlib/src/rust/src/libcore/fmt/num.rs:202:16
...
Uninitialized value was stored to memory at
    #0 0x560c04ae898a in __msan_memcpy.part.0 $RUST/src/llvm-
project/compiler-rt/lib/msan/msan_interceptors.cc:1558:3
    #1 0x560c04b2bf88 in memory::main::hd2333c1899d997f5
$CWD/src/main.rs:6:16

Uninitialized value was created by an allocation of 'a' in the stack frame
of function '_ZN6memory4main17hd2333c1899d997f5E'
    #0 0x560c04b2bc50 in memory::main::hd2333c1899d997f5 $CWD/src/main.rs:3
```

MemTagSanitizer

MemTagSanitizer detects a similar class of errors as AddressSanitizer and HardwareAddressSanitizer, but with lower overhead suitable for use as hardening for production binaries.

MemTagSanitizer is supported on the following targets:

- `aarch64-linux-android`
- `aarch64-unknown-linux-gnu`

MemTagSanitizer requires hardware support and the `mte` target feature. To enable this target feature compile with `-C target-feature="+mte"`.

See the [LLVM MemTagSanitizer documentation](#) for more details.

SafeStack

SafeStack provides backward edge control flow protection by separating the stack into data which is only accessed safely (the safe stack) and all other data (the unsafe stack).

SafeStack can be enabled with the `-Zsanitizer=safestack` option and is supported on the following targets:

- `x86_64-unknown-linux-gnu`

See the [Clang SafeStack documentation](#) for more details.

ShadowCallStack

ShadowCallStack provides backward edge control flow protection by storing a function's return address in a separately allocated 'shadow call stack' and loading the return address from that shadow call stack. AArch64 and RISC-V both have a platform register defined in their ABIs, which is `x18` and `x3 / gp` respectively, that can optionally be reserved for this purpose. Software support from the operating system and runtime may be required depending on the target platform which is detailed in the remaining section. See the [Clang ShadowCallStack documentation](#) for more details.

ShadowCallStack can be enabled with `-Zsanitizer=shadow-call-stack` option and is supported on the following targets:

AArch64 family

ShadowCallStack requires the use of the ABI defined platform register, `x18`, which is required for code generation purposes. When `x18` is not reserved, and is instead used as

a scratch register subsequently, enabling ShadowCallStack would lead to undefined behaviour due to corruption of return address or invalid memory access when the instrumentation restores return register to the link register `lr` from the already clobbered `x18` register. In other words, code that is calling into or called by functions instrumented with ShadowCallStack must reserve the `x18` register or preserve its value.

aarch64-linux-android and aarch64-unknown-fuchsia/aarch64-fuchsia

This target already reserves the `x18` register. A runtime must be provided by the application or operating system. If `bionic` is used on this target, the software support is provided. Otherwise, a runtime needs to prepare a memory region and points `x18` to the region which serves as the shadow call stack.

aarch64-unknown-none

In addition to support from a runtime by the application or operating system, the `-Zfixed-x18` flag is also mandatory.

RISC-V 64 family

ShadowCallStack uses either the `gp` register for software shadow stack, also known as `x3`, or the `ssp` register if `Zicfiss` extension is available. `gp / x3` is currently always reserved and available for ShadowCallStack instrumentation, and `ssp` in case of `Zicfiss` is only accessible through its dedicated shadow stack instructions.

Support from the runtime and operating system is required when `gp / x3` is used for software shadow stack. A runtime must prepare a memory region and point `gp / x3` to the region before executing the code.

The following targets support ShadowCallStack.

- `riscv64imac-unknown-none-elf`
- `riscv64gc-unknown-none-elf`
- `riscv64gc-unknown-fuchsia`

ThreadSanitizer

ThreadSanitizer is a data race detection tool. It is supported on the following targets:

- `aarch64-apple-darwin`
- `aarch64-unknown-linux-gnu`
- `x86_64-apple-darwin`
- `x86_64-unknown-freebsd`
- `x86_64-unknown-linux-gnu`

To work correctly ThreadSanitizer needs to be "aware" of all synchronization operations in a program. It generally achieves that through combination of library interception (for example synchronization performed through `pthread_mutex_lock` / `pthread_mutex_unlock`) and compile time instrumentation (e.g. atomic operations). Using it without instrumenting all the program code can lead to false positive reports.

ThreadSanitizer does not support atomic fences `std::sync::atomic::fence`, nor synchronization performed using inline assembly code.

See the [Clang ThreadSanitizer documentation](#) for more details.

Example

```
static mut A: usize = 0;

fn main() {
    let t = std::thread::spawn(|| {
        unsafe { A += 1 };
    });
    unsafe { A += 1 };

    t.join().unwrap();
}
```

```
$ export RUSTFLAGS=-Zsanitizer=thread RUSTDOCFLAGS=-Zsanitizer=thread
$ cargo run -Zbuild-std --target x86_64-unknown-linux-gnu
=====
WARNING: ThreadSanitizer: data race (pid=10574)
  Read of size 8 at 0x5632dfe3d030 by thread T1:
    #0 example::main::_$u7b$$u7b$closure$u7d$$u7d$::h23f64b0b2f8c9484
    ../src/main.rs:5:18 (example+0x86cec)
    ...

  Previous write of size 8 at 0x5632dfe3d030 by main thread:
    #0 example::main::h628ffc6626ed85b2 /.../src/main.rs:7:14
    (example+0x868c8)
    ...
    #11 main <null> (example+0x86a1a)

  Location is global 'example::A::h43ac149ddf992709' of size 8 at
  0x5632dfe3d030 (example+0x000000bd9030)
```

Instrumentation of external dependencies and std

The sanitizers to varying degrees work correctly with partially instrumented code. On the one extreme is LeakSanitizer that doesn't use any compile time instrumentation, on the other is MemorySanitizer that requires that all program code to be instrumented (failing to achieve that will inevitably result in false positives).

It is strongly recommended to combine sanitizers with recompiled and instrumented standard library, for example using `cargo -Zbuild-std` [functionality](#).

Working with other languages

Sanitizers rely on compiler runtime libraries to function properly. Rust links in its own compiler runtime which might conflict with runtimes required by languages such as C++. Since Rust's runtime doesn't always contain the symbols required by C++ instrumented code, you might need to skip linking it so another runtime can be linked instead.

A separate unstable option `-Zexternal-clangrt` can be used to make rustc skip linking the compiler runtime for the sanitizer. This will require you to link in an external runtime, such as from clang instead.

Build scripts and procedural macros

Use of sanitizers together with build scripts and procedural macros is technically possible, but in almost all cases it would be best avoided. This is especially true for procedural macros which would require an instrumented version of rustc.

In more practical terms when using cargo always remember to pass `--target` flag, so that rustflags will not be applied to build scripts and procedural macros.

Symbolizing the Reports

Sanitizers produce symbolized stacktraces when llvm-symbolizer binary is in `PATH`.

Additional Information

- [Sanitizers project page](#)
- [AddressSanitizer in Clang](#)
- [ControlFlowIntegrity in Clang](#)
- [DataFlowSanitizer in Clang](#)
- [HWAddressSanitizer in Clang](#)
- [Linux Kernel's KernelAddressSanitizer documentation](#)
- [LeakSanitizer in Clang](#)
- [MemorySanitizer in Clang](#)
- [MemTagSanitizer in LLVM](#)
- [ThreadSanitizer in Clang](#)

self-profile

The `-Zself-profile` compiler flag enables rustc's internal profiler. When enabled, the compiler will output three binary files in the specified directory (or the current working directory if no directory is specified). These files can be analyzed by using the tools in the `measureme` repository.

To control the data recorded in the trace files, use the `-Zself-profile-events` flag.

For example:

First, run a compilation session and provide the `-Zself-profile` flag:

```
$ rustc --crate-name foo -Zself-profile
```

This will generate three files in the working directory such as:

- `foo-1234.events`
- `foo-1234.string_data`
- `foo-1234.string_index`

Where `foo` is the name of the crate and `1234` is the process id of the rustc process.

To get a summary of where the compiler is spending its time:

```
$ ../measureme/target/release/summarize summarize foo-1234
```

To generate a flamegraph of the same data:

```
$ ../measureme/target/release/inferno foo-1234
```

To dump the event data in a Chromium-profiler compatible format:

```
$ ../measureme/target/release/crox foo-1234
```

For more information, consult the `measureme` documentation.

self-profile-events

The `-Zself-profile-events` compiler flag controls what events are recorded by the self-profiler when it is enabled via the `-Zself-profile` flag.

This flag takes a comma delimited list of event types to record.

For example:

```
$ rustc -Zself-profile -Zself-profile-events=default,args
```

Event types

- `query-provider`
 - Traces each query used internally by the compiler.
- `generic-activity`
 - Traces other parts of the compiler not covered by the query system.
- `query-cache-hit`
 - Adds tracing information that records when the in-memory query cache is "hit" and does not need to re-execute a query which has been cached.
 - Disabled by default because this significantly increases the trace file size.
- `query-blocked`
 - Tracks time that a query tries to run but is blocked waiting on another thread executing the same query to finish executing.
 - Query blocking only occurs when the compiler is built with parallel mode support.
- `incr-cache-load`
 - Tracks time that is spent loading and deserializing query results from the incremental compilation on-disk cache.

- `query-keys`
 - Adds a serialized representation of each query's query key to the tracing data.
 - Disabled by default because this significantly increases the trace file size.
- `function-args`
 - Adds additional tracing data to some `generic-activity` events.
 - Disabled by default for parity with `query-keys`.
- `llvm`
 - Adds tracing information about LLVM passes and codegeneration.
 - Disabled by default because this significantly increases the trace file size.

Event synonyms

- `none`
 - Disables all events. Equivalent to the self-profiler being disabled.
- `default`
 - The default set of events which stikes a balance between providing detailed tracing data and adding additional overhead to the compilation.
- `args`
 - Equivalent to `query-keys` and `function-args`.
- `all`
 - Enables all events.

Examples

Enable the profiler and capture the default set of events (both invocations are equivalent):

```
$ rustc -Zself-profile  
$ rustc -Zself-profile -Zself-profile-events=default
```

Enable the profiler and capture the default events and their arguments:

```
$ rustc -Zself-profile -Zself-profile-events=default,args
```

shell-argfiles

The `-Zshell-argfiles` compiler flag allows argfiles to be parsed using POSIX "shell-style" quoting. When enabled, the compiler will use `shlex` to parse the arguments from argfiles specified with `@shell:<path>`.

Because this feature controls the parsing of input arguments, the `-Zshell-argfiles` flag must be present before the argument specifying the shell-style argument file.

small-data-threshold

This flag controls the maximum static variable size that may be included in the "small data sections" (.sdata, .sbss) supported by some architectures (RISCV, MIPS, M68K, Hexagon). Can be set to `0` to disable the use of small data sections.

Target support is indicated by the `small_data_threshold_support` target option which can be:

- `none` (`SmallDataThresholdSupport::None`) for no support
- `default-for-arch` (`SmallDataThresholdSupport::DefaultForArch`) which is automatically translated into an appropriate value for the target.
- `llvm-module-flag=<flag_name>` (`SmallDataThresholdSupport::LlvmModuleFlag`) for specifying the threshold via an LLVM module flag
- `llvm-arg=<arg_name>` (`SmallDataThresholdSupport::LlvmArg`) for specifying the threshold via an LLVM argument.

src-hash-algorithm

The tracking issue for this feature is: [#70401](#).

The `-Z src-hash-algorithm` compiler flag controls which algorithm is used when hashing each source file. The hash is stored in the debug info and can be used by a debugger to verify the source code matches the executable.

Supported hash algorithms are: `md5`, `sha1`, and `sha256`. Note that not all hash algorithms are supported by all debug info formats.

By default, the compiler chooses the hash algorithm based on the target specification.

temps-dir

The `-Ztemps-dir` compiler flag specifies the directory to write the intermediate files in. If not set, the output directory is used. This option is useful if you are running more than one instance of `rustc` (e.g. with different `--crate-type` settings), and you need to make sure they are not overwriting each other's intermediate files. No files are kept unless `-C save-temps=yes` is also set.

tiny-const-eval-limit

The `-Ztiny-const-eval-limit` compiler flag sets a tiny, non-configurable limit for const eval. This flag should only be used by const eval tests in the rustc test suite.

tls_model

The tracking issue for this feature is: None.

Option `-Z tls-model` controls `TLS model` used to generate code for accessing `# [thread_local] static` items.

Supported values for this option are:

- `global-dynamic` - General Dynamic TLS Model (alternatively called Global Dynamic) is the most general option usable in all circumstances, even if the TLS data is defined in a shared library loaded at runtime and is accessed from code outside of that library. This is the default for most targets.
- `local-dynamic` - model usable if the TLS data is only accessed from the shared library or executable it is defined in. The TLS data may be in a library loaded after startup (via `dlopen`).
- `initial-exec` - model usable if the TLS data is defined in the executable or in a shared library loaded at program startup. The TLS data must not be in a library loaded after startup (via `dlopen`).
- `local-exec` - model usable only if the TLS data is defined directly in the executable, but not in a shared library, and is accessed only from that executable.
- `emulated` - Uses thread-specific data keys to implement emulated TLS. It is like using a general-dynamic TLS model for all modes.

`rustc` and LLVM may use a more optimized model than specified if they know that we are producing an executable rather than a library, or that the `static` item is private enough.

ub-checks

The tracking issue for this feature is: [#123499](#).

The `-Zub-checks` compiler flag enables additional runtime checks that detect some causes of Undefined Behavior at runtime. By default, `-Zub-checks` flag inherits the value of `-Cdebug-assertions`.

All checks are generated on a best-effort basis; even if we have a check implemented for some cause of Undefined Behavior, it may be possible for the check to not fire. If a dependency is compiled with `-Zub-checks=no` but the final binary or library is compiled with `-Zub-checks=yes`, UB checks reached by the dependency are likely to be optimized out.

When `-Zub-checks` detects UB, a non-unwinding panic is produced. That means that we will not unwind the stack and will not call any `Drop` impls, but we will execute the configured panic hook. We expect that unsafe code has been written which relies on code not unwinding which may have UB checks inserted. Ergo, an unwinding panic could easily turn works-as-intended UB into a much bigger problem. Calling the panic hook theoretically has the same implications, but we expect that the standard library panic hook will be stateless enough to be always called, and that if a user has configured a panic hook that the hook may be very helpful to debugging the detected UB.

unsound-mir-opts

The `-Zunsound-mir-opts` compiler flag enables [MIR optimization passes](#) which can cause unsound behavior. This flag should only be used by MIR optimization tests in the rustc test suite.

verbose-asm

The tracking issue for this feature is: [#126802](#).

This enables passing `-Zverbose-asm` to get contextual comments added by LLVM.

Sample code:

```
#[no_mangle]
pub fn foo(a: i32, b: i32) -> i32 {
    a + b
}
```

Default output:

```
foo:
    push    rax
    add     edi, esi
    mov     dword ptr [rsp + 4], edi
    seto    al
    jo      .LBB0_2
    mov     eax, dword ptr [rsp + 4]
    pop     rcx
    ret

.LBB0_2:
    lea     rdi, [rip + .L__unnamed_1]
    mov     rax, qword ptr [rip +
core::panicking::panic_const::panic_const_add_overflow::h9c85248fe0d735b2@GOT
PCREL]
    call    rax

.L__unnamed_2:
    .ascii  "/app/example.rs"

.L__unnamed_1:
    .quad   .L__unnamed_2
    .asciz  "\017\000\000\000\000\000\000\000\004\000\000\000\005\000\000"
```

With `-Zverbose-asm`:

```

foo:                                     # @foo
# %bb.0:
    push    rax
    add     edi, esi
    mov     dword ptr [rsp + 4], edi      # 4-byte Spill
    seto    al
    jo      .LBB0_2
# %bb.1:
    mov     eax, dword ptr [rsp + 4]      # 4-byte Reload
    pop     rcx
    ret
.LBB0_2:
    lea     rdi, [rip + .L__unnamed_1]
    mov     rax, qword ptr [rip +
core::panicking::panic_const::panic_const_add_overflow::h9c85248fe0d735b2@GOT
PCREL]
    call    rax
                                           # -- End function
.L__unnamed_2:
    .ascii  "/app/example.rs"
.L__unnamed_1:
    .quad   .L__unnamed_2
    .asciz  "\017\000\000\000\000\000\000\000\004\000\000\000\005\000\000"
                                           # DW_AT_external

```

virtual-function-elimination

This option controls whether LLVM runs the Virtual Function Elimination (VFE) optimization. This optimization is only available with LTO, so this flag can only be passed if `-Clto` is also passed.

VFE makes it possible to remove functions from vtables that are never dynamically called by the rest of the code. Without this flag, LLVM makes the really conservative assumption, that if any function in a vtable is called, no function that is referenced by this vtable can be removed. With this flag additional information is given to LLVM, so that it can determine which functions are actually called and remove the unused functions.

Limitations

At the time of writing this flag may remove vtable functions too eagerly. One such example is in this code:

```
trait Foo { fn foo(&self) { println!("foo") } }

impl Foo for usize {}

pub struct FooBox(Box<dyn Foo>);

pub fn make_foo() -> FooBox { FooBox(Box::new(0)) }

#[inline]
pub fn f(a: FooBox) { a.0.foo() }
```

In the above code the `Foo` trait is private, so an assumption is made that its functions can only be seen/called from the current crate and can therefore get optimized out, if unused. However, with `make_foo` you can produce a wrapped `dyn Foo` type outside of the current crate, which can then be used in `f`. Due to inlining of `f`, `Foo::foo` can then be called from a foreign crate. This can lead to miscompilations.

wasm-c-abi

This option controls whether Rust uses the spec-compliant C ABI when compiling for the `wasm32-unknown-unknown` target.

This makes it possible to be ABI-compatible with all other spec-compliant Wasm targets like `wasm32-wasip1`.

This compiler flag is perma-unstable, as it will be enabled by default in the future with no option to fall back to the old non-spec-compliant ABI.

Language features

aarch64_unstable_target_feature

The tracking issue for this feature is: [#44839](#)

aarch64_ver_target_feature

The tracking issue for this feature is: [#44839](#)

abi_avr_interrupt

The tracking issue for this feature is: [#69664](#)

abi_c_cmse_nonsecure_call

The tracking issue for this feature is: [#81391](#)

The [TrustZone-M feature](#) is available for targets with the Armv8-M architecture profile ([thumbv8m](#) in their target name). LLVM, the Rust compiler and the linker are providing [support](#) for the TrustZone-M feature.

One of the things provided, with this unstable feature, is the [C-cmse-nonsecure-call](#) function ABI. This ABI is used on function pointers to non-secure code to mark a non-secure function call (see [section 5.5](#) for details).

With this ABI, the compiler will do the following to perform the call:

- save registers needed after the call to Secure memory
- clear all registers that might contain confidential information
- clear the Least Significant Bit of the function address
- branches using the BLXNS instruction

To avoid using the non-secure stack, the compiler will constrain the number and type of parameters/return value.

The [extern "C-cmse-nonsecure-call"](#) ABI is otherwise equivalent to the [extern "C"](#) ABI.

```
#![no_std]
#![feature(abi_c_cmse_nonsecure_call)]

#[no_mangle]
pub fn call_nonsecure_function(addr: usize) -> u32 {
    let non_secure_function =
        unsafe { core::mem::transmute::<usize, extern "C-cmse-nonsecure-call"
fn() -> u32>(addr) };
    non_secure_function()
}
```

```
$ rustc --emit asm --crate-type lib --target thumbv8m.main-none-eabi  
function.rs
```

```
call_nonsecure_function:
```

```
    .fnstart  
    .save    {r7, lr}  
    push     {r7, lr}  
    .setfp   r7, sp  
    mov      r7, sp  
    .pad     #16  
    sub      sp, #16  
    str      r0, [sp, #12]  
    ldr      r0, [sp, #12]  
    str      r0, [sp, #8]  
    b        .LBB0_1
```

```
.LBB0_1:
```

```
    ldr      r0, [sp, #8]  
    push.w   {r4, r5, r6, r7, r8, r9, r10, r11}  
    bic      r0, r0, #1  
    mov      r1, r0  
    mov      r2, r0  
    mov      r3, r0  
    mov      r4, r0  
    mov      r5, r0  
    mov      r6, r0  
    mov      r7, r0  
    mov      r8, r0  
    mov      r9, r0  
    mov      r10, r0  
    mov      r11, r0  
    mov      r12, r0  
    msr      apsr_nzcvq, r0  
    blxns    r0  
    pop.w    {r4, r5, r6, r7, r8, r9, r10, r11}  
    str      r0, [sp, #4]  
    b        .LBB0_2
```

```
.LBB0_2:
```

```
    ldr      r0, [sp, #4]  
    add      sp, #16  
    pop      {r7, pc}
```

abi_gpu_kernel

The tracking issue for this feature is: [#135467](#)

abi_msp430_interrupt

The tracking issue for this feature is: [#38487](#)

In the MSP430 architecture, interrupt handlers have a special calling convention. You can use the `"msp430-interrupt"` ABI to make the compiler apply the right calling convention to the interrupt handlers you define.

```
#![feature(abi_msp430_interrupt)]
#![no_std]

// Place the interrupt handler at the appropriate memory address
// (Alternatively, you can use `#[used]` and remove `pub` and `#[no_mangle]`)
#[link_section = "__interrupt_vector_10"]
#[no_mangle]
pub static TIM0_VECTOR: extern "msp430-interrupt" fn() = tim0;

// The interrupt handler
extern "msp430-interrupt" fn tim0() {
    // ..
}
```

```
$ msp430-elf-objdump -CD ./target/msp430/release/app
Disassembly of section __interrupt_vector_10:
```

```
0000fff2 <TIM0_VECTOR>:
    fff2:      00 c0          interrupt service routine at 0xc000
```

```
Disassembly of section .text:
```

```
0000c000 <int::tim0>:
    c000:      00 13          reti
```

abi_ptx

The tracking issue for this feature is: [#38788](#)

When emitting PTX code, all vanilla Rust functions (`fn`) get translated to "device" functions. These functions are *not* callable from the host via the CUDA API so a crate with only device functions is not too useful!

OTOH, "global" functions *can* be called by the host; you can think of them as the real public API of your crate. To produce a global function use the `"ptx-kernel"` ABI.

```
#![feature(abi_ptx)]
#![no_std]

pub unsafe extern "ptx-kernel" fn global_function() {
    device_function();
}

pub fn device_function() {
    // ..
}
```

```
$ xargo rustc --target nvptx64-nvidia-cuda --release -- --emit=asm

$ cat $(find -name '*.s')
//
// Generated by LLVM NVPTX Back-End
//

.version 3.2
.target sm_20
.address_size 64

        // .globl      _ZN6kernel15global_function17h46111ebe6516b382E

.visible .entry _ZN6kernel15global_function17h46111ebe6516b382E()
{

        ret;

}

        // .globl      _ZN6kernel15device_function17hd6a0e4993bbf3f78E
.visible .func _ZN6kernel15device_function17hd6a0e4993bbf3f78E()
{

        ret;

}
```


abi_riscv_interrupt

The tracking issue for this feature is: [#111889](#)

abi_unadjusted

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

abi_vectorcall

The tracking issue for this feature is: [#124485](#)

Adds support for the Windows `"vectorcall"` ABI, the equivalent of `__vectorcall` in MSVC.

```
extern "vectorcall" {
    fn add_f64s(x: f64, y: f64) -> f64;
}

fn main() {
    println!("{}", add_f64s(2.0, 4.0));
}
```

abi_x86_interrupt

The tracking issue for this feature is: [#40180](#)

adt_const_params

The tracking issue for this feature is: [#95174](#)

Allows for using more complex types for const parameters, such as structs or enums.

```
#![feature(adt_const_params)]
#![allow(incomplete_features)]

use std::marker::ConstParamTy;

#[derive(ConstParamTy, PartialEq, Eq)]
enum Foo {
    A,
    B,
    C,
}

#[derive(ConstParamTy, PartialEq, Eq)]
struct Bar {
    flag: bool,
}

fn is_foo_a_and_bar_true<const F: Foo, const B: Bar>() -> bool {
    match (F, B.flag) {
        (Foo::A, true) => true,
        _ => false,
    }
}
```

alloc_error_handler

The tracking issue for this feature is: [#51540](#)

allocator_internals

This feature does not have a tracking issue, it is an unstable implementation detail of the `global_allocator` feature not intended for use outside the compiler.

`allow_internal_unsafe`

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

allow_internal_unstable

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

anonymous_lifetime_in_impl_trait

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

arbitrary_self_types

The tracking issue for this feature is: [#44874]

Allows any type implementing `core::ops::Receiver<Target=T>` to be used as the type of `self` in a method belonging to `T`.

For example,

```
#![feature(arbitrary_self_types)]

struct A;

impl A {
    fn f(self: SmartPtr<Self>) -> i32 { 1 } // note self type
}

struct SmartPtr<T>(T);

impl<T> core::ops::Receiver for SmartPtr<T> {
    type Target = T;
}

fn main() {
    let smart_ptr = SmartPtr(A);
    assert_eq!(smart_ptr.f(), 1);
}
```

The `Receiver` trait has a blanket implementation for all `T: Deref`, so in fact things like this work too:

```
#![feature(arbitrary_self_types)]

use std::rc::Rc;

struct A;

impl A {
    fn f(self: Rc<Self>) -> i32 { 1 } // Rc implements Deref
}

fn main() {
    let smart_ptr = Rc::new(A);
    assert_eq!(smart_ptr.f(), 1);
}
```

Interestingly, that works even without the `arbitrary_self_types` feature

- but that's because certain types are *effectively* hard coded, including `Rc`. ("Hard coding" isn't quite true; they use a lang-item called `LegacyReceiver` to denote their special-ness in this way). With the `arbitrary_self_types` feature, their special-ness goes away, and custom smart pointers can achieve the same.

Changes to method lookup

Method lookup previously used to work by stepping through the `Deref` chain then using the resulting list of steps in two different ways:

- To identify types that might contribute methods via their `impl` blocks (inherent methods) or via traits
- To identify the types that the method receiver (`a` in the above examples) can be converted to.

With this feature, these lists are created by instead stepping through the `Receiver` chain. However, a note is kept about whether the type can be reached also via the `Deref` chain.

The full chain (via `Receiver` hops) is used for the first purpose (identifying relevant `impl` blocks and traits); whereas the shorter list (reachable via `Deref`) is used for the second purpose. That's because, to convert the method target (`a` in `a.b()`) to the self type, Rust may need to be able to use `Deref::deref`. Type conversions, then, can only proceed as far as the end of the `Deref` chain whereas the longer `Receiver` chain can be used to explore more places where useful methods might reside.

Types suitable for use as smart pointers

This feature allows the creation of customised smart pointers - for example your own equivalent to `Rc` or `Box` with whatever capabilities you like. Those smart pointers can either implement `Deref` (if it's safe to create a reference to the referent) or `Receiver` (if it isn't).

Either way, smart pointer types should mostly *avoid having methods*. Calling methods on a smart pointer leads to ambiguity about whether you're aiming for a method on the pointer, or on the referent.

Best practice is therefore to put smart pointer functionality into associated functions instead - that's what's done in all the smart pointer types within Rust's standard library which implement `Receiver`.

If you choose to add any methods to your smart pointer type, your users may run into errors from deshadowing, as described in the next section.

Avoiding shadowing

With or without this feature, Rust emits an error if it finds two method candidates, like this:

```
use std::pin::Pin;
use std::pin::pin;

struct A;

impl A {
    fn get_ref(self: Pin<&A>) {}
}

fn main() {
    let pinned_a: Pin<&A> = pin!(A).as_ref();
    let pinned_a: Pin<&A> = pinned_a.as_ref();
    pinned_a.get_ref(); // error[E0034]: multiple applicable items in scope
}
```

(this is why Rust's smart pointers are mostly carefully designed to avoid having methods at all, and shouldn't add new methods in future.)

With `arbitrary_self_types`, we take care to spot some other kinds of conflict:

```
#![feature(arbitrary_self_types)]

use std::pin::Pin;
use std::pin::pin;

struct A;

impl A {
    fn get_ref(self: &Pin<&A>) {} // note &Pin
}

fn main() {
    let pinned_a: Pin<&mut A> = pin!(A);
    let pinned_a: Pin<&A> = pinned_a.as_ref();
    pinned_a.get_ref();
}
```

This is to guard against the case where an inner (referent) type has a method of a given name, taking the smart pointer by reference, and then the smart pointer implementer adds a similar method taking self by value. As noted in the previous section, the safe option is simply not to add methods to smart pointers, and then these errors can't occur.

arbitrary_self_types_pointers

The tracking issue for this feature is: [#44874]

This extends the [arbitrary self types](#) feature to allow methods to receive `self` by pointer. For example:

```
#![feature(arbitrary_self_types_pointers)]

struct A;

impl A {
    fn m(self: *const Self) {}
}

fn main() {
    let a = A;
    let a_ptr: *const A = &a as *const A;
    a_ptr.m();
}
```

In general this is not advised: it's thought to be better practice to wrap raw pointers in a newtype wrapper which implements the `core::ops::Receiver` trait, then you need "only" the `arbitrary_self_types` feature. For example:

```
#![feature(arbitrary_self_types)]
#![allow(dead_code)]

struct A;

impl A {
    fn m(self: Wrapper<Self>) {} // can extract the pointer and do
    // what it needs
}

struct Wrapper<T>(*const T);

impl<T> core::ops::Receiver for Wrapper<T> {
    type Target = T;
}

fn main() {
    let a = A;
    let a_ptr: *const A = &a as *const A;
    let a_wrapper = Wrapper(a_ptr);
    a_wrapper.m();
}
```


arm_target_feature

The tracking issue for this feature is: [#44839](#)

asm_experimental_arch

The tracking issue for this feature is: [#93335](#)

This feature tracks `asm!` and `global_asm!` support for the following architectures:

- NVPTX
- PowerPC
- Hexagon
- MIPS32r2 and MIPS64r2
- wasm32
- BPF
- SPIR-V
- AVR
- MSP430
- M68k
- CSKY
- SPARC

Register classes

Architecture	Register class	Registers	LLVM constraint code
MIPS	<code>reg</code>	<code>\$(2-25)</code>	<code>r</code>
MIPS	<code>freg</code>	<code>\$f[0-31]</code>	<code>f</code>
NVPTX	<code>reg16</code>	None*	<code>h</code>
NVPTX	<code>reg32</code>	None*	<code>r</code>
NVPTX	<code>reg64</code>	None*	<code>l</code>
Hexagon	<code>reg</code>	<code>r[0-28]</code>	<code>r</code>
Hexagon	<code>preg</code>	<code>p[0-3]</code>	Only clobbers
PowerPC	<code>reg</code>	<code>r0, r[3-12], r[14-28]</code>	<code>r</code>
PowerPC	<code>reg_nonzero</code>	<code>r[3-12], r[14-28]</code>	<code>b</code>
PowerPC	<code>freg</code>	<code>f[0-31]</code>	<code>f</code>

PowerPC	<code>vreg</code>	<code>v[0-31]</code>	<code>v</code>
PowerPC	<code>cr</code>	<code>cr[0-7]</code> , <code>cr</code>	Only clobbers
PowerPC	<code>xer</code>	<code>xer</code>	Only clobbers
wasm32	<code>local</code>	None*	<code>r</code>
BPF	<code>reg</code>	<code>r[0-10]</code>	<code>r</code>
BPF	<code>wreg</code>	<code>w[0-10]</code>	<code>w</code>
AVR	<code>reg</code>	<code>r[2-25]</code> , <code>XH</code> , <code>XL</code> , <code>ZH</code> , <code>ZL</code>	<code>r</code>
AVR	<code>reg_upper</code>	<code>r[16-25]</code> , <code>XH</code> , <code>XL</code> , <code>ZH</code> , <code>ZL</code>	<code>d</code>
AVR	<code>reg_pair</code>	<code>r3r2</code> .. <code>r25r24</code> , <code>X</code> , <code>Z</code>	<code>r</code>
AVR	<code>reg_iw</code>	<code>r25r24</code> , <code>X</code> , <code>Z</code>	<code>w</code>
AVR	<code>reg_ptr</code>	<code>X</code> , <code>Z</code>	<code>e</code>
MSP430	<code>reg</code>	<code>r[0-15]</code>	<code>r</code>
M68k	<code>reg</code>	<code>d[0-7]</code> , <code>a[0-7]</code>	<code>r</code>
M68k	<code>reg_data</code>	<code>d[0-7]</code>	<code>d</code>
M68k	<code>reg_addr</code>	<code>a[0-3]</code>	<code>a</code>
CSKY	<code>reg</code>	<code>r[0-31]</code>	<code>r</code>
CSKY	<code>freg</code>	<code>f[0-31]</code>	<code>f</code>
SPARC	<code>reg</code>	<code>r[2-29]</code>	<code>r</code>
SPARC	<code>yreg</code>	<code>y</code>	Only clobbers

Notes:

- NVPTX doesn't have a fixed register set, so named registers are not supported.
- WebAssembly doesn't have registers, so named registers are not supported.

Register class supported types

Architecture	Register class	Target feature	Allowed types
--------------	----------------	----------------	---------------

MIPS32	<code>reg</code>	None	<code>i8</code> , <code>i16</code> , <code>i32</code> , <code>f32</code>
MIPS32	<code>freg</code>	None	<code>f32</code> , <code>f64</code>
MIPS64	<code>reg</code>	None	<code>i8</code> , <code>i16</code> , <code>i32</code> , <code>i64</code> , <code>f32</code> , <code>f64</code>
MIPS64	<code>freg</code>	None	<code>f32</code> , <code>f64</code>
NVPTX	<code>reg16</code>	None	<code>i8</code> , <code>i16</code>
NVPTX	<code>reg32</code>	None	<code>i8</code> , <code>i16</code> , <code>i32</code> , <code>f32</code>
NVPTX	<code>reg64</code>	None	<code>i8</code> , <code>i16</code> , <code>i32</code> , <code>f32</code> , <code>i64</code> , <code>f64</code>
Hexagon	<code>reg</code>	None	<code>i8</code> , <code>i16</code> , <code>i32</code> , <code>f32</code>
Hexagon	<code>preg</code>	N/A	Only clobbers
PowerPC	<code>reg</code>	None	<code>i8</code> , <code>i16</code> , <code>i32</code> , <code>i64</code> (powerpc64 only)
PowerPC	<code>reg_nonzero</code>	None	<code>i8</code> , <code>i16</code> , <code>i32</code> , <code>i64</code> (powerpc64 only)
PowerPC	<code>freg</code>	None	<code>f32</code> , <code>f64</code>
PowerPC	<code>vreg</code>	<code>altivec</code>	<code>i8x16</code> , <code>i16x8</code> , <code>i32x4</code> , <code>f32x4</code>
PowerPC	<code>vreg</code>	<code>vsx</code>	<code>f32</code> , <code>f64</code> , <code>i64x2</code> , <code>f64x2</code>
PowerPC	<code>cr</code>	N/A	Only clobbers
PowerPC	<code>xer</code>	N/A	Only clobbers
wasm32	<code>local</code>	None	<code>i8</code> <code>i16</code> <code>i32</code> <code>i64</code> <code>f32</code> <code>f64</code>
BPF	<code>reg</code>	None	<code>i8</code> <code>i16</code> <code>i32</code> <code>i64</code>
BPF	<code>wreg</code>	<code>alu32</code>	<code>i8</code> <code>i16</code> <code>i32</code>
AVR	<code>reg</code> , <code>reg_upper</code>	None	<code>i8</code>
AVR	<code>reg_pair</code> , <code>reg_iw</code> , <code>reg_ptr</code>	None	<code>i16</code>
MSP430	<code>reg</code>	None	<code>i8</code> , <code>i16</code>
M68k	<code>reg</code> , <code>reg_addr</code>	None	<code>i16</code> , <code>i32</code>
M68k	<code>reg_data</code>	None	<code>i8</code> , <code>i16</code> , <code>i32</code>
CSKY	<code>reg</code>	None	<code>i8</code> , <code>i16</code> , <code>i32</code>

CSKY	<code>freg</code>	None	<code>f32</code> ,
SPARC	<code>reg</code>	None	<code>i8</code> , <code>i16</code> , <code>i32</code> , <code>i64</code> (SPARC64 only)
SPARC	<code>yreg</code>	N/A	Only clobbers

Register aliases

Architecture	Base register	Aliases
Hexagon	<code>r29</code>	<code>sp</code>
Hexagon	<code>r30</code>	<code>fr</code>
Hexagon	<code>r31</code>	<code>lr</code>
PowerPC	<code>r1</code>	<code>sp</code>
PowerPC	<code>r31</code>	<code>fp</code>
PowerPC	<code>r[0-31]</code>	<code>[0-31]</code>
PowerPC	<code>f[0-31]</code>	<code>fr[0-31]</code>
BPF	<code>r[0-10]</code>	<code>w[0-10]</code>
AVR	<code>XH</code>	<code>r27</code>
AVR	<code>XL</code>	<code>r26</code>
AVR	<code>ZH</code>	<code>r31</code>
AVR	<code>ZL</code>	<code>r30</code>
MSP430	<code>r0</code>	<code>pc</code>
MSP430	<code>r1</code>	<code>sp</code>
MSP430	<code>r2</code>	<code>sr</code>
MSP430	<code>r3</code>	<code>cg</code>
MSP430	<code>r4</code>	<code>fp</code>
M68k	<code>a5</code>	<code>bp</code>
M68k	<code>a6</code>	<code>fp</code>
M68k	<code>a7</code>	<code>sp</code> , <code>usp</code> , <code>ssp</code> , <code>isp</code>
CSKY	<code>r[0-3]</code>	<code>a[0-3]</code>
CSKY	<code>r[4-11]</code>	<code>l[0-7]</code>
CSKY	<code>r[12-13]</code>	<code>t[0-1]</code>

CSKY	<code>r14</code>	<code>sp</code>
CSKY	<code>r15</code>	<code>lr</code>
CSKY	<code>r[16-17]</code>	<code>l[8-9]</code>
CSKY	<code>r[18-25]</code>	<code>t[2-9]</code>
CSKY	<code>r28</code>	<code>rgb</code>
CSKY	<code>r29</code>	<code>rtb</code>
CSKY	<code>r30</code>	<code>svbr</code>
CSKY	<code>r31</code>	<code>tls</code>
SPARC	<code>r[0-7]</code>	<code>g[0-7]</code>
SPARC	<code>r[8-15]</code>	<code>o[0-7]</code>
SPARC	<code>r[16-23]</code>	<code>l[0-7]</code>
SPARC	<code>r[24-31]</code>	<code>i[0-7]</code>

Notes:

- TI does not mandate a frame pointer for MSP430, but toolchains are allowed to use one; LLVM uses `r4`.

Unsupported registers

Architecture	Unsupported register	Reason
All	<code>sp</code> , <code>r14</code> / <code>o6</code> (SPARC)	The stack pointer must be restored to its original value at the end of an asm code block.
All	<code>fr</code> (Hexagon), <code>fp</code> (PowerPC), <code>\$fp</code> (MIPS), <code>y</code> (AVR), <code>r4</code> (MSP430), <code>a6</code> (M68k), <code>r30</code> / <code>i6</code> (SPARC)	The frame pointer cannot be used as an input or output.
All	<code>r19</code> (Hexagon), <code>r29</code> (PowerPC),	These are used internally by LLVM as "base pointer" for functions with

	<code>r30</code> (PowerPC)	complex stack frames.
MIPS	<code>\$0</code> or <code>\$zero</code>	This is a constant zero register which can't be modified.
MIPS	<code>\$1</code> or <code>\$at</code>	Reserved for assembler.
MIPS	<code>\$26 / \$k0</code> , <code>\$27 / \$k1</code>	OS-reserved registers.
MIPS	<code>\$28 / \$gp</code>	Global pointer cannot be used as inputs or outputs.
MIPS	<code>\$ra</code>	Return address cannot be used as inputs or outputs.
Hexagon	<code>lr</code>	This is the link register which cannot be used as an input or output.
PowerPC	<code>r2</code> , <code>r13</code>	These are system reserved registers.
PowerPC	<code>lr</code>	The link register cannot be used as an input or output.
PowerPC	<code>ctr</code>	The counter register cannot be used as an input or output.
PowerPC	<code>vrsave</code>	The vrsave register cannot be used as an input or output.
AVR	<code>r0</code> , <code>r1</code> , <code>r1r0</code>	Due to an issue in LLVM, the <code>r0</code> and <code>r1</code> registers cannot be used as inputs or outputs. If modified, they must be restored to their original values before the end of the block.
MSP430	<code>r0</code> , <code>r2</code> , <code>r3</code>	These are the program counter, status register, and constant generator respectively. Neither the status register nor constant generator can be written to.
M68k	<code>a4</code> , <code>a5</code>	Used internally by LLVM for the base pointer and global base pointer.
CSKY	<code>r7</code> , <code>r28</code>	Used internally by LLVM for the base pointer and global base pointer.
CSKY	<code>r8</code>	Used internally by LLVM for the frame pointer.
CSKY	<code>r14</code>	Used internally by LLVM for the stack

		pointer.
CSKY	<code>r15</code>	This is the link register.
CSKY	<code>r[26-30]</code>	Reserved by its ABI.
CSKY	<code>r31</code>	This is the TLS register.
SPARC	<code>r0 / g0</code>	This is always zero and cannot be used as inputs or outputs.
SPARC	<code>r1 / g1</code>	Used internally by LLVM.
SPARC	<code>r5 / g5</code>	Reserved for system. (SPARC32 only)
SPARC	<code>r6 / g6 , r7 / g7</code>	Reserved for system.
SPARC	<code>r31 / i7</code>	Return address cannot be used as inputs or outputs.

Template modifiers

Architecture	Register class	Modifier	Example output	LLVM modifier
MIPS	<code>reg</code>	None	<code>\$2</code>	None
MIPS	<code>freg</code>	None	<code>\$f0</code>	None
NVPTX	<code>reg16</code>	None	<code>rs0</code>	None
NVPTX	<code>reg32</code>	None	<code>r0</code>	None
NVPTX	<code>reg64</code>	None	<code>rd0</code>	None
Hexagon	<code>reg</code>	None	<code>r0</code>	None
PowerPC	<code>reg</code>	None	<code>0</code>	None
PowerPC	<code>reg_nonzero</code>	None	<code>3</code>	None
PowerPC	<code>freg</code>	None	<code>0</code>	None
PowerPC	<code>vreg</code>	None	<code>0</code>	None
SPARC	<code>reg</code>	None	<code>%o0</code>	None
CSKY	<code>reg</code>	None	<code>r0</code>	None
CSKY	<code>freg</code>	None	<code>f0</code>	None

Flags covered by `preserves_flags`

These flags registers must be restored upon exiting the asm block if the `preserves_flags` option is set:

- AVR
 - The status register `SREG` .
- MSP430
 - The status register `r2` .
- M68k
 - The condition code register `ccr` .
- SPARC
 - Integer condition codes (`icc` and `xcc`)
 - Floating-point condition codes (`fcc[0-3]`)
- CSKY
 - Condition/carry bit (C) in `PSR` .

asm_experimental_arch

The tracking issue for this feature is: [#133416](#)

This tracks support for additional registers in architectures where inline assembly is already stable.

Register classes

Architecture	Register class	Registers	LLVM constraint code
s390x	vreg	v[0-31]	v

Notes:

- s390x vreg is clobber-only in stable.

Register class supported types

Architecture	Register class	Target feature	Allowed types
s390x	vreg	vector	i32, f32, i64, f64, i128, f128, i8x16, i16x8, i32x4, i64x2, f32x4, f64x2

Register aliases

Architecture	Base register	Aliases
--------------	---------------	---------

Unsupported registers

Architecture	Unsupported register	Reason
--------------	----------------------	--------

Template modifiers

Architecture	Register class	Modifier	Example output	LLVM modifier
s390x	vreg	None	%v0	None

asm_goto

The tracking issue for this feature is: [#119364](#)

This feature adds a `label <block>` operand type to `asm!`.

Example:

```
unsafe {
  asm!(
    "jmp {}",
    label {
      println!("Jumped from asm!");
    }
  );
}
```

The block must have unit type or diverge. The block starts a new safety context, so despite outer `unsafe`, you need extra unsafe to perform unsafe operations within `label <block>`.

When `label <block>` is used together with `noreturn` option, it means that the assembly will not fallthrough. It's allowed to jump to a label within the assembly. In this case, the entire `asm!` expression will have an unit type as opposed to diverging, if not all label blocks diverge. The `asm!` expression still diverges if `noreturn` option is used and all label blocks diverge.

asm_goto_with_outputs

The tracking issue for this feature is: [#119364](#)

asm_unwind

The tracking issue for this feature is: [#93334](#)

This feature adds a `may_unwind` option to `asm!` which allows an `asm` block to unwind stack and be part of the stack unwinding process. This option is only supported by the LLVM backend right now.

associated_const_equality

The tracking issue for this feature is: [#92827](#)

associated_type_defaults

The tracking issue for this feature is: [#29661](#)

async_fn_in_dyn_trait

The tracking issue for this feature is: [#133119](#)

async_fn_track_caller

The tracking issue for this feature is: [#110011](#)

async_for_loop

The tracking issue for this feature is: [#118898](#)

async_trait_bounds

The tracking issue for this feature is: [#62290](#)

auto_traits

The tracking issue for this feature is [#13231](#)

The `auto_traits` feature gate allows you to define auto traits.

Auto traits, like `Send` or `Sync` in the standard library, are marker traits that are automatically implemented for every type, unless the type, or a type it contains, has explicitly opted out via a negative impl. (Negative impls are separately controlled by the `negative_impls` feature.)

```
impl !Trait for Type {}
```

Example:

```
#![feature(negative_impls)]
#![feature(auto_traits)]

auto trait Valid {}

struct True;
struct False;

impl !Valid for False {}

struct MaybeValid<T>(T);

fn must_be_valid<T: Valid>(_t: T) { }

fn main() {
    // works
    must_be_valid( MaybeValid(True) );

    // compiler error - trait bound not satisfied
    // must_be_valid( MaybeValid(False) );
}
```

Automatic trait implementations

When a type is declared as an `auto trait`, we will automatically create impls for every

struct/enum/union, unless an explicit impl is provided. These automatic impls contain a where clause for each field of the form `T: AutoTrait`, where `T` is the type of the field and `AutoTrait` is the auto trait in question. As an example, consider the struct `List` and the auto trait `Send`:

```
struct List<T> {
    data: T,
    next: Option<Box<List<T>>>,
}
```

Presuming that there is no explicit impl of `Send` for `List`, the compiler will supply an automatic impl of the form:

```
struct List<T> {
    data: T,
    next: Option<Box<List<T>>>,
}

unsafe impl<T> Send for List<T>
where
    T: Send, // from the field `data`
    Option<Box<List<T>>>: Send, // from the field `next`
{ }
```

Explicit impls may be either positive or negative. They take the form:

```
impl<...> AutoTrait for StructName<...> { }
impl<...> !AutoTrait for StructName<...> { }
```

Coinduction: Auto traits permit cyclic matching

Unlike ordinary trait matching, auto traits are **coinductive**. This means, in short, that cycles which occur in trait matching are considered ok. As an example, consider the recursive struct `List` introduced in the previous section. In attempting to determine whether `List: Send`, we would wind up in a cycle: to apply the impl, we must show that `Option<Box<List>>: Send`, which will in turn require `Box<List>: Send` and then finally `List: Send` again. Under ordinary trait matching, this cycle would be an error, but for an auto trait it is considered a successful match.

Items

Auto traits cannot have any trait items, such as methods or associated types. This ensures that we can generate default implementations.

Supertraits

Auto traits cannot have supertraits. This is for soundness reasons, as the interaction of coinduction with implied bounds is difficult to reconcile.

avx512_target_feature

The tracking issue for this feature is: [#44839](#)

box_patterns

The tracking issue for this feature is: [#29641](#)

Box patterns let you match on `Box<T>` s:

```
#![feature(box_patterns)]

fn main() {
    let b = Some(Box::new(5));
    match b {
        Some(box n) if n < 0 => {
            println!("Box contains negative number {n}");
        },
        Some(box n) if n >= 0 => {
            println!("Box contains non-negative number {n}");
        },
        None => {
            println!("No box");
        },
        _ => unreachable!()
    }
}
```

bpf_target_feature

The tracking issue for this feature is: [#44839](#)

builtin_syntax

The tracking issue for this feature is: [#110680](#)

c_variadic

The tracking issue for this feature is: [#44930](#)

The `c_variadic` language feature enables C-variadic functions to be defined in Rust. They may be called both from within Rust and via FFI.

Examples

```
#![feature(c_variadic)]

pub unsafe extern "C" fn add(n: usize, mut args: ...) -> usize {
    let mut sum = 0;
    for _ in 0..n {
        sum += args.arg::<usize>();
    }
    sum
}
```

cfg_boolean_literals

The tracking issue for this feature is: [#131204](#)

The `cfg_boolean_literals` feature makes it possible to use the `true` / `false` literal as `cfg` predicate. They always evaluate to `true`/`false` respectively.

Examples

```
#![feature(cfg_boolean_literals)]

#[cfg(true)]
const A: i32 = 5;

#[cfg(all(false))]
const A: i32 = 58 * 89;
```

cfg_contract_checks

The tracking issue for this feature is: [#128044](#)

cfg_emscripten_wasm_eh

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

cfg_overflow_checks

The tracking issue for this feature is: [#111466](#)

cfg_relocation_model

The tracking issue for this feature is: [#114929](#)

cfg_sanitiz

The tracking issue for this feature is: [#39699](#)

The `cfg_sanitiz` feature makes it possible to execute different code depending on whether a particular sanitizer is enabled or not.

Examples

```
#![feature(cfg_sanitiz)]

#[cfg(sanitiz = "thread")]
fn a() {
    // ...
}

#[cfg(not(sanitiz = "thread"))]
fn a() {
    // ...
}

fn b() {
    if cfg!(sanitiz = "leak") {
        // ...
    } else {
        // ...
    }
}
```

cfg_sanitizer_cfi

The tracking issue for this feature is: [#89653](#)

cfg_target_compact

The tracking issue for this feature is: [#96901](#)

cfg_target_has_atomic

The tracking issue for this feature is: [#94039](#)

cfg_target_has_atomic_equal_alignment

The tracking issue for this feature is: [#93822](#)

cfg_target_thread_local

The tracking issue for this feature is: [#29594](#)

cfg_ub_checks

The tracking issue for this feature is: [#123499](#)

cfg_version

The tracking issue for this feature is: [#64796](#)

The `cfg_version` feature makes it possible to execute different code depending on the compiler version. It will return true if the compiler version is greater than or equal to the specified version.

Examples

```
#![feature(cfg_version)]

#[cfg(version("1.42"))] // 1.42 and above
fn a() {
    // ...
}

#[cfg(not(version("1.42")))] // 1.41 and below
fn a() {
    // ...
}

fn b() {
    if cfg!(version("1.42")) {
        // ...
    } else {
        // ...
    }
}
```

cfi_encoding

The tracking issue for this feature is: [#89653](#)

The `cfi_encoding` feature allows the user to define a CFI encoding for a type. It allows the user to use a different names for types that otherwise would be required to have the same name as used in externally defined C functions.

Examples

```
#![feature(cfi_encoding, extern_types)]

#[cfi_encoding = "3Foo"]
pub struct Type1(i32);

extern {
    #[cfi_encoding = "3Bar"]
    type Type2;
}
```

closure_lifetime_binder

The tracking issue for this feature is: [#97362](#)

closure_track_caller

The tracking issue for this feature is: [#87417](#)

Allows using the `#[track_caller]` attribute on closures and coroutines. Calls made to the closure or coroutine will have caller information available through `std::panic::Location::caller()`, just like using `#[track_caller]` on a function.

cmse_nonsecure_entry

The tracking issue for this feature is: [#75835](#)

The [TrustZone-M feature](#) is available for targets with the Armv8-M architecture profile (`thumbv8m` in their target name). LLVM, the Rust compiler and the linker are providing [support](#) for the TrustZone-M feature.

One of the things provided, with this unstable feature, is the `C-cmse-nonsecure-entry` ABI. This ABI marks a Secure function as an entry function (see [section 5.4](#) for details). With this ABI, the compiler will do the following:

- add a special symbol on the function which is the `__acle_se_` prefix and the standard function name
- constrain the number of parameters to avoid using the Non-Secure stack
- before returning from the function, clear registers that might contain Secure information
- use the `BXNS` instruction to return

Because the stack can not be used to pass parameters, there will be compilation errors if:

- the total size of all parameters is too big (for example more than four 32 bits integers)
- the entry function is not using a C ABI

The special symbol `__acle_se_` will be used by the linker to generate a secure gateway veneer.

```
#![no_std]
#![feature(cmse_nonsecure_entry)]

#[no_mangle]
pub extern "C-cmse-nonsecure-entry" fn entry_function(input: u32) -> u32 {
    input + 6
}
```

```
$ rustc --emit obj --crate-type lib --target thumbv8m.main-none-eabi
function.rs
```

```
$ arm-none-eabi-objdump -D function.o
```

```
00000000 <entry_function>:
```

```

0:  b580      push    {r7, lr}
2:  466f      mov     r7, sp
4:  b082      sub     sp, #8
6:  9001      str     r0, [sp, #4]
8:  1d81      adds   r1, r0, #6
a:  460a      mov     r2, r1
c:  4281      cmp     r1, r0
e:  9200      str     r2, [sp, #0]
10: d30b      bcc.n   2a <entry_function+0x2a>
12: e7ff      b.n     14 <entry_function+0x14>
14: 9800      ldr     r0, [sp, #0]
16: b002      add     sp, #8
18: e8bd 4080  ldmia.w sp!, {r7, lr}
1c: 4671      mov     r1, lr
1e: 4672      mov     r2, lr
20: 4673      mov     r3, lr
22: 46f4      mov     ip, lr
24: f38e 8800  msr     CPSR_f, lr
28: 4774      bxns    lr
2a: f240 0000  movw    r0, #0
2e: f2c0 0000  movt    r0, #0
32: f240 0200  movw    r2, #0
36: f2c0 0200  movt    r2, #0
3a: 211c      movs    r1, #28
3c: f7ff fffe  bl      0
<_ZN4core9panicking5panic17h5c028258ca2fb3f5E>
40: defe      udf     #254 ; 0xfe
```

compiler_builtins

This feature is internal to the Rust compiler and is not intended for general use.

const_async_blocks

The tracking issue for this feature is: [#85368](#)

const_closures

The tracking issue for this feature is: [#106003](#)

const_destruct

The tracking issue for this feature is: [#133214](#)

const_for

The tracking issue for this feature is: [#87575](#)

const_precise_live_drops

The tracking issue for this feature is: [#73255](#)

const_trait_impl

The tracking issue for this feature is: [#67792](#)

const_try

The tracking issue for this feature is: [#74935](#)

contracts

The tracking issue for this feature is: [#128044](#)

contracts_internals

The tracking issue for this feature is: [#128044](#)

coroutine_clone

The tracking issue for this feature is: [#95360](#)

coroutines

The tracking issue for this feature is: [#43122](#)

The `coroutines` feature gate in Rust allows you to define coroutine or coroutine literals. A coroutine is a "resumable function" that syntactically resembles a closure but compiles to much different semantics in the compiler itself. The primary feature of a coroutine is that it can be suspended during execution to be resumed at a later date. Coroutines use the `yield` keyword to "return", and then the caller can `resume` a coroutine to resume execution just after the `yield` keyword.

Coroutines are an extra-unstable feature in the compiler right now. Added in [RFC 2033](#) they're mostly intended right now as a information/constraint gathering phase. The intent is that experimentation can happen on the nightly compiler before actual stabilization. A further RFC will be required to stabilize coroutines and will likely contain at least a few small tweaks to the overall design.

A syntactical example of a coroutine is:

```
#![feature(coroutines, coroutine_trait, stmt_expr_attributes)]

use std::ops::{Coroutine, CoroutineState};
use std::pin::Pin;

fn main() {
    let mut coroutine = #[coroutine] || {
        yield 1;
        return "foo"
    };

    match Pin::new(&mut coroutine).resume(()) {
        CoroutineState::Yielded(1) => {}
        _ => panic!("unexpected value from resume"),
    }
    match Pin::new(&mut coroutine).resume(()) {
        CoroutineState::Complete("foo") => {}
        _ => panic!("unexpected value from resume"),
    }
}
```

Coroutines are closure-like literals which are annotated with `#[coroutine]` and can contain a `yield` statement. The `yield` statement takes an optional expression of a value to yield out of the coroutine. All coroutine literals implement the `Coroutine` trait in

the `std::ops` module. The `Coroutine` trait has one main method, `resume`, which resumes execution of the coroutine at the previous suspension point.

An example of the control flow of coroutines is that the following example prints all numbers in order:

```
#![feature(coroutines, coroutine_trait, stmt_expr_attributes)]

use std::ops::Coroutine;
use std::pin::Pin;

fn main() {
    let mut coroutine = #[coroutine] || {
        println!("2");
        yield;
        println!("4");
    };

    println!("1");
    Pin::new(&mut coroutine).resume(());
    println!("3");
    Pin::new(&mut coroutine).resume(());
    println!("5");
}
```

At this time the main use case of coroutines is an implementation primitive for `async` / `await` and `gen` syntax, but coroutines will likely be extended to other primitives in the future. Feedback on the design and usage is always appreciated!

The Coroutine trait

The `Coroutine` trait in `std::ops` currently looks like:

```
pub trait Coroutine<R = ()> {
    type Yield;
    type Return;
    fn resume(self: Pin<&mut Self>, resume: R) -> CoroutineState<Self::Yield,
Self::Return>;
}
```

The `Coroutine::Yield` type is the type of values that can be yielded with the `yield` statement. The `Coroutine::Return` type is the returned type of the coroutine. This is

typically the last expression in a coroutine's definition or any value passed to `return` in a coroutine. The `resume` function is the entry point for executing the `Coroutine` itself.

The return value of `resume`, `CoroutineState`, looks like:

```
pub enum CoroutineState<Y, R> {  
    Yielded(Y),  
    Complete(R),  
}
```

The `Yielded` variant indicates that the coroutine can later be resumed. This corresponds to a `yield` point in a coroutine. The `Complete` variant indicates that the coroutine is complete and cannot be resumed again. Calling `resume` after a coroutine has returned `Complete` will likely result in a panic of the program.

Closure-like semantics

The closure-like syntax for coroutines alludes to the fact that they also have closure-like semantics. Namely:

- When created, a coroutine executes no code. A closure literal does not actually execute any of the closure's code on construction, and similarly a coroutine literal does not execute any code inside the coroutine when constructed.
- Coroutines can capture outer variables by reference or by move, and this can be tweaked with the `move` keyword at the beginning of the closure. Like closures all coroutines will have an implicit environment which is inferred by the compiler. Outer variables can be moved into a coroutine for use as the coroutine progresses.
- Coroutine literals produce a value with a unique type which implements the `std::ops::Coroutine` trait. This allows actual execution of the coroutine through the `Coroutine::resume` method as well as also naming it in return types and such.
- Traits like `Send` and `Sync` are automatically implemented for a `Coroutine` depending on the captured variables of the environment. Unlike closures, coroutines also depend on variables live across suspension points. This means that although the ambient environment may be `Send` or `Sync`, the coroutine itself may not be due to internal variables live across `yield` points being not-`Send` or not-`Sync`. Note that coroutines do not implement traits like `Copy` or `Clone` automatically.

- Whenever a coroutine is dropped it will drop all captured environment variables.

Coroutines as state machines

In the compiler, coroutines are currently compiled as state machines. Each `yield` expression will correspond to a different state that stores all live variables over that suspension point. Resumption of a coroutine will dispatch on the current state and then execute internally until a `yield` is reached, at which point all state is saved off in the coroutine and a value is returned.

Let's take a look at an example to see what's going on here:

```
#![feature(coroutines, coroutine_trait, stmt_expr_attributes)]

use std::ops::Coroutine;
use std::pin::Pin;

fn main() {
    let ret = "foo";
    let mut coroutine = #[coroutine] move || {
        yield 1;
        return ret
    };

    Pin::new(&mut coroutine).resume(());
    Pin::new(&mut coroutine).resume(());
}
```

This coroutine literal will compile down to something similar to:

```

#![feature(arbitrary_self_types, coroutine_trait)]

use std::ops::{Coroutine, CoroutineState};
use std::pin::Pin;

fn main() {
    let ret = "foo";
    let mut coroutine = {
        enum __Coroutine {
            Start(&'static str),
            Yield1(&'static str),
            Done,
        }

        impl Coroutine for __Coroutine {
            type Yield = i32;
            type Return = &'static str;

            fn resume(mut self: Pin<&mut Self>, resume: ()) ->
CoroutineState<i32, &'static str> {
                use std::mem;
                match mem::replace(&mut *self, __Coroutine::Done) {
                    __Coroutine::Start(s) => {
                        *self = __Coroutine::Yield1(s);
                        CoroutineState::Yielded(1)
                    }

                    __Coroutine::Yield1(s) => {
                        *self = __Coroutine::Done;
                        CoroutineState::Complete(s)
                    }

                    __Coroutine::Done => {
                        panic!("coroutine resumed after completion")
                    }
                }
            }
        }

        __Coroutine::Start(ret)
    };

    Pin::new(&mut coroutine).resume(());
    Pin::new(&mut coroutine).resume(());
}

```

Notably here we can see that the compiler is generating a fresh type, `__Coroutine` in this case. This type has a number of states (represented here as an `enum`) corresponding to each of the conceptual states of the coroutine. At the beginning we're closing over our

outer variable `foo` and then that variable is also live over the `yield` point, so it's stored in both states.

When the coroutine starts it'll immediately yield 1, but it saves off its state just before it does so indicating that it has reached the yield point. Upon resuming again we'll execute the `return ret` which returns the `Complete` state.

Here we can also note that the `Done` state, if resumed, panics immediately as it's invalid to resume a completed coroutine. It's also worth noting that this is just a rough desugaring, not a normative specification for what the compiler does.

coverage_attribute

The tracking issue for this feature is: [#84605](#)

The `coverage` attribute can be used to selectively disable coverage instrumentation in an annotated function. This might be useful to:

- Avoid instrumentation overhead in a performance critical function
- Avoid generating coverage for a function that is not meant to be executed, but still target 100% coverage for the rest of the program.

Example

```
#![feature(coverage_attribute)]

// `foo()` will get coverage instrumentation (by default)
fn foo() {
    // ...
}

#[coverage(off)]
fn bar() {
    // ...
}
```


csky_target_feature

The tracking issue for this feature is: [#44839](#)

custom_inner_attributes

The tracking issue for this feature is: [#54726](#)

custom_mir

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

custom_test_frameworks

The tracking issue for this feature is: [#50297](#)

The `custom_test_frameworks` feature allows the use of `#[test_case]` and `#![test_runner]`. Any function, const, or static can be annotated with `#[test_case]` causing it to be aggregated (like `#[test]`) and be passed to the test runner determined by the `#![test_runner]` crate attribute.

```
#![feature(custom_test_frameworks)]
#![test_runner(my_runner)]

fn my_runner(tests: &[&i32]) {
    for t in tests {
        if **t == 0 {
            println!("PASSED");
        } else {
            println!("FAILED");
        }
    }
}

#[test_case]
const WILL_PASS: i32 = 0;

#[test_case]
const WILL_FAIL: i32 = 4;
```

decl_macro

The tracking issue for this feature is: [#39412](#)

default_field_values

The tracking issue for this feature is: [#132162](#)

The RFC for this feature is: [#3681](#)

The `default_field_values` feature allows users to specify a const value for individual fields in struct definitions, allowing those to be omitted from initializers.

Examples

```
#![feature(default_field_values)]

#[derive(Default)]
struct Pet {
    name: Option<String>, // impl Default for Pet will use Default::default()
    // for name
    age: i128 = 42, // impl Default for Pet will use the literal 42 for age
}

fn main() {
    let a = Pet { name: Some(String::new()), .. }; // Pet { name: Some(""),
    // age: 42 }
    let b = Pet::default(); // Pet { name: None, age: 42 }
    assert_eq!(a.age, b.age);
    // The following would be a compilation error: `name` needs to be
    // specified
    // let _ = Pet { .. };
}
```

#[derive(Default)]

When deriving Default, the provided values are then used. On enum variants, the variant must still be marked with `#[default]` and have all its fields with default values.

```
#![feature(default_field_values)]

#[derive(Default)]
enum A {
    #[default]
    B {
        x: i32 = 0,
        y: i32 = 0,
    },
    C,
}
```

Enum variants

This feature also supports enum variants for both specifying default values and `#[derive(Default)]`.

Interaction with `#[non_exhaustive]`

A struct or enum variant marked with `#[non_exhaustive]` is not allowed to have default field values.

Lints

When manually implementing the `Default` trait for a type that has default field values, if any of these are overridden in the impl the `default_overrides_default_fields` lint will trigger. This lint is in place to avoid surprising diverging behavior between `S { .. }` and `S::default()`, where using the same type in both ways could result in different values. The appropriate way to write a manual `Default` implementation is to use the functional update syntax:

```
#![feature(default_field_values)]

struct Pet {
    name: String,
    age: i128 = 42, // impl Default for Pet will use the literal 42 for age
}

impl Default for Pet {
    fn default() -> Pet {
        Pet {
            name: "no-name".to_string(),
            ..
        }
    }
}
```


deprecated_safe

The tracking issue for this feature is: [#94978](#)

deprecated_suggestion

The tracking issue for this feature is: [#94785](#)

deref_patterns

The tracking issue for this feature is: [#87121](#)

doc_auto_cfg

The tracking issue for this feature is: [#43781](#)

doc_cfg

The tracking issue for this feature is: [#43781](#)

The `doc_cfg` feature allows an API be documented as only available in some specific platforms. This attribute has two effects:

1. In the annotated item's documentation, there will be a message saying "Available on (platform) only".
2. The item's doc-tests will only run on the specific platform.

In addition to allowing the use of the `#[doc(cfg)]` attribute, this feature enables the use of a special conditional compilation flag, `#[cfg(doc)]`, set whenever building documentation on your crate.

This feature was introduced as part of PR [#43348](#) to allow the platform-specific parts of the standard library be documented.

```
#![feature(doc_cfg)]

#[cfg(any(windows, doc))]
#[doc(cfg(windows))]
/// The application's icon in the notification area (a.k.a. system tray).
///
/// # Examples
///
/// ```no_run
/// extern crate my_awesome_ui_library;
/// use my_awesome_ui_library::current_app;
/// use my_awesome_ui_library::windows::notification;
///
/// let icon = current_app().get::<notification::Icon>();
/// icon.show();
/// icon.show_message("Hello");
/// ```
pub struct Icon {
    // ...
}
```

doc_cfg_hide

The tracking issue for this feature is: [#43781](#)

doc_masked

The tracking issue for this feature is: [#44027](#)

The `doc_masked` feature allows a crate to exclude types from a given crate from appearing in lists of trait implementations. The specifics of the feature are as follows:

1. When rustdoc encounters an `extern crate` statement annotated with a `#
[doc(masked)]` attribute, it marks the crate as being masked.
2. When listing traits a given type implements, rustdoc ensures that traits from masked crates are not emitted into the documentation.
3. When listing types that implement a given trait, rustdoc ensures that types from masked crates are not emitted into the documentation.

This feature was introduced in PR [#44026](#) to ensure that compiler-internal and implementation-specific types and traits were not included in the standard library's documentation. Such types would introduce broken links into the documentation.

doc_notable_trait

The tracking issue for this feature is: [#45040](#)

The `doc_notable_trait` feature allows the use of the `#[doc(notable_trait)]` attribute, which will display the trait in a "Notable traits" dialog for functions returning types that implement the trait. For example, this attribute is applied to the `Iterator`, `Future`, `io::Read`, and `io::Write` traits in the standard library.

You can do this on your own traits like so:

```
#![feature(doc_notable_trait)]

#[doc(notable_trait)]
pub trait MyTrait {}

pub struct MyStruct;
impl MyTrait for MyStruct {}

/// The docs for this function will have a button that displays a dialog
about
/// `MyStruct` implementing `MyTrait`.
pub fn my_fn() -> MyStruct { MyStruct }
```

This feature was originally implemented in PR [#45039](#).

See also its documentation in [the rustdoc book](#).

dropck_eyepatch

The tracking issue for this feature is: [#34761](#)

dyn_compatible_for_dispatch

The tracking issue for this feature is: [#43561](#)

dyn_star

The tracking issue for this feature is: [#102425](#)

ermsb_target_feature

The tracking issue for this feature is: [#44839](#)

exhaustive_patterns

The tracking issue for this feature is: [#51085](#)

explicit_tail_calls

The tracking issue for this feature is: [#112788](#)

extended_varargs_abi_support

The tracking issue for this feature is: [#100189](#)

This feature adds the possibility of using `sysv64`, `win64` or `efiapi` calling conventions on functions with varargs.

extern_system_varargs

The tracking issue for this feature is: [#136946](#)

extern_types

The tracking issue for this feature is: [#43467](#)

f128

The tracking issue for this feature is: [#116909](#)

Enable the `f128` type for IEEE 128-bit floating numbers (quad precision).

f16

The tracking issue for this feature is: [#116909](#)

Enable the `f16` type for IEEE 16-bit floating numbers (half precision).

ffi_const

The tracking issue for this feature is: [#58328](#)

The `#[ffi_const]` attribute applies clang's `const` attribute to foreign functions declarations.

That is, `#[ffi_const]` functions shall have no effects except for its return value, which can only depend on the values of the function parameters, and is not affected by changes to the observable state of the program.

Applying the `#[ffi_const]` attribute to a function that violates these requirements is undefined behaviour.

This attribute enables Rust to perform common optimizations, like sub-expression elimination, and it can avoid emitting some calls in repeated invocations of the function with the same argument values regardless of other operations being performed in between these functions calls (as opposed to `#[ffi_pure]` functions).

Pitfalls

A `#[ffi_const]` function can only read global memory that would not affect its return value for the whole execution of the program (e.g. immutable global memory). `#[ffi_const]` functions are referentially-transparent and therefore more strict than `#[ffi_pure]` functions.

A common pitfall involves applying the `#[ffi_const]` attribute to a function that reads memory through pointer arguments which do not necessarily point to immutable global memory.

A `#[ffi_const]` function that returns `unit` has no effect on the abstract machine's state, and a `#[ffi_const]` function cannot be `#[ffi_pure]`.

A `#[ffi_const]` function must not diverge, neither via a side effect (e.g. a call to `abort`) nor by infinite loops.

When translating C headers to Rust FFI, it is worth verifying for which targets the `const` attribute is enabled in those headers, and using the appropriate `cfg` macros in the Rust

side to match those definitions. While the semantics of `const` are implemented identically by many C and C++ compilers, e.g., clang, GCC, ARM C/C++ compiler, IBM ILE C/C++, etc. they are not necessarily implemented in this way on all of them. It is therefore also worth verifying that the semantics of the C toolchain used to compile the binary being linked against are compatible with those of the `#[ffi_const]`.

ffi_pure

The tracking issue for this feature is: [#58329](#)

The `#[ffi_pure]` attribute applies clang's `pure` attribute to foreign functions declarations.

That is, `#[ffi_pure]` functions shall have no effects except for its return value, which shall not change across two consecutive function calls with the same parameters.

Applying the `#[ffi_pure]` attribute to a function that violates these requirements is undefined behavior.

This attribute enables Rust to perform common optimizations, like sub-expression elimination and loop optimizations. Some common examples of pure functions are `strlen` or `memcmp`.

These optimizations are only applicable when the compiler can prove that no program state observable by the `#[ffi_pure]` function has changed between calls of the function, which could alter the result. See also the `#[ffi_const]` attribute, which provides stronger guarantees regarding the allowable behavior of a function, enabling further optimization.

Pitfalls

A `#[ffi_pure]` function can read global memory through the function parameters (e.g. pointers), globals, etc. `#[ffi_pure]` functions are not referentially-transparent, and are therefore more relaxed than `#[ffi_const]` functions.

However, accessing global memory through volatile or atomic reads can violate the requirement that two consecutive function calls shall return the same value.

A `pure` function that returns unit has no effect on the abstract machine's state.

A `#[ffi_pure]` function must not diverge, neither via a side effect (e.g. a call to `abort`) nor by infinite loops.

When translating C headers to Rust FFI, it is worth verifying for which targets the `pure`

attribute is enabled in those headers, and using the appropriate `cfg` macros in the Rust side to match those definitions. While the semantics of `pure` are implemented identically by many C and C++ compilers, e.g., clang, GCC, ARM C/C++ compiler, IBM ILE C/C++, etc. they are not necessarily implemented in this way on all of them. It is therefore also worth verifying that the semantics of the C toolchain used to compile the binary being linked against are compatible with those of the `#[ffi_pure]`.

fmt_debug

The tracking issue for this feature is: [#129709](#)

fn_align

The tracking issue for this feature is: [#82232](#)

fn_delegation

The tracking issue for this feature is: [#118212](#)

freeze_impls

The tracking issue for this feature is: [#121675](#)

fundamental

The tracking issue for this feature is: [#29635](#)

gen_blocks

The tracking issue for this feature is: [#117078](#)

generic_arg_infer

The tracking issue for this feature is: [#85077](#)

generic_assert

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

generic_const_exprs

The tracking issue for this feature is: [#76560](#)

generic_const_items

The tracking issue for this feature is: [#113521](#)

generic_pattern_types

The tracking issue for this feature is: [#136574](#)

global_registration

The tracking issue for this feature is: [#125119](#)

guard_patterns

The tracking issue for this feature is: [#129967](#)

half_open_range_patterns_in_slice

The tracking issue for this feature is: [#67264](#) It is a future part of the `exclusive_range_pattern` feature, tracked at [#37854](#).

This feature allow using top-level half-open range patterns in slices.

```
#![feature(half_open_range_patterns_in_slices)]

fn main() {
    let xs = [13, 1, 5, 2, 3, 1, 21, 8];
    let [a @ 3.., b @ ..3, c @ 4..6, ..] = xs else { return; };
}
```

Note that this feature is not required if the patterns are wrapped between parenthesis.

```
fn main() {
    let xs = [13, 1];
    let [(a @ 3..), c] = xs else { return; };
}
```

hexagon_target_feature

The tracking issue for this feature is: [#44839](#)

if_let_guard

The tracking issue for this feature is: [#51114](#)

impl_trait_in_assoc_type

The tracking issue for this feature is: [#63063](#)

impl_trait_in_bindings

The tracking issue for this feature is: [#63065](#)

impl_trait_in_fn_trait_return

The tracking issue for this feature is: [#99697](#)

import_trait_associated_functions

The tracking issue for this feature is: [#134691](#)

inherent_associated_types

The tracking issue for this feature is: [#8995](#)

inline_const_pat

The tracking issue for this feature is: [#76001](#)

This feature allows you to use inline constant expressions in pattern position:

```
#![feature(inline_const_pat)]

const fn one() -> i32 { 1 }

let some_int = 3;
match some_int {
    const { 1 + 2 } => println!("Matched 1 + 2"),
    const { one() } => println!("Matched const fn returning 1"),
    _ => println!("Didn't match anything :("),
}
```

intra-doc-pointers

The tracking issue for this feature is: [#80896](#)

Rustdoc does not currently allow disambiguating between `*const` and `*mut`, and raw pointers in intra-doc links are unstable until it does.

```
#![feature(intra_doc_pointers)]  
//! [pointer::add]
```

intrinsic

The tracking issue for this feature is: None.

Intrinsics are rarely intended to be stable directly, but are usually exported in some sort of stable manner. Prefer using the stable interfaces to the intrinsic directly when you can.

Intrinsics with fallback logic

Many intrinsics can be written in pure rust, albeit inefficiently or without supporting some features that only exist on some backends. Backends can simply not implement those intrinsics without causing any code miscompilations or failures to compile. All intrinsic fallback bodies are automatically made cross-crate inlineable (like `#[inline]`) by the codegen backend, but not the MIR inliner.

```
#![feature(intrinsics)]
#![allow(internal_features)]

#[rustc_intrinsic]
const unsafe fn const_deallocate(_ptr: *mut u8, _size: usize, _align: usize)
{}
```

Since these are just regular functions, it is perfectly ok to create the intrinsic twice:

```
#![feature(intrinsics)]
#![allow(internal_features)]

#[rustc_intrinsic]
const unsafe fn const_deallocate(_ptr: *mut u8, _size: usize, _align: usize)
{}
```

```
mod foo {
    #[rustc_intrinsic]
    const unsafe fn const_deallocate(_ptr: *mut u8, _size: usize, _align:
    usize) {
        panic!("noisy const dealloc")
    }
}
```

The behaviour on backends that override the intrinsic is exactly the same. On other backends, the intrinsic behaviour depends on which implementation is called, just like with any regular function.

Intrinsics lowered to MIR instructions

Various intrinsics have native MIR operations that they correspond to. Instead of requiring backends to implement both the intrinsic and the MIR operation, the `lower_intrinsics` pass will convert the calls to the MIR operation. Backends do not need to know about these intrinsics at all. These intrinsics only make sense without a body, and can either be declared as a "rust-intrinsic" or as a `#[rustc_intrinsic]`. The body is never used, as calls to the intrinsic do not exist anymore after MIR analyses.

Intrinsics without fallback logic

These must be implemented by all backends.

`#[rustc_intrinsic]` declarations

These are written like intrinsics with fallback bodies, but the body is irrelevant. Use `loop {}` for the body or call the intrinsic recursively and add `#[rustc_intrinsic_must_be_overridden]` to the function to ensure that backends don't invoke the body.

Legacy extern ABI based intrinsics

These are imported as if they were FFI functions, with the special `rust-intrinsic` ABI. For example, if one was in a freestanding context, but wished to be able to `transmute` between types, and perform efficient pointer arithmetic, one would import those functions via a declaration like


```
#![feature(intrinsics)]
#![allow(internal_features)]

extern "rust-intrinsic" {
    fn transmute<T, U>(x: T) -> U;

    fn arith_offset<T>(dst: *const T, offset: isize) -> *const T;
}
```

As with any other FFI functions, these are by default always `unsafe` to call. You can add `#[rustc_safe_intrinsic]` to the intrinsic to make it safe to call.

keylocker_x86

The tracking issue for this feature is: [#134813](#)

lahfsahf_target_feature

The tracking issue for this feature is: [#44839](#)

lang_items

The tracking issue for this feature is: None.

The `rustc` compiler has certain pluggable operations, that is, functionality that isn't hard-coded into the language, but is implemented in libraries, with a special marker to tell the compiler it exists. The marker is the attribute `#[lang = "..."]` and there are various different values of `...`, i.e. various different 'lang items'. Most of them can only be defined once.

Lang items are loaded lazily by the compiler; e.g. if one never uses `Box` then there is no need to define a function for `exchange_malloc`. `rustc` will emit an error when an item is needed but not found in the current crate or any that it depends on.

Some features provided by lang items:

- overloadable operators via traits: the traits corresponding to the `==`, `<`, dereferencing (`*`) and `+` (etc.) operators are all marked with lang items; those specific four are `eq`, `partial_ord`, `deref / deref_mut`, and `add` respectively.
- panicking: the `panic` and `panic_impl` lang items, among others.
- stack unwinding: the lang item `eh_personality` is a function used by the failure mechanisms of the compiler. This is often mapped to GCC's personality function (see the [std implementation](#) for more information), but programs which don't trigger a panic can be assured that this function is never called. Additionally, a `eh_catch_typeinfo` static is needed for certain targets which implement Rust panics on top of C++ exceptions.
- the traits in `core::marker` used to indicate types of various kinds; e.g. lang items `sized`, `sync` and `copy`.
- memory allocation, see below.

Most lang items are defined by `core`, but if you're trying to build an executable without the `std` crate, you might run into the need for lang item definitions.

Example: Implementing a Box

`Box` pointers require two lang items: one for the type itself and one for allocation. A freestanding program that uses the `Box` sugar for dynamic allocations via `malloc` and

`free :`

```
#![feature(lang_items, core_intrinsics, rustc_private, panic_unwind,
rustc_attrs)]
#![allow(internal_features)]
#![no_std]
#![no_main]

extern crate libc;
extern crate unwind;

use core::ffi::{c_int, c_void};
use core::intrinsics;
use core::panic::PanicInfo;
use core::ptr::NonNull;

pub struct Global; // the global allocator
struct Unique<T>(NonNull<T>);

#[lang = "owned_box"]
pub struct Box<T, A = Global>(Unique<T>, A);

impl<T> Box<T> {
    pub fn new(x: T) -> Self {
        #[rustc_box]
        Box::new(x)
    }
}

impl<T, A> Drop for Box<T, A> {
    fn drop(&mut self) {
        unsafe {
            libc::free(self.0.0.as_ptr() as *mut c_void);
        }
    }
}

#[lang = "exchange_malloc"]
unsafe fn allocate(size: usize, _align: usize) -> *mut u8 {
    let p = libc::malloc(size) as *mut u8;

    // Check if `malloc` failed:
    if p.is_null() {
        intrinsics::abort();
    }

    p
}

#[no_mangle]
```

```
extern "C" fn main(_argc: c_int, _argv: *const *const u8) -> c_int {
    let _x = Box::new(1);

    0
}

#[lang = "eh_personality"]
fn rust_eh_personality() {}

#[panic_handler]
fn panic_handler(_info: &PanicInfo) -> ! { intrinsics::abort() }
```

Note the use of `abort`: the `exchange_malloc` lang item is assumed to return a valid pointer, and so needs to do the check internally.

List of all language items

An up-to-date list of all language items can be found [here](#) in the compiler code.

large_assignments

The tracking issue for this feature is: [#83518](#)

lazy_type_alias

The tracking issue for this feature is: [#112792](#)

let_chains

The tracking issue for this feature is: [#53667](#)

lifetime_capture_rules_2024

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

link_arg_attribute

The tracking issue for this feature is: [#99427](#)

The `link_arg_attribute` feature allows passing arguments into the linker from inside of the source code. Order is preserved for link attributes as they were defined on a single extern block:

```
#![feature(link_arg_attribute)]

#[link(kind = "link-arg", name = "--start-group")]
#[link(kind = "static", name = "c")]
#[link(kind = "static", name = "gcc")]
#[link(kind = "link-arg", name = "--end-group")]
extern "C" {}
```

link_cfg

This feature is internal to the Rust compiler and is not intended for general use.

link_llvm_intrinsics

The tracking issue for this feature is: [#29602](#)

linkage

The tracking issue for this feature is: [#29603](#)

loongarch_target_feature

The tracking issue for this feature is: [#44839](#)

m68k_target_feature

The tracking issue for this feature is: [#134328](#)

macro_metavar_expr

The tracking issue for this feature is: [#83527](#)

macro_metavar_expr_concat

The tracking issue for this feature is: [#124225](#)

marker_trait_attr

The tracking issue for this feature is: [#29864](#)

Normally, Rust keeps you from adding trait implementations that could overlap with each other, as it would be ambiguous which to use. This feature, however, carves out an exception to that rule: a trait can opt-in to having overlapping implementations, at the cost that those implementations are not allowed to override anything (and thus the trait itself cannot have any associated items, as they're pointless when they'd need to do the same thing for every type anyway).

```
#![feature(marker_trait_attr)]

#[marker] trait CheapToClone: Clone {}

impl<T: Copy> CheapToClone for T {}

// These could potentially overlap with the blanket implementation above,
// so are only allowed because CheapToClone is a marker trait.
impl<T: CheapToClone, U: CheapToClone> CheapToClone for (T, U) {}
impl<T: CheapToClone> CheapToClone for std::ops::Range<T> {}

fn cheap_clone<T: CheapToClone>(t: T) -> T {
    t.clone()
}
```

This is expected to replace the unstable `overlapping_marker_traits` feature, which applied to all empty traits (without needing an opt-in).

min_generic_const_args

The tracking issue for this feature is: [#132980](#)

min_specialization

The tracking issue for this feature is: [#31844](#)

mips_target_feature

The tracking issue for this feature is: [#44839](#)

more_maybe_bounds

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

more_qualified_paths

The `more_qualified_paths` feature can be used in order to enable the use of qualified paths in patterns.

The tracking issue for this feature is: [#86935](#).

Example

```
#![feature(more_qualified_paths)]

fn main() {
    // destructure through a qualified path
    let <Foo as A>::Assoc { br } = StructStruct { br: 2 };
}

struct StructStruct {
    br: i8,
}

struct Foo;

trait A {
    type Assoc;
}

impl A for Foo {
    type Assoc = StructStruct;
}
```


multiple_supertrait_upcastable

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

must_not_suspend

The tracking issue for this feature is: [#83310](#)

mut_ref

The tracking issue for this feature is: [#123076](#)

naked_functions

The tracking issue for this feature is: [#90957](#)

native_link_modifiers_as_needed

The tracking issue for this feature is: [#81490](#)

The `native_link_modifiers_as_needed` feature allows you to use the `as-needed` modifier.

`as-needed` is only compatible with the `dynamic` and `framework` linking kinds. Using any other kind will result in a compiler error.

`+as-needed` means that the library will be actually linked only if it satisfies some undefined symbols at the point at which it is specified on the command line, making it similar to static libraries in this regard.

This modifier translates to `--as-needed` for ld-like linkers, and to `-dead_strip_dylibs` / `-needed_library` / `-needed_framework` for ld64. The modifier does nothing for linkers that don't support it (e.g. `link.exe`).

The default for this modifier is unclear, some targets currently specify it as `+as-needed`, some do not. We may want to try making `+as-needed` a default for all targets.

needs_panic_runtime

The tracking issue for this feature is: [#32837](#)

negative_bounds

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

negative_impls

The tracking issue for this feature is [#68318](#).

With the feature gate `negative_impls`, you can write negative impls as well as positive ones:

```
#![feature(negative_impls)]
trait DerefMut { }
impl<T: ?Sized> !DerefMut for &T { }
```

Negative impls indicate a semver guarantee that the given trait will not be implemented for the given types. Negative impls play an additional purpose for auto traits, described below.

Negative impls have the following characteristics:

- They do not have any items.
- They must obey the orphan rules as if they were a positive impl.
- They cannot "overlap" with any positive impls.

Semver interaction

It is a breaking change to remove a negative impl. Negative impls are a commitment not to implement the given trait for the named types.

Orphan and overlap rules

Negative impls must obey the same orphan rules as a positive impl. This implies you cannot add a negative impl for types defined in upstream crates and so forth.

Similarly, negative impls cannot overlap with positive impls, again using the same "overlap" check that we ordinarily use to determine if two impls overlap. (Note that positive impls typically cannot overlap with one another either, except as permitted by specialization.)

Interaction with auto traits

Declaring a negative impl `impl !SomeAutoTrait for SomeType` for an auto-trait serves two purposes:

- as with any trait, it declares that `SomeType` will never implement `SomeAutoTrait`;
- it disables the automatic `SomeType: SomeAutoTrait` impl that would otherwise have been generated.

Note that, at present, there is no way to indicate that a given type does not implement an auto trait *but that it may do so in the future*. For ordinary types, this is done by simply not declaring any impl at all, but that is not an option for auto traits. A workaround is that one could embed a marker type as one of the fields, where the marker type is `!AutoTrait`.

Immediate uses

Negative impls are used to declare that `&T: !DerefMut` and `&mut T: !Clone`, as required to fix the soundness of `Pin` described in [#66544](#).

This serves two purposes:

- For proving the correctness of unsafe code, we can use that impl as evidence that no `DerefMut` or `Clone` impl exists.
- It prevents downstream crates from creating such impls.

never_patterns

The tracking issue for this feature is: [#118155](#)

never_type

The tracking issue for this feature is: [#35121](#)

never_type_fallback

The tracking issue for this feature is: [#65992](#)

new_range

The tracking issue for this feature is: [#123741](#)

Switch the syntaxes `a..`, `a..b`, and `a..=b` to resolve the new range types.

no_core

The tracking issue for this feature is: [#29639](#)

no_sanitize

The tracking issue for this feature is: [#39699](#)

The `no_sanitize` attribute can be used to selectively disable sanitizer instrumentation in an annotated function. This might be useful to: avoid instrumentation overhead in a performance critical function, or avoid instrumenting code that contains constructs unsupported by given sanitizer.

The precise effect of this annotation depends on particular sanitizer in use. For example, with `no_sanitize(thread)`, the thread sanitizer will no longer instrument non-atomic store / load operations, but it will instrument atomic operations to avoid reporting false positives and provide meaning full stack traces.

Examples

```
#![feature(no_sanitize)]

#[no_sanitize(address)]
fn foo() {
    // ...
}
```

non_exhaustive_omitted_patterns_lint

The tracking issue for this feature is: [#89554](#)

non_lifetime_binders

The tracking issue for this feature is: [#108185](#)

offset_of_enum

The tracking issue for this feature is: [#120141](#)

offset_of_slice

The tracking issue for this feature is: [#126151](#)

omit_gdb_pretty_printer_section

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

optimize_attribute

The tracking issue for this feature is: [#54882](#)

panic_runtime

The tracking issue for this feature is: [#32837](#)

patchable_function_entry

The tracking issue for this feature is: [#123115](#)

pattern_complexity_limit

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

pattern_types

The tracking issue for this feature is: [#123646](#)

pin_ergonomics

The tracking issue for this feature is: [#130494](#)

postfix-match

`postfix-match` adds the feature for matching upon values postfix the expressions that generate the values.

The tracking issue for this feature is: [#121618](#).

```
#![feature(postfix_match)]

enum Foo {
    Bar,
    Baz
}

fn get_foo() -> Foo {
    Foo::Bar
}

get_foo().match {
    Foo::Bar => {},
    Foo::Baz => panic!(),
}
```

powerpc_target_feature

The tracking issue for this feature is: [#44839](#)

precise_capturing_in_traits

The tracking issue for this feature is: [#130044](#)

prelude_import

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

prfchw_target_feature

The tracking issue for this feature is: [#44839](#)

proc_macro_hygiene

The tracking issue for this feature is: [#54727](#)

profiler_runtime

The tracking issue for this feature is: [#42524](#).

ref_pat_eat_one_layer_2024

The tracking issue for this feature is: [#123076](#)

This feature is incomplete and not yet intended for general use.

This implements experimental, Edition-dependent match ergonomics under consideration for inclusion in Rust. For more information, see the corresponding typing rules for [Editions 2021 and earlier](#) and for [Editions 2024 and later](#).

For alternative experimental match ergonomics, see the feature [ref_pat_eat_one_layer_2024_structural](#).

ref_pat_eat_one_layer_2024_structural

The tracking issue for this feature is: [#123076](#)

This feature is incomplete and not yet intended for general use.

This implements experimental, Edition-dependent match ergonomics under consideration for inclusion in Rust. For more information, see the corresponding typing rules for [Editions 2021 and earlier](#) and for [Editions 2024 and later](#).

For alternative experimental match ergonomics, see the feature [ref_pat_eat_one_layer_2024](#).

register_tool

The tracking issue for this feature is: [#66079](#)

repr_simd

The tracking issue for this feature is: [#27731](#)

repr128

The tracking issue for this feature is: [#56071](#)

The `repr128` feature adds support for `#[repr(u128)]` on `enum`s.

```
#![feature(repr128)]

#[repr(u128)]
enum Foo {
    Bar(u64),
}
```

return_type_notation

The tracking issue for this feature is: [#109417](#)

riscv_target_feature

The tracking issue for this feature is: [#44839](#)

rtm_target_feature

The tracking issue for this feature is: [#44839](#)

rust_cold_cc

The tracking issue for this feature is: [#97544](#)

rustc_allow_const_fn_unstable

The tracking issue for this feature is: [#69399](#)

rustc_attrs

This feature has no tracking issue, and is therefore internal to the compiler, not being intended for general use.

Note: `rustc_attrs` enables many rustc-internal attributes and this page only discuss a few of them.

The `rustc_attrs` feature allows debugging rustc type layouts by using `#[rustc_layout(...)]` to debug layout at compile time (it even works with `cargo check`) as an alternative to `rustc -Z print-type-sizes` that is way more verbose.

Options provided by `#[rustc_layout(...)]` are `debug`, `size`, `align`, `abi`. Note that it only works on sized types without generics.

Examples

```
#![feature(rustc_attrs)]

#[rustc_layout(abi, size)]
pub enum X {
    Y(u8, u8, u8),
    Z(isize),
}
```

When that is compiled, the compiler will error with something like

```
error: abi: Aggregate { sized: true }
--> src/lib.rs:4:1
|
4 | / pub enum T {
5 | |     Y(u8, u8, u8),
6 | |     Z(isize),
7 | | }
  | |_^

error: size: Size { raw: 16 }
--> src/lib.rs:4:1
|
4 | / pub enum T {
5 | |     Y(u8, u8, u8),
6 | |     Z(isize),
7 | | }
  | |_^

error: aborting due to 2 previous errors
```

rustc_private

The tracking issue for this feature is: [#27812](#)

This feature allows access to unstable internal compiler crates such as `rustc_driver`.

The presence of this feature changes the way the linkage format for dylibs is calculated in a way that is necessary for linking against dylibs that statically link `std` (such as `rustc_driver`). This makes this feature "viral" in linkage; its use in a given crate makes its use required in dependent crates which link to it (including integration tests, which are built as separate crates).

rustdoc_internals

The tracking issue for this feature is: [#90418](#)

rustdoc_missing_doc_code_examples

The tracking issue for this feature is: [#101730](#)

s390x_target_feature

The tracking issue for this feature is: [#44839](#)

sha512_sm_x86

The tracking issue for this feature is: [#126624](#)

simd_ffi

The tracking issue for this feature is: [#27731](#)

sparc_target_feature

The tracking issue for this feature is: [#132783](#)

specialization

The tracking issue for this feature is: [#31844](#)

sse4a_target_feature

The tracking issue for this feature is: [#44839](#)

staged_api

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

stmt_expr_attributes

The tracking issue for this feature is: [#15701](#)

strict_provenance_lints

The tracking issue for this feature is: [#95228](#)

The `strict_provenance_lints` feature allows to enable the `fuzzy_provenance_casts` and `lossy_provenance_casts` lints. These lint on casts between integers and pointers, that are recommended against or invalid in the strict provenance model.

Example

```
#![feature(strict_provenance_lints)]
#![warn(fuzzy_provenance_casts)]

fn main() {
    let _dangling = 16_usize as *const u8;
    //~^ WARNING: strict provenance disallows casting integer `usize` to
    pointer `*const u8`
}
```

string_deref_patterns

The tracking issue for this feature is: [#87121](#)

This feature permits pattern matching `String` to `&str` through its `Deref` implementation.

```
#![feature(string_deref_patterns)]

pub enum Value {
    String(String),
    Number(u32),
}

pub fn is_it_the_answer(value: Value) -> bool {
    match value {
        Value::String("42") => true,
        Value::Number(42) => true,
        _ => false,
    }
}
```

Without this feature other constructs such as match guards have to be used.

```
pub fn is_it_the_answer(value: Value) -> bool {
    match value {
        Value::String(s) if s == "42" => true,
        Value::Number(42) => true,
        _ => false,
    }
}
```

structural_match

The tracking issue for this feature is: [#31434](#)

supertrait_item_shadowing

The tracking issue for this feature is: [#89151](#)

tbm_target_feature

The tracking issue for this feature is: [#44839](#)

test_unstable_lint

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

thread_local

The tracking issue for this feature is: [#29594](#)

trait_alias

The tracking issue for this feature is: [#41517](#)

The `trait_alias` feature adds support for trait aliases. These allow aliases to be created for one or more traits (currently just a single regular trait plus any number of auto-traits), and used wherever traits would normally be used as either bounds or trait objects.

```
#![feature(trait_alias)]

trait Foo = std::fmt::Debug + Send;
trait Bar = Foo + Sync;

// Use trait alias as bound on type parameter.
fn foo<T: Foo>(v: &T) {
    println!("{:?}", v);
}

pub fn main() {
    foo(&1);

    // Use trait alias for trait objects.
    let a: &Bar = &123;
    println!("{:?}", a);
    let b = Box::new(456) as Box<dyn Foo>;
    println!("{:?}", b);
}
```


transmute_generic_consts

The tracking issue for this feature is: [#109929](#)

transparent_unions

The tracking issue for this feature is [#60405](#)

The `transparent_unions` feature allows you mark `union`s as `#[repr(transparent)]`. A `union` may be `#[repr(transparent)]` in exactly the same conditions in which a `struct` may be `#[repr(transparent)]` (generally, this means the `union` must have exactly one non-zero-sized field). Some concrete illustrations follow.

```
#![feature(transparent_unions)]

// This union has the same representation as `f32`.
#[repr(transparent)]
union SingleFieldUnion {
    field: f32,
}

// This union has the same representation as `usize`.
#[repr(transparent)]
union MultiFieldUnion {
    field: usize,
    nothing: (),
}
```

For consistency with transparent `struct`s, `union`s must have exactly one non-zero-sized field. If all fields are zero-sized, the `union` must not be `#[repr(transparent)]`:

```
#![feature(transparent_unions)]

// This (non-transparent) union is already valid in stable Rust:
pub union GoodUnion {
    pub nothing: (),
}

// Error: transparent union needs exactly one non-zero-sized field, but has 0
// #[repr(transparent)]
// pub union BadUnion {
//     pub nothing: (),
// }
```

The one exception is if the `union` is generic over `T` and has a field of type `T`, it may be `#[repr(transparent)]` even if `T` is a zero-sized type:

```

#![feature(transparent_unions)]

// This union has the same representation as `T`.
#[repr(transparent)]
pub union GenericUnion<T: Copy> { // Unions with non-`Copy` fields are
    unstable.
    pub field: T,
    pub nothing: (),
}

// This is okay even though `()` is a zero-sized type.
pub const THIS_IS_OKAY: GenericUnion<()> = GenericUnion { field: () };

```

Like transparent `struct`s, a transparent `union` of type `U` has the same layout, size, and ABI as its single non-ZST field. If it is generic over a type `T`, and all its fields are ZSTs except for exactly one field of type `T`, then it has the same layout and ABI as `T` (even if `T` is a ZST when monomorphized).

Like transparent `struct`s, transparent `union`s are FFI-safe if and only if their underlying representation type is also FFI-safe.

A `union` may not be eligible for the same nonnull-style optimizations that a `struct` or `enum` (with the same fields) are eligible for. Adding `#[repr(transparent)]` to `union` does not change this. To give a more concrete example, it is unspecified whether `size_of::()` is equal to `size_of::, where T is a union (regardless of whether or not it is transparent). The Rust compiler is free to perform this optimization if possible, but is not required to, and different compiler versions may differ in their application of these optimizations.`

trivial_bounds

The tracking issue for this feature is: [#48214](#)

try_blocks

The tracking issue for this feature is: [#31436](#)

The `try_blocks` feature adds support for `try` blocks. A `try` block creates a new scope one can use the `?` operator in.

```
#![feature(try_blocks)]

use std::num::ParseIntError;

let result: Result<i32, ParseIntError> = try {
    "1".parse::<i32>()?
    + "2".parse::<i32>()?
    + "3".parse::<i32>()?
};
assert_eq!(result, Ok(6));

let result: Result<i32, ParseIntError> = try {
    "1".parse::<i32>()?
    + "foo".parse::<i32>()?
    + "3".parse::<i32>()?
};
assert!(result.is_err());
```

type_alias_impl_trait

The tracking issue for this feature is: [#63063](#)

type_changing_struct_update

The tracking issue for this feature is: [#86555](#)

This implements [RFC2528](#). When turned on, you can create instances of the same struct that have different generic type or lifetime parameters.

```
#![allow(unused_variables, dead_code)]
#![feature(type_changing_struct_update)]

fn main () {
    struct Foo<T, U> {
        field1: T,
        field2: U,
    }

    let base: Foo<String, i32> = Foo {
        field1: String::from("hello"),
        field2: 1234,
    };
    let updated: Foo<f64, i32> = Foo {
        field1: 3.14,
        ..base
    };
}
```

unboxed_closures

The tracking issue for this feature is [#29625](#)

See Also: [fn_traits](#)

The `unboxed_closures` feature allows you to write functions using the `"rust-call"` ABI, required for implementing the `Fn*` family of traits. `"rust-call"` functions must have exactly one (non self) argument, a tuple representing the argument list.

```
#![feature(unboxed_closures)]

extern "rust-call" fn add_args(args: (u32, u32)) -> u32 {
    args.0 + args.1
}

fn main() {}
```


unqualified_local_imports

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

unsafe_binders

The tracking issue for this feature is: [#130516](#)

unsafe_fields

The tracking issue for this feature is: [#132922](#)

unsized_const_params

The tracking issue for this feature is: [#95174](#)

unsized_fn_params

The tracking issue for this feature is: [#48055](#)

unsized_locals

The tracking issue for this feature is: [#48055](#)

This implements [RFC1909](#). When turned on, you can have unsized arguments and locals:

```
#![allow(incomplete_features)]
#![feature(unsized_locals, unsized_fn_params)]

use std::any::Any;

fn main() {
    let x: Box<dyn Any> = Box::new(42);
    let x: dyn Any = *x;
    //   ^ unsized local variable
    //           ^^ unsized temporary
    foo(x);
}

fn foo(_: dyn Any) {}
//      ^^^^^^^^ unsized argument
```

The RFC still forbids the following unsized expressions:

```

#![feature(unsized_locals)]

use std::any::Any;

struct MyStruct<T: ?Sized> {
    content: T,
}

struct MyTupleStruct<T: ?Sized>(T);

fn answer() -> Box<dyn Any> {
    Box::new(42)
}

fn main() {
    // You CANNOT have unsized statics.
    static X: dyn Any = *answer(); // ERROR
    const Y: dyn Any = *answer(); // ERROR

    // You CANNOT have struct initialized unsized.
    MyStruct { content: *answer() }; // ERROR
    MyTupleStruct(*answer()); // ERROR
    (42, *answer()); // ERROR

    // You CANNOT have unsized return types.
    fn my_function() -> dyn Any { *answer() } // ERROR

    // You CAN have unsized local variables...
    let mut x: dyn Any = *answer(); // OK
    // ...but you CANNOT reassign to them.
    x = *answer(); // ERROR

    // You CANNOT even initialize them separately.
    let y: dyn Any; // OK
    y = *answer(); // ERROR

    // Not mentioned in the RFC, but by-move captured variables are also
    Sized.
    let x: dyn Any = *answer();
    (move || { // ERROR
        let y = x;
    })();

    // You CAN create a closure with unsized arguments,
    // but you CANNOT call it.
    // This is an implementation detail and may be changed in the future.
    let f = |x: dyn Any| {};
    f(*answer()); // ERROR
}

```

By-value trait objects

With this feature, you can have by-value `self` arguments without `Self: Sized` bounds.

```
#![feature(unsized_fn_params)]

trait Foo {
    fn foo(self) {}
}

impl<T: ?Sized> Foo for T {}

fn main() {
    let slice: Box<[i32]> = Box::new([1, 2, 3]);
    <[i32] as Foo>::foo(*slice);
}
```

And `Foo` will also be object-safe.

```
#![feature(unsized_fn_params)]

trait Foo {
    fn foo(self) {}
}

impl<T: ?Sized> Foo for T {}

fn main () {
    let slice: Box<dyn Foo> = Box::new([1, 2, 3]);
    // doesn't compile yet
    <dyn Foo as Foo>::foo(*slice);
}
```

One of the objectives of this feature is to allow `Box<dyn FnOnce>`.

Variable length arrays

The RFC also describes an extension to the array literal syntax: `[e; dyn n]`. In the syntax, `n` isn't necessarily a constant expression. The array is dynamically allocated on the stack and has the type of `[T]`, instead of `[T; n]`.

VLA's are not implemented yet. The syntax isn't final, either. We may need an alternative syntax for Rust 2015 because, in Rust 2015, expressions like `[e; dyn(1)]` would be ambiguous. One possible alternative proposed in the RFC is `[e; n]`: if `n` captures one or more local variables, then it is considered as `[e; dyn n]`.

It's advised not to casually use the `#![feature(unsized_locals)]` feature. Typical use-cases are:

- Another pitfall is repetitive allocation and temporaries. Currently the compiler simply extends the stack frame every time it encounters an unsized assignment. So for example, the code

and the code

```
#![feature(unsized_locals)]

fn main() {
    for _ in 0..10 {
        let x: Box<i32> = Box::new([1, 2, 3, 4, 5]);
        let _x = *x;
    }
}
```

will unnecessarily extend the stack frame.

unsized_tuple_coercion

The tracking issue for this feature is: [#42877](#)

This is a part of [RFC0401](#). According to the RFC, there should be an implementation like this:

```
impl<..., T, U: ?Sized> Unsized<(..., U)> for (... , T) where T: Unsized<U> {}
```

This implementation is currently gated behind `#[feature(unsized_tuple_coercion)]` to avoid insta-stability. Therefore you can use it like this:

```
#![feature(unsized_tuple_coercion)]

fn main() {
    let x : ([i32; 3], [i32; 3]) = ([1, 2, 3], [4, 5, 6]);
    let y : &([i32; 3], [i32]) = &x;
    assert_eq!(y.1[0], 4);
}
```

used_with_arg

The tracking issue for this feature is: [#93798](#)

wasm_target_feature

The tracking issue for this feature is: [#44839](#)

`with_negative_coherence`

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

x86_amx_intrinsics

The tracking issue for this feature is: [#126622](#)

x87_target_feature

The tracking issue for this feature is: [#44839](#)

xop_target_feature

The tracking issue for this feature is: [#127208](#)

yeet_expr

The tracking issue for this feature is: [#96373](#)

The `yeet_expr` feature adds support for `do yeet` expressions, which can be used to early-exit from a function or `try` block.

These are highly experimental, thus the placeholder syntax.

```
#![feature(yeet_expr)]

fn foo() -> Result<String, i32> {
    do yeet 4;
}
assert_eq!(foo(), Err(4));

fn bar() -> Option<String> {
    do yeet;
}
assert_eq!(bar(), None);
```

Library Features

abort_unwind

The tracking issue for this feature is: [#130338](#)

acceptfilter

The tracking issue for this feature is: [#121891](#)

addr_parse_ascii

The tracking issue for this feature is: [#101035](#)

alloc_error_hook

The tracking issue for this feature is: [#51245](#)

alloc_internals

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

alloc_layout_extra

The tracking issue for this feature is: [#55724](#)

allocator_api

The tracking issue for this feature is [#32838](#)

Sometimes you want the memory for one collection to use a different allocator than the memory for another collection. In this case, replacing the global allocator is not a workable option. Instead, you need to pass in an instance of an `AllocRef` to each collection for which you want a custom allocator.

TBD

anonymous_pipe

The tracking issue for this feature is: [#127154](#)

array_chunks

The tracking issue for this feature is: [#74985](#)

array_into_iter_constructors

The tracking issue for this feature is: [#91583](#)

array_ptr_get

The tracking issue for this feature is: [#119834](#)

array_repeat

The tracking issue for this feature is: [#126695](#)

array_try_from_fn

The tracking issue for this feature is: [#89379](#)

array_try_map

The tracking issue for this feature is: [#79711](#)

array_windows

The tracking issue for this feature is: [#75027](#)

as_array_of_cells

The tracking issue for this feature is: [#88248](#)

ascii_char

The tracking issue for this feature is: [#110998](#)

ascii_char_variants

The tracking issue for this feature is: [#110998](#)

assert_matches

The tracking issue for this feature is: [#82775](#)

async_drop

The tracking issue for this feature is: [#126482](#)

async_fn_traits

See Also: [fn_traits](#)

The `async_fn_traits` feature allows for implementation of the `AsyncFn*` traits for creating custom closure-like types that return futures.

The main difference to the `Fn*` family of traits is that `AsyncFn` can return a future that borrows from itself (`FnOnce::Output` has no lifetime parameters, while `AsyncFnMut::CallRefFuture` does).

async_gen_internals

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

async_iter_from_iter

The tracking issue for this feature is: [#81798](#)

async_iterator

The tracking issue for this feature is: [#79024](#)

atomic_from_mut

The tracking issue for this feature is: [#76314](#)

atomic_try_update

The tracking issue for this feature is: [#135894](#)

autodiff

The tracking issue for this feature is: [#124509](#)

backtrace_frames

The tracking issue for this feature is: [#79676](#)

bigint_helper_methods

The tracking issue for this feature is: [#85532](#)

bikeshed_guaranteed_no_drop

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

binary_heap_drain_sorted

The tracking issue for this feature is: [#59278](#)

binary_heap_into_iter_sorted

The tracking issue for this feature is: [#59278](#)

bound_as_ref

The tracking issue for this feature is: [#80996](#)

box_as_ptr

The tracking issue for this feature is: [#129090](#)

box_into_boxed_slice

The tracking issue for this feature is: [#71582](#)

box_into_inner

The tracking issue for this feature is: [#80437](#)

box_uninit_write

The tracking issue for this feature is: [#129397](#)

box_vec_non_null

The tracking issue for this feature is: [#130364](#)

breakpoint

The tracking issue for this feature is: [#133724](#)

bstr

The tracking issue for this feature is: [#134915](#)

bstr_internals

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

btree_cursors

The tracking issue for this feature is: [#107540](#)

btree_entry_insert

The tracking issue for this feature is: [#65225](#)

btree_extract_if

The tracking issue for this feature is: [#70530](#)

btree_set_entry

The tracking issue for this feature is: [#133549](#)

btreemap_alloc

The tracking issue for this feature is: [#32838](#)

buf_read_has_data_left

The tracking issue for this feature is: [#86423](#)

bufreader_peek

The tracking issue for this feature is: [#128405](#)

c_size_t

The tracking issue for this feature is: [#88345](#)

c_str_module

The tracking issue for this feature is: [#112134](#)

c_void_variant

This feature is internal to the Rust compiler and is not intended for general use.

can_vector

The tracking issue for this feature is: [#69941](#)

cell_leak

The tracking issue for this feature is: [#69099](#)

cell_update

The tracking issue for this feature is: [#50186](#)

cfg_accessible

The tracking issue for this feature is: [#64797](#)

cfg_eval

The tracking issue for this feature is: [#82679](#)

cfg_match

The tracking issue for this feature is: [#115585](#)

char_internals

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

char_max_len

The tracking issue for this feature is: [#121714](#)

clone_to_uninit

The tracking issue for this feature is: [#126799](#)

cmp_minmax

The tracking issue for this feature is: [#115939](#)

coerce_pointee_validated

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

coerce_unsized

The tracking issue for this feature is: [#18598](#)

cold_path

The tracking issue for this feature is: [#136873](#)

concat_bytes

The tracking issue for this feature is: [#87555](#)

concat_idents

The tracking issue for this feature is: [#29599](#)

The `concat_idents` feature adds a macro for concatenating multiple identifiers into one identifier.

Examples

```
#![feature(concat_idents)]

fn main() {
    fn foobar() -> u32 { 23 }
    let f = concat_idents!(foo, bar);
    assert_eq!(f(), 23);
}
```

const_alloc_error

The tracking issue for this feature is: [#92523](#)

const_array_as_mut_slice

The tracking issue for this feature is: [#133333](#)

const_array_each_ref

The tracking issue for this feature is: [#133289](#)

const_box

The tracking issue for this feature is: [#92521](#)

const_btrees_len

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

const_carrying_mul_add

The tracking issue for this feature is: [#85532](#)

const_cell

The tracking issue for this feature is: [#131283](#)

const_char_classify

The tracking issue for this feature is: [#132241](#)

const_copy_from_slice

The tracking issue for this feature is: [#131415](#)

const_deref

The tracking issue for this feature is: [#88955](#)

const_eq_ignore_ascii_case

The tracking issue for this feature is: [#131719](#)

const_eval_select

The tracking issue for this feature is: [#124625](#)

const_format_args

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

const_heap

The tracking issue for this feature is: [#79597](#)

const_ops

The tracking issue for this feature is: [#90080](#)

const_ptr_sub_ptr

The tracking issue for this feature is: [#95892](#)

const_range_bounds

The tracking issue for this feature is: [#108082](#)

const_raw_ptr_comparison

The tracking issue for this feature is: [#53020](#)

const_slice_from_mut_ptr_range

The tracking issue for this feature is: [#89792](#)

const_slice_from_ptr_range

The tracking issue for this feature is: [#89792](#)

const_slice_reverse

The tracking issue for this feature is: [#135120](#)

const_sockaddr_setters

The tracking issue for this feature is: [#131714](#)

const_str_from_utf8

The tracking issue for this feature is: [#91006](#)

const_swap_nonoverlapping

The tracking issue for this feature is: [#133668](#)

const_type_id

The tracking issue for this feature is: [#77125](#)

const_type_name

The tracking issue for this feature is: [#63084](#)

const_vec_string_slice

The tracking issue for this feature is: [#129041](#)

container_error_extra

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

context_ext

The tracking issue for this feature is: [#123392](#)

convert_float_to_int

The tracking issue for this feature is: [#67057](#)

core_intrinsics

This feature is internal to the Rust compiler and is not intended for general use.

core_intrinsics_fallbacks

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

core_io_borrowed_buf

The tracking issue for this feature is: [#117693](#)

core_private_bignum

This feature is internal to the Rust compiler and is not intended for general use.

core_private_diy_float

This feature is internal to the Rust compiler and is not intended for general use.

coroutine_trait

The tracking issue for this feature is: [#43122](#)

cow_is_borrowed

The tracking issue for this feature is: [#65143](#)

cstr_bytes

The tracking issue for this feature is: [#112115](#)

cstr_internals

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

cursor_split

The tracking issue for this feature is: [#86369](#)

deadline_api

The tracking issue for this feature is: [#46316](#)

debug_closure_helpers

The tracking issue for this feature is: [#117729](#)

dec2flt

This feature is internal to the Rust compiler and is not intended for general use.

deref_pure_trait

The tracking issue for this feature is: [#87121](#)

derive_clone_copy

This feature is internal to the Rust compiler and is not intended for general use.

derive_coerce_pointee

The tracking issue for this feature is: [#123430](#)

derive_const

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

derive_eq

This feature is internal to the Rust compiler and is not intended for general use.

dir_entry_ext2

The tracking issue for this feature is: [#85573](#)

discriminant_kind

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

disjoint_bitor

The tracking issue for this feature is: [#135758](#)

dispatch_from_dyn

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

downcast_unchecked

The tracking issue for this feature is: [#90850](#)

drain_keep_rest

The tracking issue for this feature is: [#101122](#)

duration_constants

The tracking issue for this feature is: [#57391](#)

duration_constructors

The tracking issue for this feature is: [#120301](#)

Add the methods `from_mins`, `from_hours` and `from_days` to `Duration`.

duration_millis_float

The tracking issue for this feature is: [#122451](#)

duration_units

The tracking issue for this feature is: [#120301](#)

edition_panic

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

error_generic_member_access

The tracking issue for this feature is: [#99301](#)

error_iter

The tracking issue for this feature is: [#58520](#)

error_reporter

The tracking issue for this feature is: [#90172](#)

error_type_id

The tracking issue for this feature is: [#60784](#)

exact_size_is_empty

The tracking issue for this feature is: [#35428](#)

exclusive_wrapper

The tracking issue for this feature is: [#98407](#)

exit_status_error

The tracking issue for this feature is: [#84908](#)

exitcode_exit_method

The tracking issue for this feature is: [#97100](#)

extend_one

The tracking issue for this feature is: [#72631](#)

extend_one_unchecked

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

extract_if

The tracking issue for this feature is: [#43244](#)

fd

This feature is internal to the Rust compiler and is not intended for general use.

fd_read

This feature is internal to the Rust compiler and is not intended for general use.

file_buffered

The tracking issue for this feature is: [#130804](#)

float_erf

The tracking issue for this feature is: [#136321](#)

float_gamma

The tracking issue for this feature is: [#99842](#)

float_minimum_maximum

The tracking issue for this feature is: [#91079](#)

flt2dec

This feature is internal to the Rust compiler and is not intended for general use.

fmt_helpers_for_derive

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

fmt_internals

This feature is internal to the Rust compiler and is not intended for general use.

fn_ptr_trait

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

fn_traits

The tracking issue for this feature is [#29625](#)

See Also: [unboxed_closures](#)

The `fn_traits` feature allows for implementation of the `Fn*` traits for creating custom closure-like types.

```
#![feature(unboxed_closures)]
#![feature(fn_traits)]

struct Adder {
    a: u32
}

impl FnOnce<(u32, )> for Adder {
    type Output = u32;
    extern "rust-call" fn call_once(self, b: (u32, )) -> Self::Output {
        self.a + b.0
    }
}

fn main() {
    let adder = Adder { a: 3 };
    assert_eq!(adder(2), 5);
}
```

forget_unsized

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

format_args_nl

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

formatting_options

The tracking issue for this feature is: [#118117](#)

freeze

The tracking issue for this feature is: [#121675](#)

future_join

The tracking issue for this feature is: [#91642](#)

gen_future

The tracking issue for this feature is: [#50547](#)

generic_assert_internals

The tracking issue for this feature is: [#44838](#)

get_disjoint_mut_helpers

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

get_mut_unchecked

The tracking issue for this feature is: [#63292](#)

hash_extract_if

The tracking issue for this feature is: [#59618](#)

hash_raw_entry

The tracking issue for this feature is: [#56167](#)

hash_set_entry

The tracking issue for this feature is: [#60896](#)

hasher_prefixfree_extras

The tracking issue for this feature is: [#96762](#)

hashmap_internals

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

hint_must_use

The tracking issue for this feature is: [#94745](#)

inplace_iteration

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

int_from_ascii

The tracking issue for this feature is: [#134821](#)

int_roundings

The tracking issue for this feature is: [#88581](#)

integer_atomics

The tracking issue for this feature is: [#99069](#)

internal_impls_macro

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

internal_output_capture

This feature is internal to the Rust compiler and is not intended for general use.

io_const_error

The tracking issue for this feature is: [#133448](#)

io_const_error_internals

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

io_error_inprogress

The tracking issue for this feature is: [#130840](#)

io_error_more

The tracking issue for this feature is: [#86442](#)

`io_error_uncategorized`

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

io_slice_as_bytes

The tracking issue for this feature is: [#132818](#)

ip

The tracking issue for this feature is: [#27709](#)

ip_as_octets

The tracking issue for this feature is: [#137259](#)

ip_from

The tracking issue for this feature is: [#131360](#)

is_ascii_octdigit

The tracking issue for this feature is: [#101288](#)

is_loongarch_feature_detected

The tracking issue for this feature is: [#117425](#)

is_riscv_feature_detected

The tracking issue for this feature is: [#111192](#)

iter_advance_by

The tracking issue for this feature is: [#77404](#)

iter_array_chunks

The tracking issue for this feature is: [#100450](#)

iter_chain

The tracking issue for this feature is: [#125964](#)

iter_collect_into

The tracking issue for this feature is: [#94780](#)

iter_from_coroutine

The tracking issue for this feature is: [#43122](#)

iter_intersperse

The tracking issue for this feature is: [#79524](#)

iter_is_partitioned

The tracking issue for this feature is: [#62544](#)

iter_map_windows

The tracking issue for this feature is: [#87155](#)

iter_next_chunk

The tracking issue for this feature is: [#98326](#)

iter_order_by

The tracking issue for this feature is: [#64295](#)

iter_partition_in_place

The tracking issue for this feature is: [#62543](#)

iterator_try_collect

The tracking issue for this feature is: [#94047](#)

iterator_try_reduce

The tracking issue for this feature is: [#87053](#)

junction_point

The tracking issue for this feature is: [#121709](#)

layout_for_ptr

The tracking issue for this feature is: [#69835](#)

lazy_cell_into_inner

The tracking issue for this feature is: [#125623](#)

lazy_get

The tracking issue for this feature is: [#129333](#)

legacy_receiver_trait

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

liballoc_internals

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

libstd_sys_internals

This feature is internal to the Rust compiler and is not intended for general use.

likely_unlikely

The tracking issue for this feature is: [#136873](#)

linked_list_cursors

The tracking issue for this feature is: [#58533](#)

linked_list_remove

The tracking issue for this feature is: [#69210](#)

linked_list_retain

The tracking issue for this feature is: [#114135](#)

linux_pidfd

The tracking issue for this feature is: [#82971](#)

local_waker

The tracking issue for this feature is: [#118959](#)

lock_value_accessors

The tracking issue for this feature is: [#133407](#)

log_syntax

The tracking issue for this feature is: [#29598](#)

map_try_insert

The tracking issue for this feature is: [#82766](#)

mapped_lock_guards

The tracking issue for this feature is: [#117108](#)

maybe_uninit_array_assume_init

The tracking issue for this feature is: [#96097](#)

maybe_uninit_as_bytes

The tracking issue for this feature is: [#93092](#)

maybe_uninit_fill

The tracking issue for this feature is: [#117428](#)

maybe_uninit_slice

The tracking issue for this feature is: [#63569](#)

maybe_uninit_uninit_array

The tracking issue for this feature is: [#96097](#)

maybe_uninit_uninit_array_transpose

The tracking issue for this feature is: [#96097](#)

maybe_uninit_write_slice

The tracking issue for this feature is: [#79995](#)

mem_copy_fn

The tracking issue for this feature is: [#98262](#)

mixed_integer_ops_unsigned_sub

The tracking issue for this feature is: [#126043](#)

more_float_constants

The tracking issue for this feature is: [#103883](#)

mpmc_channel

The tracking issue for this feature is: [#126840](#)

new_range_api

The tracking issue for this feature is: [#125687](#)

new_zeroed_alloc

The tracking issue for this feature is: [#129396](#)

non_null_from_ref

The tracking issue for this feature is: [#130823](#)

nonnull_provenance

The tracking issue for this feature is: [#135243](#)

nonzero_bitwise

The tracking issue for this feature is: [#128281](#)

nonzero_from_mut

The tracking issue for this feature is: [#106290](#)

nonzero_internals

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

nonzero_ops

The tracking issue for this feature is: [#84186](#)

numfmt

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

once_cell_get_mut

The tracking issue for this feature is: [#121641](#)

once_cell_try

The tracking issue for this feature is: [#109737](#)

once_cell_try_insert

The tracking issue for this feature is: [#116693](#)

one_sided_range

The tracking issue for this feature is: [#69780](#)

option_array_transpose

The tracking issue for this feature is: [#130828](#)

option_zip

The tracking issue for this feature is: [#70086](#)

os_str_slice

The tracking issue for this feature is: [#118485](#)

os_string_pathbuf_leak

The tracking issue for this feature is: [#125965](#)

os_string_truncate

The tracking issue for this feature is: [#133262](#)

panic_abort

The tracking issue for this feature is: [#32837](#)

panic_always_abort

The tracking issue for this feature is: [#84438](#)

panic_backtrace_config

The tracking issue for this feature is: [#93346](#)

panic_can_unwind

The tracking issue for this feature is: [#92988](#)

panic_internals

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

panic_payload_as_str

The tracking issue for this feature is: [#125175](#)

panic_unwind

The tracking issue for this feature is: [#32837](#)

panic_update_hook

The tracking issue for this feature is: [#92649](#)

path_add_extension

The tracking issue for this feature is: [#127292](#)

path_file_prefix

The tracking issue for this feature is: [#86319](#)

pattern

The tracking issue for this feature is: [#27721](#)

pattern_type_macro

The tracking issue for this feature is: [#123646](#)

peer_credentials_unix_socket

The tracking issue for this feature is: [#42839](#)

phantom_variance_markers

The tracking issue for this feature is: [#135806](#)

pin_coerce_unsized_trait

The tracking issue for this feature is: [#123430](#)

pointer_is_aligned_to

The tracking issue for this feature is: [#96284](#)

pointer_like_trait

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

portable_simd

The tracking issue for this feature is: [#86656](#)

print_internals

This feature is internal to the Rust compiler and is not intended for general use.

proc_macro_def_site

The tracking issue for this feature is: [#54724](#)

proc_macro_diagnostic

The tracking issue for this feature is: [#54140](#)

proc_macro_expand

The tracking issue for this feature is: [#90765](#)

proc_macro_internals

The tracking issue for this feature is: [#27812](#)

proc_macro_quote

The tracking issue for this feature is: [#54722](#)

proc_macro_span

The tracking issue for this feature is: [#54725](#)

proc_macro_totokens

The tracking issue for this feature is: [#130977](#)

proc_macro_tracked_env

The tracking issue for this feature is: [#99515](#)

process_exitcode_internals

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

process_internals

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

profiler_runtime_lib

This feature is internal to the Rust compiler and is not intended for general use.

ptr_alignment_type

The tracking issue for this feature is: [#102070](#)

ptr_as_ref_unchecked

The tracking issue for this feature is: [#122034](#)

ptr_as_uninit

The tracking issue for this feature is: [#75402](#)

ptr_internals

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

ptr_mask

The tracking issue for this feature is: [#98290](#)

ptr_metadata

The tracking issue for this feature is: [#81513](#)

ptr_sub_ptr

The tracking issue for this feature is: [#95892](#)

pub_crate_should_not_need_unstable_attr

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

random

The tracking issue for this feature is: [#130703](#)

range_into_bounds

The tracking issue for this feature is: [#136903](#)

raw_os_error_ty

The tracking issue for this feature is: [#107792](#)

raw_slice_split

The tracking issue for this feature is: [#95595](#)

raw_vec_internals

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

read_buf

The tracking issue for this feature is: [#78485](#)

reentrant_lock

The tracking issue for this feature is: [#121440](#)

restricted_std

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

result_flattening

The tracking issue for this feature is: [#70142](#)

round_char_boundary

The tracking issue for this feature is: [#93743](#)

rt

This feature is internal to the Rust compiler and is not intended for general use.

`rwlock_downgrade`

The tracking issue for this feature is: [#128203](#)

sealed

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

seek_stream_len

The tracking issue for this feature is: [#59359](#)

select_unpredictable

The tracking issue for this feature is: [#133962](#)

set_ptr_value

The tracking issue for this feature is: [#75091](#)

setgroups

The tracking issue for this feature is: [#90747](#)

sgx_platform

The tracking issue for this feature is: [#56975](#)

sized_type_properties

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

slice_as_array

The tracking issue for this feature is: [#133508](#)

slice_as_chunks

The tracking issue for this feature is: [#74985](#)

slice_concat_ext

The tracking issue for this feature is: [#27747](#)

slice_concat_trait

The tracking issue for this feature is: [#27747](#)

slice_from_ptr_range

The tracking issue for this feature is: [#89792](#)

slice_index_methods

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

slice_internals

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

slice_iter_mut_as_mut_slice

The tracking issue for this feature is: [#93079](#)

slice_partition_dedup

The tracking issue for this feature is: [#54279](#)

slice_pattern

The tracking issue for this feature is: [#56345](#)

slice_ptr_get

The tracking issue for this feature is: [#74265](#)

slice_range

The tracking issue for this feature is: [#76393](#)

slice_split_once

The tracking issue for this feature is: [#112811](#)

slice_swap_unchecked

The tracking issue for this feature is: [#88539](#)

slice_take

The tracking issue for this feature is: [#62280](#)

solid_ext

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

sort_floats

The tracking issue for this feature is: [#93396](#)

split_array

The tracking issue for this feature is: [#90091](#)

split_as_slice

The tracking issue for this feature is: [#96137](#)

std_internals

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

stdarch_arm_feature_detection

The tracking issue for this feature is: [#111190](#)

stdarch_mips_feature_detection

The tracking issue for this feature is: [#111188](#)

stdarch_powerpc_feature_detection

The tracking issue for this feature is: [#111191](#)

stdio_makes_pipe

The tracking issue for this feature is: [#98288](#)

step_trait

The tracking issue for this feature is: [#42168](#)

str_as_str

The tracking issue for this feature is: [#130366](#)

str_from_raw_parts

The tracking issue for this feature is: [#119206](#)

str_from_utf16_endian

The tracking issue for this feature is: [#116258](#)

str_internals

This feature is internal to the Rust compiler and is not intended for general use.

str_lines_remainder

The tracking issue for this feature is: [#77998](#)

str_split_inclusive_remainder

The tracking issue for this feature is: [#77998](#)

str_split_remainder

The tracking issue for this feature is: [#77998](#)

str_split_whitespace_remainder

The tracking issue for this feature is: [#77998](#)

strict_overflow_ops

The tracking issue for this feature is: [#118260](#)

strict_provenance_atomic_ptr

The tracking issue for this feature is: [#99108](#)

string_extend_from_within

The tracking issue for this feature is: [#103806](#)

string_from_utf8_lossy_owned

The tracking issue for this feature is: [#129436](#)

string_into_chars

The tracking issue for this feature is: [#133125](#)

string_remove_matches

The tracking issue for this feature is: [#72826](#)

substr_range

The tracking issue for this feature is: [#126769](#)

sync_poison_mod

The tracking issue for this feature is: [#134646](#)

sync_unsafe_cell

The tracking issue for this feature is: [#95439](#)

tcp_deferaccept

The tracking issue for this feature is: [#119639](#)

tcp_linger

The tracking issue for this feature is: [#88494](#)

tcp_quickack

The tracking issue for this feature is: [#96256](#)

tcpListener_into_incoming

The tracking issue for this feature is: [#88373](#)

temporary_niche_types

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

test

The tracking issue for this feature is: None.

The internals of the `test` crate are unstable, behind the `test` flag. The most widely used part of the `test` crate are benchmark tests, which can test the performance of your code. Let's make our `src/lib.rs` look like this (comments elided):

```
#![feature(test)]

extern crate test;

pub fn add_two(a: i32) -> i32 {
    a + 2
}

#[cfg(test)]
mod tests {
    use super::*;
    use test::Bencher;

    #[test]
    fn it_works() {
        assert_eq!(4, add_two(2));
    }

    #[bench]
    fn bench_add_two(b: &mut Bencher) {
        b.iter(|| add_two(2));
    }
}
```

Note the `test` feature gate, which enables this unstable feature.

We've imported the `test` crate, which contains our benchmarking support. We have a new function as well, with the `bench` attribute. Unlike regular tests, which take no arguments, benchmark tests take a `&mut Bencher`. This `Bencher` provides an `iter` method, which takes a closure. This closure contains the code we'd like to benchmark.

We can run benchmark tests with `cargo bench`:

```
$ cargo bench
  Compiling adder v0.0.1 (file:///home/steve/tmp/adder)
  Running target/release/adder-91b3e234d4ed382a

running 2 tests
test tests::it_works ... ignored
test tests::bench_add_two ... bench:          1 ns/iter (+/- 0)

test result: ok. 0 passed; 0 failed; 1 ignored; 1 measured
```

Our non-benchmark test was ignored. You may have noticed that `cargo bench` takes a bit longer than `cargo test`. This is because Rust runs our benchmark a number of times, and then takes the average. Because we're doing so little work in this example, we have a `1 ns/iter (+/- 0)`, but this would show the variance if there was one.

Advice on writing benchmarks:

- Move setup code outside the `iter` loop; only put the part you want to measure inside
- Make the code do "the same thing" on each iteration; do not accumulate or change state
- Make the outer function idempotent too; the benchmark runner is likely to run it many times
- Make the inner `iter` loop short and fast so benchmark runs are fast and the calibrator can adjust the run-length at fine resolution
- Make the code in the `iter` loop do something simple, to assist in pinpointing performance improvements (or regressions)

Gotcha: optimizations

There's another tricky part to writing benchmarks: benchmarks compiled with optimizations activated can be dramatically changed by the optimizer so that the benchmark is no longer benchmarking what one expects. For example, the compiler might recognize that some calculation has no external effects and remove it entirely.

```
#![feature(test)]

extern crate test;
use test::Bencher;

#[bench]
fn bench_xor_1000_ints(b: &mut Bencher) {
    b.iter(|| {
        (0..1000).fold(0, |old, new| old ^ new);
    });
}
```

gives the following results

```
running 1 test
test bench_xor_1000_ints ... bench:          0 ns/iter (+/- 0)

test result: ok. 0 passed; 0 failed; 0 ignored; 1 measured
```

The benchmarking runner offers two ways to avoid this. Either, the closure that the `iter` method receives can return an arbitrary value which forces the optimizer to consider the result used and ensures it cannot remove the computation entirely. This could be done for the example above by adjusting the `b.iter` call to

```
b.iter(|| {
    // Note lack of `;` (could also use an explicit `return`).
    (0..1000).fold(0, |old, new| old ^ new)
});
```

Or, the other option is to call the generic `test::black_box` function, which is an opaque "black box" to the optimizer and so forces it to consider any argument as used.

```
#![feature(test)]

extern crate test;

b.iter(|| {
    let n = test::black_box(1000);

    (0..n).fold(0, |a, b| a ^ b)
})
```

Neither of these read or modify the value, and are very cheap for small values. Larger values can be passed indirectly to reduce overhead (e.g. `black_box(&huge_struct)`).

Performing either of the above changes gives the following benchmarking results

```
running 1 test
test bench_xor_1000_ints ... bench:      131 ns/iter (+/- 3)

test result: ok. 0 passed; 0 failed; 0 ignored; 1 measured
```

However, the optimizer can still modify a testcase in an undesirable manner even when using either of the above.

thin_box

The tracking issue for this feature is: [#92791](#)

thread_id_value

The tracking issue for this feature is: [#67939](#)

thread_local_internals

This feature is internal to the Rust compiler and is not intended for general use.

thread_raw

The tracking issue for this feature is: [#97523](#)

thread_sleep_until

The tracking issue for this feature is: [#113752](#)

thread_spawn_hook

The tracking issue for this feature is: [#132951](#)

trace_macros

The tracking issue for this feature is [#29598](#).

With `trace_macros` you can trace the expansion of macros in your code.

Examples

```
#![feature(trace_macros)]

fn main() {
    trace_macros!(true);
    println!("Hello, Rust!");
    trace_macros!(false);
}
```

The `cargo build` output:

```
note: trace_macro
--> src/main.rs:5:5
5 |      println!("Hello, Rust!");
  |      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
= note: expanding `println! { "Hello, Rust!" }`
= note: to `print ! ( concat ! ( "Hello, Rust!" , "\n" ) )`
= note: expanding `print! { concat ! ( "Hello, Rust!" , "\n" ) }`
= note: to `$crate :: io :: _print ( format_args ! ( concat ! ( "Hello,
Rust!" , "\n" ) )
)`
```

```
Finished dev [unoptimized + debuginfo] target(s) in 0.60 secs
```

track_path

The tracking issue for this feature is: [#99515](#)

transmutability

The tracking issue for this feature is: [#99571](#)

trusted_fused

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

trusted_len

The tracking issue for this feature is: [#37572](#)

trusted_len_next_unchecked

The tracking issue for this feature is: [#37572](#)

trusted_random_access

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

trusted_step

The tracking issue for this feature is: [#85731](#)

try_find

The tracking issue for this feature is: [#63178](#)

try_reserve_kind

The tracking issue for this feature is: [#48043](#)

try_trait_v2

The tracking issue for this feature is: [#84277](#)

try_trait_v2_residual

The tracking issue for this feature is: [#91285](#)

try_trait_v2_yeet

The tracking issue for this feature is: [#96374](#)

try_with_capacity

The tracking issue for this feature is: [#91913](#)

tuple_trait

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

type_ascription

The tracking issue for this feature is: [#23416](#)

ub_checks

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

uefi_std

The tracking issue for this feature is: [#100499](#)

unbounded_shifts

The tracking issue for this feature is: [#129375](#)

unchecked_neg

The tracking issue for this feature is: [#85122](#)

unchecked_shifts

The tracking issue for this feature is: [#85122](#)

unicode_internals

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

unique_rc_arc

The tracking issue for this feature is: [#112566](#)

unix_file_vectored_at

The tracking issue for this feature is: [#89517](#)

unix_set_mark

The tracking issue for this feature is: [#96467](#)

unix_socket_ancillary_data

The tracking issue for this feature is: [#76915](#)

unix_socket_peek

The tracking issue for this feature is: [#76923](#)

unsafe_cell_access

The tracking issue for this feature is: [#136327](#)

unsafe_pin_internals

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.

unsigned_is_multiple_of

The tracking issue for this feature is: [#128101](#)

unsigned_nonzero_div_ceil

The tracking issue for this feature is: [#132968](#)

unsigned_signed_diff

The tracking issue for this feature is: [#126041](#)

unsize

The tracking issue for this feature is: [#18598](#)

unwrap_infallible

The tracking issue for this feature is: [#61695](#)

update_panic_count

This feature is internal to the Rust compiler and is not intended for general use.

utf16_extra

The tracking issue for this feature is: [#94919](#)

variant_count

The tracking issue for this feature is: [#73662](#)

vec_deque_iter_as_slices

The tracking issue for this feature is: [#123947](#)

vec_deque_pop_if

The tracking issue for this feature is: [#135889](#)

vec_into_raw_parts

The tracking issue for this feature is: [#65816](#)

vec_push_within_capacity

The tracking issue for this feature is: [#100486](#)

vec_split_at_spare

The tracking issue for this feature is: [#81944](#)

wasi_ext

The tracking issue for this feature is: [#71213](#)

windows_by_handle

The tracking issue for this feature is: [#63010](#)

windows_c

This feature is internal to the Rust compiler and is not intended for general use.

windows_change_time

The tracking issue for this feature is: [#121478](#)

windows_handle

This feature is internal to the Rust compiler and is not intended for general use.

windows_net

This feature is internal to the Rust compiler and is not intended for general use.

windows_process_exit_code_from

The tracking issue for this feature is: [#111688](#)

windows_process_extensions_async_pipes

The tracking issue for this feature is: [#98289](#)

windows_process_extensions_force_quotes

The tracking issue for this feature is: [#82227](#)

windows_process_extensions_main_thread_handle

The tracking issue for this feature is: [#96723](#)

windows_process_extensions_raw_tribute

The tracking issue for this feature is: [#114854](#)

windows_process_extensions_show_window

The tracking issue for this feature is: [#127544](#)

windows_stdio

This feature is internal to the Rust compiler and is not intended for general use.

wrapping_int_impl

The tracking issue for this feature is: [#32463](#)

wrapping_next_power_of_two

The tracking issue for this feature is: [#32463](#)

write_all_vectored

The tracking issue for this feature is: [#70436](#)

yeet_desugar_details

This feature has no tracking issue, and is therefore likely internal to the compiler, not being intended for general use.
